

Thomas Project Codebase



Generated by Antigravity AI Assistant

■ File: index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>■■■■■■■■■■</title>
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

■ File: src\App.tsx

```
import React from 'react';
import { Routes, Route, Navigate, useNavigate } from 'react-router-dom';
import { LoginPage } from '../components/LoginPage';
import { PatientDashboard } from '../components/PatientDashboard';
import { DoctorDashboard } from '../components/DoctorDashboard';
import { CaseManagerDashboard } from '../components/CaseManagerDashboard';
import { AboutPage } from '../components/AboutPage';
import { useAuthStore } from '../store/useAuthStore';

export default function App() {
  const { currentUser, login, logout } = useAuthStore();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/login');
  };

  const navigateTo = (page: string) => {
    if (page === 'dashboard') {
      navigate(currentUser ? `/${currentUser.role}` : '/login');
    } else {
      navigate(`/${page}`);
    }
  };

  const ProtectedRoute = ({ children, role }: { children: React.ReactNode, role: string }) => {
    if (!currentUser) return <Navigate to="/login" replace />;
    if (currentUser.role !== role) return <Navigate to={`/${currentUser.role}`} replace />;
    return <&gt;{children}&lt;/>;
  };

  return (
    <Routes>
      <Route path="/login" element={
        !currentUser ? <LoginPage onLogin={({user}) => {
          login(user);
          navigate(`/${user.role}`);
        }} /> : <Navigate to={`/${currentUser.role}`} replace />
      } />

      <Route path="/patient" element={
        <ProtectedRoute role="patient">
          <PatientDashboard user={currentUser!} onLogout={handleLogout}
        </ProtectedRoute>
      } />

      <Route path="/doctor" element={
        <ProtectedRoute role="doctor">
          <DoctorDashboard user={currentUser!} onLogout={handleLogout}
        </ProtectedRoute>
      } />

      <Route path="/caseManager" element={
        <ProtectedRoute role="caseManager">
          <CaseManagerDashboard user={currentUser!} onLogout={handleLogout}
        </ProtectedRoute>
      } />

      <Route path="/about" element={
        !currentUser ? <Navigate to="/login" replace /> : <AboutPage
      </Route>
      <Route path="/" element={<Navigate to={currentUser ? `/${currentUser.role}` : "/login"} replace />} />
    </Routes>
  );
}
```

■ File: src\components\AboutPage.tsx

```

import { Activity, ArrowLeft, Heart, Users, Zap } from 'lucide-react';

interface AboutPageProps {
  onNavigate: (page: 'dashboard' | 'about') => void;
  onLogout: () => void;
}

export function AboutPage({ onNavigate, onLogout }: AboutPageProps) {
  return (
    <div className="min-h-screen bg-background">
      { /* Header */ }
      <header className="bg-white border-b border-border shadow-sm sticky top-0 z-50">
        <div className="max-w-full mx-auto px-4">
          <div className="flex items-center justify-between h-16">
            <div className="flex items-center gap-3">
              <div className="w-10 h-10 bg-primary rounded-lg flex items-center justify-center">
                <Activity className="w-6 h-6 text-white" />
              </div>
              <h3 className="text-primary">■■■■■■■■</h3>
            </div>
            <div className="flex gap-3">
              <button
                onClick={() => onNavigate('dashboard')}
                className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg hover:bg-gray-50 transition-colors">
                <ArrowLeft className="w-5 h-5" />
                ■■
              </button>
            </div>
          </div>
        </div>
      </header>

      <div className="max-w-full mx-auto px-4 py-12 pb-4">
        { /* Hero Section */ }
        <div className="text-center mb-16">
          <div className="inline-flex items-center justify-center w-20 h-20 bg-primary rounded-full mb-6 shadow-lg">
            <Activity className="w-10 h-10 text-white" />
          </div>
          <h1 className="text-text-primary mb-4">■■■■■■■■■■</h1>
        </div>

        { /* Features */ }
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-16">
          <div className="bg-white rounded-xl shadow-md p-6 border border-border text-center">
            <div className="w-16 h-16 bg-blue-100 rounded-full flex items-center justify-center mx-auto mb-4">
              <Heart className="w-8 h-8 text-primary" />
            </div>
            <h3 className="text-text-primary mb-2">■■■■■■■■</h3>
            <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
          </div>

          <div className="bg-white rounded-xl shadow-md p-6 border border-border text-center">
            <div className="w-16 h-16 bg-cyan-100 rounded-full flex items-center justify-center mx-auto mb-4">
              <Zap className="w-8 h-8 text-secondary" />
            </div>
            <h3 className="text-text-primary mb-2">■■■■</h3>
            <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
          </div>

          <div className="bg-white rounded-xl shadow-md p-6 border border-border text-center">
            <div className="w-16 h-16 bg-green-100 rounded-full flex items-center justify-center mx-auto mb-4">
              <Users className="w-8 h-8 text-success" />
            </div>
            <h3 className="text-text-primary mb-2">■■■■■■■■</h3>
            <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
          </div>

          <div className="bg-white rounded-xl shadow-md p-6 border border-border text-center">
            <div className="w-16 h-16 bg-gray-100 rounded-full flex items-center justify-center mx-auto mb-4">
              <Activity className="w-8 h-8 text-primary" />
            </div>
            <h3 className="text-text-primary mb-2">■■■■■■■■</h3>
            <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
          </div>
        </div>
      </div>
    </div>
  );
}

```

```

        <div className="w-16 h-16 bg-purple-100 rounded-full flex items-center
justify-center mx-auto mb-4">
            <Shield className="w-8 h-8 text-purple-600" />
        </div>
        <h3 className="text-text-primary mb-2">■■■■</h3>
        <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
    </div>
    </div>

    { /* System Features */ }
    <div className="bg-white rounded-xl shadow-lg p-8 border border-border mb-16">
        <h2 className="text-text-primary mb-8 text-center">■■■■</h2>
        <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
            <div>
                <h3 className="text-primary mb-4">■■■■</h3>
                <ul className="space-y-2 text-text-secondary">
                    <li>
                        <div className="w-2 h-2 bg-primary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-primary rounded-full mt-2"></div>
                        <span>8 ■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-primary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-primary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                </ul>
            </div>

            <div>
                <h3 className="text-secondary mb-4">■■■■</h3>
                <ul className="space-y-2 text-text-secondary">
                    <li>
                        <div className="w-2 h-2 bg-secondary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-secondary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-secondary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-secondary rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                </ul>
            </div>

            <div>
                <h3 className="text-success mb-4">■■■■</h3>
                <ul className="space-y-2 text-text-secondary">
                    <li>
                        <div className="w-2 h-2 bg-success rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-success rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-success rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                    <li>
                        <div className="w-2 h-2 bg-success rounded-full mt-2"></div>
                        <span>■■■■■■■■■■</span>
                    </li>
                </ul>
            </div>
        </div>
    </div>

    { /* Questionnaires */ }

```


■ File: src\components\CaseManagerDashboard.tsx

```
import { useState, useEffect } from 'react';
import { User } from '../store/useAuthStore';
import { Header } from '../Header';
import { API_BASE_URL } from '../config';
import { AlertTriangle, Calendar, Phone, Mail, CheckCircle, Clock, Bell } from
  'lucide-react';

interface CaseManagerDashboardProps {
  user: User;
  onLogout: () => void;
  onNavigate: (page: 'dashboard' | 'about') => void;
}

interface HighRiskPatient {
  id: string;
  name: string;
  age: number;
  phone: string;
  email: string;
  midasScore: number;
  headacheDays: number;
  lastVisit: string;
  nextAppointment: string | null;
  riskFactors: string[];
  status: 'pending' | 'contacted' | 'scheduled';
  notes?: string;
}

const mockHighRiskPatients: HighRiskPatient[] = [
  {
    id: '1',
    name: '王',
    age: 35,
    phone: '0912-345-678',
    email: 'wang@example.com',
    midasScore: 24,
    headacheDays: 15,
    lastVisit: '2024-11-20',
    nextAppointment: null,
    riskFactors: ['MIDAS Grade IV', '高血压', '糖尿病'],
    status: 'pending',
  },
  {
    id: '2',
    name: '张',
    age: 42,
    phone: '0923-456-789',
    email: 'zhang@example.com',
    midasScore: 22,
    headacheDays: 15,
    lastVisit: '2024-11-18',
    nextAppointment: '2024-12-05',
    riskFactors: ['MIDAS Grade IV', '高血压'],
    status: 'scheduled',
  },
  {
    id: '3',
    name: '李',
    age: 38,
    phone: '0934-567-890',
    email: 'li@example.com',
    midasScore: 19,
    headacheDays: 12,
    lastVisit: '2024-11-15',
    nextAppointment: null,
    riskFactors: ['高血压', '糖尿病'],
    status: 'contacted',
  },
  {
    id: '4',
    name: '陈',
    age: 45,
    phone: '0945-678-901',
    email: 'chen@example.com',
    midasScore: 21,
    headacheDays: 14,
    lastVisit: '2024-11-10',
    nextAppointment: null,
    riskFactors: ['MIDAS Grade IV', '高血压'],
  },
];
```

```

    status: 'pending',
  },
];

export function CaseManagerDashboard({ user, onLogout, onNavigate }:
  CaseManagerDashboardProps) {
  const [patients, setPatients] = useState<HighRiskPatient[]>([]);
  const [selectedPatient, setSelectedPatient] = useState<HighRiskPatient | null>(null);
  const [notes, setNotes] = useState('');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [showDatePicker, setShowDatePicker] = useState(false);

  const fetchPatients = async () => {
    try {
      setLoading(true);
      const response = await fetch(`${API_BASE_URL}/api/casemanager/patients`);
      if (response.ok) {
        const data = await response.json();
        setPatients(data);
        setError(null);
      } else {
        setError('■■■■■■■■■■');
      }
    } catch (err) {
      console.error(err);
      setError('■■■■■■■■■■ Spring Boot ■■■■■■');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchPatients();
  }, []);

  useEffect(() => {
    if (selectedPatient) {
      setNotes(selectedPatient.notes || '');
    } else {
      setNotes('');
    }
    setShowDatePicker(false);
  }, [selectedPatient]);

  const handleStatusChange = async (patientId: string, newStatus: 'pending' | 'contacted'
    | 'scheduled') => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/casemanager/tracking/${patientId}`
        , {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ status: newStatus }),
        });
      if (response.ok) {
        setPatients((prev) => {
          prev.map((p) => (p.id === patientId ? { ...p, status: newStatus } : p))
        });
        if (selectedPatient && selectedPatient.id === patientId) {
          setSelectedPatient((prev) => prev ? { ...prev, status: newStatus } : null);
        }
      } else {
        alert('■■■■■■');
      }
    } catch (err) {
      console.error(err);
      alert('■■■■■■■■■■');
    }
  };

  const handleScheduleAppointment = async (patientId: string, date: string) => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/casemanager/tracking/${patientId}`
        , {
          method: 'PUT',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ nextAppointment: date, status: 'scheduled' }),
        });
      if (response.ok) {
        setPatients((prev) => {
          prev.map((p) => {
            p.id === patientId ? { ...p, nextAppointment: date, status: 'scheduled' } : p
          })
        });
      }
    }
  };

```



```

    )
  );
  if (selectedPatient && selectedPatient.id === patientId) {
    setSelectedPatient((prev) => {
      prev ? { ...prev, nextAppointment: date, status: 'scheduled' } : null
    });
  }
  alert('■■■■■■■■■■');
} else {
  alert('■■■■■■■■■■');
}
} catch (err) {
  console.error(err);
  alert('■■■■■■■■■■■■■■■■■■■■');
}
};

const handleSaveNotes = async () => {
  if (!selectedPatient) return;
  try {
    const response = await fetch(`${API_BASE_URL}/api/casemanager/tracking/${selectedPatient.id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ notes })
    });
    if (response.ok) {
      setPatients((prev) => {
        prev.map((p) => (p.id === selectedPatient.id ? { ...p, notes } : p))
      });
      setSelectedPatient((prev) => { prev ? { ...prev, notes } : null });
      alert('■■■■■■■■■■');
    } else {
      alert('■■■■■■■■■■');
    }
  } catch (err) {
    console.error(err);
    alert('■■■■■■■■■■■■■■■■■■■■');
  }
};

const getStatusColor = (status: string) => {
  switch (status) {
    case 'pending':
      return 'text-danger bg-red-100';
    case 'contacted':
      return 'text-warning bg-yellow-100';
    case 'scheduled':
      return 'text-success bg-green-100';
    default:
      return 'text-text-secondary bg-gray-100';
  }
};

const getStatusLabel = (status: string) => {
  switch (status) {
    case 'pending':
      return '■■■■';
    case 'contacted':
      return '■■■■';
    case 'scheduled':
      return '■■■■';
    default:
      return '■■';
  }
};

const pendingCount = patients.filter((p) => p.status === 'pending').length;
const contactedCount = patients.filter((p) => p.status === 'contacted').length;
const scheduledCount = patients.filter((p) => p.status === 'scheduled').length;

return (
  <div className="min-h-screen bg-background">
    <Header user={user} onLogout={onLogout} onNavigate={onNavigate} />

    <div className="max-w-full mx-auto px-4 py-8">
      <div className="mb-8">
        <h1 className="text-text-primary mb-2">■■■■■■■■■■</h1>
        <p className="text-text-secondary">■■■■■■■■■■■■■■■■■■■■</p>
      </div>
      { /* Summary Cards */ }
    </div>
  </div>

```



```

        onClick={handleSaveNotes}
        className="w-full mt-3 px-4 py-2 bg-success hover:bg-green-600
text-white rounded-lg transition-all"
      &gt;
        ■■■■
      &lt;/button&gt;
    &lt;/div&gt;
  &lt;/div&gt;
) : (
  &lt;div className="bg-white rounded-xl shadow-md p-6 border border-border flex
items-center justify-center h-96"&gt;
    &lt;div className="text-center text-text-secondary"&gt;
      &lt;AlertTriangle className="w-16 h-16 mx-auto mb-4 opacity-50" /&gt;
      &lt;p&gt;■■■■■■■■■■&lt;/p&gt;
    &lt;/div&gt;
  &lt;/div&gt;
)}
&lt;/div&gt;
&lt;/div&gt;
&lt;/div&gt;
);
}

```

■ File: src\components\DoctorDashboard.tsx

```
import { useState, useEffect } from 'react';
import { User } from '../store/useAuthStore';
import { Header } from '../Header';
import { API_BASE_URL } from '../config';
import { PatientRecordModal } from './PatientRecordModal';
import { PrescriptionModal } from './PrescriptionModal';
import { Users, TrendingUp, FileText, Search } from 'lucide-react';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend,
  ResponsiveContainer, BarChart, Bar } from 'recharts';
import { toast } from 'sonner';

interface DoctorDashboardProps {
  user: User;
  onLogout: () => void;
  onNavigate: (page: 'dashboard' | 'about') => void;
}

interface Patient {
  id: string;
  name: string;
  age: number;
  gender: string;
  lastVisit: string;
  midasScore: number;
  headacheDays: number;
  riskLevel: 'low' | 'medium' | 'high';
}

const patientTrendData = [
  { month: '6', avgMidas: 14, avgHeadacheDays: 9 },
  { month: '7', avgMidas: 15, avgHeadacheDays: 10 },
  { month: '8', avgMidas: 13, avgHeadacheDays: 8 },
  { month: '9', avgMidas: 16, avgHeadacheDays: 11 },
  { month: '10', avgMidas: 14, avgHeadacheDays: 9 },
  { month: '11', avgMidas: 15, avgHeadacheDays: 10 },
];

export function DoctorDashboard({ user, onLogout, onNavigate }: DoctorDashboardProps) {
  const [patients, setPatients] = useState<Patient[]>([]);
  const [selectedPatient, setSelectedPatient] = useState<Patient | null>(null);
  const [searchTerm, setSearchTerm] = useState('');
  const [showRecordModal, setShowRecordModal] = useState(false);
  const [showPrescriptionModal, setShowPrescriptionModal] = useState(false);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    fetch(`${API_BASE_URL}/api/users/patients`)
      .then(res => res.json())
      .then(data => setPatients(data))
      .catch(err => console.error("Failed to fetch patients", err))
      .finally(() => setIsLoading(false));
  }, []);

  const filteredPatients = patients.filter((patient) =>
    patient.name.toLowerCase().includes(searchTerm.toLowerCase())
  );

  const totalPatientsCount = patients.length;
  const totalCompletedQ = patients.reduce((sum, p: any) => sum + (p.completedCount || 0),
    0);
  const totalExpectedQ = totalPatientsCount * 8;
  const completionRate = totalExpectedQ > 0
    ? Math.round((totalCompletedQ / totalExpectedQ) * 100)
    : 0;

  const getRiskColor = (level: string) => {
    switch (level) {
      case 'high':
        return 'text-danger bg-red-100';
      case 'medium':
        return 'text-warning bg-yellow-100';
      case 'low':
        return 'text-success bg-green-100';
      default:
        return 'text-text-secondary bg-gray-100';
    }
  };

  const getRiskLabel = (level: string) => {
```

```

switch (level) {
  case 'high':
    return '■■■■';
  case 'medium':
    return '■■■■';
  case 'low':
    return '■■■■';
  default:
    return '■■■■';
}
};

const patientDetailData = selectedPatient
? [
  { date: '11/18', intensity: 7, duration: 5 },
  { date: '11/20', intensity: 8, duration: 6 },
  { date: '11/22', intensity: 6, duration: 4 },
  { date: '11/24', intensity: 7, duration: 5 },
  { date: '11/25', intensity: 5, duration: 3 },
]
: [];

const handleViewRecord = () => {
  if (selectedPatient) {
    setShowRecordModal(true);
  }
};

const handleOpenPrescription = () => {
  if (selectedPatient) {
    setShowPrescriptionModal(true);
  }
};

const handlePrescriptionSubmit = async (prescription: any) => {
  if (!selectedPatient || !user) return;
  try {
    await fetch(`${API_BASE_URL}/api/prescriptions`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        doctorId: user.id,
        patientId: selectedPatient.id,
        medicationDetails: prescription.medication,
        instructions: prescription.notes || prescription.instructions
      })
    });
    toast.success('■■■■■■■■■■■■■■■■■■■■');
  } catch (err) {
    console.error("Save prescription failed", err);
    toast.error('■■■■■■■■■■■■■■■■■■■■');
  }
  setShowPrescriptionModal(false);
};

return (
  <div className="min-h-screen bg-background">
    <Header user={user} onLogout={onLogout} onNavigate={onNavigate} />

    <div className="max-w-full mx-auto px-4 py-8">
      <div className="mb-8">
        <h1 className="text-text-primary mb-2">■■■■</h1>
        <p className="text-text-secondary">■■■■■■■■■■</p>
      </div>

      { /* Stats Cards */ }
      <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-8">
        <div className="bg-white rounded-xl shadow-md p-6 border border-border">
          <div className="flex items-center justify-between mb-2">
            <div className="w-12 h-12 bg-blue-100 rounded-lg flex items-center justify-center">
              <Users className="w-6 h-6 text-primary" />
            </div>
            <div>
              <div className="text-text-secondary">■■■■■■■■■■</div>
              <h3 className="text-primary">{patients.length} ■■■■■</h3>
            </div>
          </div>

          <div className="bg-white rounded-xl shadow-md p-6 border border-border">
            <div className="flex items-center justify-between mb-2">
              <div className="w-12 h-12 bg-red-100 rounded-lg flex items-center justify-center">

```

```

        <TrendingUp className="w-6 h-6 text-danger" />
      </div>
    </div>
    <div className="text-text-secondary">■■■■■ (<{<}<'<}<10■■■■</div>
    <h3 className="text-danger">{patients.filter(p =< p.riskLevel ===
'high').length} ■</h3>
  </div>

  <div className="bg-white rounded-xl shadow-md p-6 border border-border">
    <div className="flex items-center justify-between mb-2">
      <div className="w-12 h-12 bg-green-100 rounded-lg flex items-center
justify-center">
        <FileText className="w-6 h-6 text-success" />
      </div>
    </div>
    <div className="text-text-secondary">■■■■■■■</div>
    <h3 className="text-success">{totalPatientsCount < 0 ? `<{completionRate}%` :
'0%'}</h3>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
    {/ Patient List */}
    <div className="bg-white rounded-xl shadow-md p-6 border border-border">
      <div className="mb-4">
        <h3 className="text-text-primary mb-4">■■■■</h3>
        <div className="relative">
          <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-5 h-5
text-text-secondary" />
          <input
            type="text"
            value={searchTerm}
            onChange={(e) =< setSearchTerm(e.target.value)}
            placeholder="■■■■■■■■■■"
            className="w-full pl-10 pr-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          />
        </div>
      </div>
      <div className="space-y-3 max-h-96 overflow-y-auto">
        {isLoading ? (
          <div className="text-center text-text-secondary py-8">■■■■■■■■■■...</div>
        ) : filteredPatients.length === 0 ? (
          <div className="text-center text-text-secondary py-8">■■■■■■■■■■</div>
        ) : (
          filteredPatients.map((patient) =< (
            <button
              key={patient.id}
              onClick={() =< setSelectedPatient(patient)}
              className={`w-full text-left p-4 rounded-lg border-2 transition-all <{
                selectedPatient?.id === patient.id
                  ? 'bg-medical-blue border-primary'
                  : 'bg-white border-border hover:border-primary'
              }`}
            >
              <div className="flex items-center justify-between mb-2">
                <div>
                  <h4 className="text-text-primary">{patient.name}</h4>
                  <p className="text-text-secondary">
                    {patient.age} ■ · {patient.gender}
                  </p>
                </div>
                <span className={`px-3 py-1 rounded-full
${getRiskColor(patient.riskLevel)}`>
                  {getRiskLabel(patient.riskLevel)}
                </span>
              </div>
                <div className="flex items-center gap-4 text-text-secondary">
                  <span>MIDAS: {patient.midasScore}</span>
                  <span>■■■■: {patient.headacheDays}</span>
                </div>
                <div className="text-text-secondary mt-2">
                  ■■■■: {patient.lastVisit}
                </div>
              </button>
            </div>
          ))
        )}
      </div>
    </div>
  </div>
  {/ Patient Detail or Overall Trend */}

```



```

        <Bar dataKey="avgHeadacheDays" fill="#06b6d4" name="■■■■■■" />
      </BarChart>
    </ResponsiveContainer>
  </>
)
</div>
</div>
</div>

{/* Patient Record Modal */}
{showRecordModal && selectedPatient && (
  <PatientRecordModal
    patient={selectedPatient}
    onClose={() => setShowRecordModal(false)}
  />
)}

{/* Prescription Modal */}
{showPrescriptionModal && selectedPatient && (
  <PrescriptionModal
    patient={selectedPatient}
    onClose={() => setShowPrescriptionModal(false)}
    onSubmit={handlePrescriptionSubmit}
  />
)}
</div>
);
}

```

■ File: src\components\HeadacheDiary.tsx

```
import { useState, useEffect } from 'react';
import { Plus, Calendar, Clock, Activity, Edit2, Trash2 } from 'lucide-react';
import { useAuthStore } from '../store/useAuthStore';
import { API_BASE_URL } from '../config';

interface DiaryEntry {
  id?: string;
  userId?: string;
  date: string;
  time: string;
  intensity: number;
  symptoms: string[];
  medication: string;
  notes: string;
}

export function HeadacheDiary() {
  const currentUser = useAuthStore((state) => state.currentUser);
  const [entries, setEntries] = useState<DiaryEntry[]>([]);
  const [isLoading, setIsLoading] = useState(false);

  useEffect(() => {
    if (currentUser?.id) {
      setIsLoading(true);
      fetch(`${API_BASE_URL}/api/diaries/user/${currentUser.id}`)
        .then(res => res.json())
        .then(data => setEntries(data))
        .catch(err => console.error("Failed to fetch diaries:", err))
        .finally(() => setIsLoading(false));
    }
  }, [currentUser]);

  const [showForm, setShowForm] = useState(false);
  const [editingId, setEditingId] = useState<string | null>(null);
  const [formData, setFormData] = useState({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toLocaleTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [] as string[],
    medication: '',
    notes: '',
  });

  const symptomOptions = [
    '■■■■■',
    '■■■■■',
    '■■■',
    '■■■',
    '■■■',
    '■■■',
    '■■■',
    '■■■',
    '■■■■■',
  ];

  const handleSymptomToggle = (symptom: string) => {
    setFormData((prev) => ({
      ...prev,
      symptoms: prev.symptoms.includes(symptom)
        ? prev.symptoms.filter((s) => s !== symptom)
        : [...prev.symptoms, symptom],
    }));
  };

  const handleEdit = (entry: DiaryEntry) => {
    setEditingId(entry.id || null);
    setFormData({
      date: entry.date,
      time: entry.time,
      intensity: entry.intensity,
      symptoms: entry.symptoms,
      medication: entry.medication,
      notes: entry.notes,
    });
    setShowForm(true);
  };

  const handleDelete = async (id: string | undefined) => {
    if (!id) return;
    if (confirm('■■■■■■■■■■')) {

```

```

    try {
      await fetch(`${API_BASE_URL}/api/diaries/${id}`, { method: 'DELETE' });
      setEntries(entries.filter(entry => entry.id !== id));
    } catch (err) {
      console.error("Failed to delete", err);
    }
  }
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!currentUser) return;

  const entryData: DiaryEntry = {
    ...formData,
    userId: currentUser.id,
  };

  if (editingId) {
    entryData.id = editingId;
  }

  try {
    const res = await fetch(`${API_BASE_URL}/api/diaries`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(entryData)
    });
    const savedEntry = await res.json();

    if (editingId) {
      setEntries(entries.map(entry => entry.id === editingId ? savedEntry : entry));
    } else {
      setEntries([savedEntry, ...entries]);
    }
  } catch (err) {
    console.error("Failed to save", err);
  }

  setShowForm(false);
  setEditingId(null);
  setFormData({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toLocaleTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [],
    medication: '',
    notes: '',
  });
};

const handleCancelForm = () => {
  setShowForm(false);
  setEditingId(null);
  setFormData({
    date: new Date().toISOString().split('T')[0],
    time: new Date().toLocaleTimeString().slice(0, 5),
    intensity: 5,
    symptoms: [],
    medication: '',
    notes: '',
  });
};

return (
  <div className="space-y-6">
    <div>
      <div className="flex justify-between items-center">
        <div>
          <h2 className="text-text-primary">
            <p className="text-text-secondary">
          </div>
          <button
            onClick={() => setShowForm(!showForm)}
            className="flex items-center gap-2 px-4 py-2 bg-primary hover:bg-primary-dark
            text-white rounded-lg transition-all shadow-md hover:shadow-lg"
          >
            <Plus className="w-5 h-5" />
          </button>
        </div>
      </div>
    </div>
  </div>
);

```

```

{/* Entry Form */}
{showForm &&& (
  &lt;div className="bg-white rounded-xl shadow-lg p-6 border border-border"&gt;
    &lt;h3 className="mb-6 text-text-primary"&gt;
      {editingId ? '■■■■■■■' : '■■■■■■■'}
    &lt;/h3&gt;
    &lt;form onSubmit={handleSubmit} className="space-y-6"&gt;
      &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-6"&gt;
        { /* Date */ }
        &lt;div&gt;
          &lt;label className="block mb-2"&gt;■■■&lt;/label&gt;
          &lt;input
            type="date"
            value={formData.date}
            onChange={(e) => setFormData({ ...formData, date: e.target.value })}
            className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
            required
          /&gt;
        &lt;/div&gt;

        { /* Time */ }
        &lt;div&gt;
          &lt;label className="block mb-2"&gt;■■■&lt;/label&gt;
          &lt;input
            type="time"
            value={formData.time}
            onChange={(e) => setFormData({ ...formData, time: e.target.value })}
            className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
            required
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      { /* Intensity */ }
      &lt;div&gt;
        &lt;label className="block mb-2"&gt;■■■■■{formData.intensity} / 10&lt;/label&gt;
        &lt;input
          type="range"
          min="1"
          max="10"
          value={formData.intensity}
          onChange={(e) => setFormData({ ...formData, intensity:
parseInt(e.target.value) }}
          className="w-full h-2 bg-gray-200 rounded-lg appearance-none
cursor-pointer accent-primary"
        /&gt;
        &lt;div className="flex justify-between text-text-secondary mt-1"&gt;
          &lt;span&gt;■■■&lt;/span&gt;
          &lt;span&gt;■■■&lt;/span&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      { /* Symptoms */ }
      &lt;div&gt;
        &lt;label className="block mb-3"&gt;■■■■■■■&lt;/label&gt;
        &lt;div className="grid grid-cols-2 md:grid-cols-4 gap-3"&gt;
          {symptomOptions.map((symptom) => (
            &lt;button
              key={symptom}
              type="button"
              onClick={() => handleSymptomToggle(symptom)}
              className={`p-3 rounded-lg border-2 transition-all ${
                formData.symptoms.includes(symptom)
                  ? 'bg-primary text-white border-primary'
                  : 'bg-white text-text-secondary border-border
hover:border-primary'
              }`}
            &gt;
              {symptom}
            &lt;/button&gt;
          ))}
        &lt;/div&gt;
      &lt;/div&gt;

      { /* Medication */ }
      &lt;div&gt;
        &lt;label className="block mb-2"&gt;■■■&lt;/label&gt;
        &lt;input
          type="text"
          value={formData.medication}

```

[illegible]

```

        {symptom}
      </span>
    ))
  </div>
</div>

{entry.medication && (
  <div>
    <span className="text-text-secondary">■■■■</span>
    <span className="ml-2 text-text-primary">{entry.medication}</span>
  </div>
)}

{entry.notes && (
  <div>
    <span className="text-text-secondary">■■■■</span>
    <span className="ml-2 text-text-primary">{entry.notes}</span>
  </div>
)}
</div>

{/* Action Buttons */}
<div className="flex gap-2 mt-4 pt-4 border-t border-border">
  <button
    onClick={() => handleEdit(entry)}
    className="flex items-center gap-2 px-4 py-2 text-primary
hover:bg-medical-blue rounded-lg transition-colors"
  >
    <Edit2 className="w-4 h-4" />
    ■■
  </button>
  <button
    onClick={() => handleDelete(entry.id)}
    className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
rounded-lg transition-colors"
  >
    <Trash2 className="w-4 h-4" />
    ■■
  </button>
</div>
</div>
)}
</div>
);
}

```

■ File: src\components\Header.tsx

```
import { User } from '../App';
import { Activity, Logout, Info, Menu, X } from 'lucide-react';
import { useState } from 'react';

interface HeaderProps {
  user: User;
  onLogout: () => void;
  onNavigate: (page: 'dashboard' | 'about') => void;
}

export function Header({ user, onLogout, onNavigate }: HeaderProps) {
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);

  const getRoleLabel = (role: string) => {
    switch (role) {
      case 'patient':
        return '■■';
      case 'doctor':
        return '■■';
      case 'caseManager':
        return '■■■■■';
      default:
        return '';
    }
  };

  return (
    <header className="bg-white border-b border-border shadow-sm sticky top-0 z-50">
      <div className="max-w-full mx-auto px-4">
        <div className="flex items-center justify-between h-16">
          {/* Logo */}
          <div className="flex items-center gap-3 cursor-pointer" onClick={() =>
            onNavigate('dashboard')}>
            <div className="w-10 h-10 bg-primary rounded-lg flex items-center
              justify-center">
              <Activity className="w-6 h-6 text-white" />
            </div>
            <div className="hidden sm:block">
              <h3 className="text-primary">■■■■■■■</h3>
            </div>
          </div>

          {/* Desktop Navigation */}
          <div className="hidden md:flex items-center gap-6">
            <button
              onClick={() => onNavigate('about')}
              className="flex items-center gap-2 text-text-secondary hover:text-primary
                transition-colors">
              <Info className="w-5 h-5" />
              ■■■■
            </button>

            <div className="flex items-center gap-3 px-4 py-2 bg-medical-blue rounded-lg">
              <div className="w-8 h-8 bg-primary rounded-full flex items-center
                justify-center text-white">
                {user.name.charAt(0).toUpperCase()}
              </div>
              <div className="text-left">
                <div className="text-text-primary">{user.name}</div>
                <div className="text-text-secondary">{getRoleLabel(user.role || '')}</div>
              </div>
            </div>

            <button
              onClick={onLogout}
              className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
                rounded-lg transition-colors">
              <Logout className="w-5 h-5" />
              ■■
            </button>
          </div>

          {/* Mobile Menu Button */}
          <button
            onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
            className="md:hidden p-2 text-text-secondary hover:text-primary">
          </button>
        </div>
      </div>
    </header>
  );
}
```



```

        {mobileMenuOpen ? &lt;X className="w-6 h-6" /&gt; : &lt;Menu className="w-6 h-6" /&gt;}
    &lt;/button&gt;
    &lt;/div&gt;

    {/ * Mobile Menu */}
    {mobileMenuOpen &amp;&amp; (
        &lt;div className="md:hidden py-4 border-t border-border"&gt;
            &lt;div className="flex flex-col gap-4"&gt;
                &lt;div className="flex items-center gap-3 px-4 py-2 bg-medical-blue
rounded-lg"&gt;
                    &lt;div className="w-8 h-8 bg-primary rounded-full flex items-center
justify-center text-white"&gt;
                        {user.name.charAt(0).toUpperCase()}
                    &lt;/div&gt;
                    &lt;div&gt;
                        &lt;div className="text-text-primary"&gt;{user.name}&lt;/div&gt;
                        &lt;div className="text-text-secondary"&gt;{getRoleLabel(user.role ||
'' )}&lt;/div&gt;
                    &lt;/div&gt;
                    &lt;/div&gt;

                    &lt;button
                        onClick={() =&gt; {
                            onNavigate('about');
                            setMobileMenuOpen(false);
                        }}
                        className="flex items-center gap-2 px-4 py-2 text-text-secondary
hover:text-primary transition-colors"
                    &gt;
                        &lt;Info className="w-5 h-5" /&gt;
                        ■■■■
                    &lt;/button&gt;

                    &lt;button
                        onClick={onLogout}
                        className="flex items-center gap-2 px-4 py-2 text-danger hover:bg-red-50
rounded-lg transition-colors"
                    &gt;
                        &lt;Logout className="w-5 h-5" /&gt;
                        ■■
                    &lt;/button&gt;
                &lt;/div&gt;
            &lt;/div&gt;
        )}
    &lt;/div&gt;
    &lt;/header&gt;
    );
}

```

■ **File:** src\components\LoginPage.tsx

```
import { useState } from 'react';
import { User, UserRole } from '../store/useAuthStore';
import { Activity } from 'lucide-react';
import { API_BASE_URL } from '../config';

interface LoginPageProps {
  onLogin: (user: User) => void;
}

export function LoginPage({ onLogin }: LoginPageProps) {
  const [activeTab, setActiveTab] = useState<'login' | 'register'>('login');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  // Register form states
  const [registerName, setRegisterName] = useState('');
  const [registerEmail, setRegisterEmail] = useState('');
  const [registerPassword, setRegisterPassword] = useState('');
  const [registerRole, setRegisterRole] = useState<UserRole>('patient');

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    if (email && password) {
      try {
        // Email
        const loginAccount = email.toLowerCase().trim();

        const response = await fetch(`${API_BASE_URL}/api/auth/login`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ name: loginAccount, password }),
        });

        if (response.ok) {
          const userData = await response.json();
          onLogin(userData);
        } else {
          alert('patient1 / 1234 doctor1 / 1234');
        }
      } catch (err) {
        console.error(err);
        alert('Java Spring Boot');
      }
    }
  };

  const handleRegister = async (e: React.FormEvent) => {
    e.preventDefault();
    if (registerName && registerEmail && registerPassword && registerRole) {
      try {
        // Email
        const response = await fetch(`${API_BASE_URL}/api/auth/register`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            name: registerName.trim(),
            email: registerEmail.toLowerCase().trim(),
            password: registerPassword,
            role: registerRole,
          }),
        });

        if (response.ok) {
          const userData = await response.json();
          alert('Email${registerEmail.toLowerCase().trim()}\n');
          onLogin(userData);
        } else {
          const errorData = await response.json();
          alert(`${errorData.message} | `);
        }
      } catch (err) {
        console.error(err);
        alert('Java Spring Boot');
      }
    }
  };

  return (
    <div className="min-h-screen flex items-center justify-center bg-gradient-to-br
```

```

from-blue-50 via-white to-blue-50 p-4">
<div className="w-full max-w-md">
  /* Logo and Header */
  <div className="text-center mb-8">
    <div className="inline-flex items-center justify-center w-16 h-16 bg-primary
rounded-full mb-4 shadow-lg">
      <Activity className="w-8 h-8 text-white" strokeWidth={2.5} />
    </div>
    <h2 className="text-primary mb-2">■■■■■■■■■■</h2>
    <p className="text-text-secondary">■■■■■■■■■■</p>
  </div>

  /* Login/Register Card */
  <div className="bg-white rounded-2xl shadow-xl border border-border
overflow-hidden">
    /* Tabs */
    <div className="flex border-b border-border">
      <button
        onClick={() => setActiveTab('login')}
        className={`flex-1 py-4 transition-all ${
          activeTab === 'login'
            ? 'text-primary border-b-2 border-primary bg-blue-50'
            : 'text-text-secondary hover:bg-gray-50'
        }`>
      </button>
      <button
        onClick={() => setActiveTab('register')}
        className={`flex-1 py-4 transition-all ${
          activeTab === 'register'
            ? 'text-primary border-b-2 border-primary bg-blue-50'
            : 'text-text-secondary hover:bg-gray-50'
        }`>
      </button>
    </div>

    /* Login Form */
    {activeTab === 'login' & & (
      <div className="p-8">
        <div className="mb-6">
          <h3 className="text-text-primary mb-1">■■■■</h3>
          <p className="text-text-secondary">■■■■■■■■■■</p>
        </div>

        <form onSubmit={handleLogin} className="space-y-5">
          <div>
            <label className="block mb-2 text-text-primary">Email</label>
            <input
              type="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
              placeholder="your@email.com"
              required
            />
          </div>

          <div>
            <label className="block mb-2 text-text-primary">■■■■</label>
            <input
              type="password"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
              placeholder="....."
              required
            />
          </div>

          <button
            type="submit"
            className="w-full bg-primary hover:bg-primary-dark text-white py-3
rounded-lg transition-all shadow-md hover:shadow-lg"
          >
            ■■

```

```

        </button>
    </form>
</div>
})

{/* Register Form */}
{activeTab === 'register' && (
    <div className="p-8">
        <div className="mb-6">
            <h3 className="text-text-primary mb-1">■■■■</h3>
            <p className="text-text-secondary">■■■■■■■■■■</p>
        </div>

        <form onSubmit={handleRegister} className="space-y-5">
            <div>
                <label className="block mb-2 text-text-primary">■■</label>
                <input
                    type="text"
                    value={registerName}
                    onChange={(e) => setRegisterName(e.target.value)}
                    className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
                    placeholder="■■■■"
                    required
                />
            </div>

            <div>
                <label className="block mb-2 text-text-primary">Email</label>
                <input
                    type="email"
                    value={registerEmail}
                    onChange={(e) => setRegisterEmail(e.target.value)}
                    className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
                    placeholder="your@email.com"
                    required
                />
            </div>

            <div>
                <label className="block mb-2 text-text-primary">■■</label>
                <input
                    type="password"
                    value={registerPassword}
                    onChange={(e) => setRegisterPassword(e.target.value)}
                    className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50"
                    placeholder="■■ 6 ■■■"
                    minLength={6}
                    required
                />
                <p className="text-text-secondary mt-2">■■■■■■■■■■■■■■■■■■■■</p>
            </div>

            <div>
                <label className="block mb-2 text-text-primary">■■</label>
                <select
                    value={registerRole || ''}
                    onChange={(e) => setRegisterRole(e.target.value as UserRole)}
                    className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary focus:border-transparent
transition-all bg-gray-50 cursor-pointer"
                    required
                >
                    <option value="patient">■■</option>
                    <option value="doctor">■■</option>
                    <option value="caseManager">■■■■</option>
                </select>
            </div>

            <button
                type="submit"
                className="w-full bg-primary hover:bg-primary-dark text-white py-3
rounded-lg transition-all shadow-md hover:shadow-lg"
                >■■
            </button>
        </form>
    </div>
)

```

```
        &lt;/div&gt;
    })
    &lt;/div&gt;
    &lt;/div&gt;
    &lt;/div&gt;
);
}
```

■ File: src\components\PatientDashboard.tsx

```

import { useState, useEffect } from 'react';
import { User, useAuthStore } from '../store/useAuthStore';
import { Header } from '../Header';
import { HeadacheDiary } from '../HeadacheDiary';
import { QuestionnaireList } from '../QuestionnaireList';
import { PatientStats } from '../PatientStats';
import { Calendar, FileText, BarChart3 } from 'lucide-react';
import { API_BASE_URL } from '../config';

interface PatientDashboardProps {
  user: User;
  onLogout: () => void;
  onNavigate: (page: 'dashboard' | 'about') => void;
}

type TabType = 'diary' | 'questionnaire' | 'stats';

export function PatientDashboard({ user, onLogout, onNavigate }: PatientDashboardProps) {
  const [activeTab, setActiveTab] = useState<TabType>('diary');
  const currentUser = useAuthStore((state) => state.currentUser);

  // Profile settings state
  const [showProfileModal, setShowProfileModal] = useState(false);
  const [editAge, setEditAge] = useState<number | '>'>('');
  const [editGender, setEditGender] = useState('');
  const [editPhone, setEditPhone] = useState('');

  useEffect(() => {
    if (currentUser) {
      setEditAge(currentUser.age ?? '');
      setEditGender(currentUser.gender ?? '');
      setEditPhone(currentUser.phone ?? '');
    }
  }, [currentUser]);

  const handleSaveProfile = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!currentUser) return;
    try {
      const response = await fetch(`${API_BASE_URL}/api/users/${currentUser.id}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          age: editAge === '' ? null : Number(editAge),
          gender: editGender,
          phone: editPhone
        })
      });
      if (response.ok) {
        const updatedUser = await response.json();
        // Update store state instantly
        useAuthStore.getState().updateUserFields({
          age: updatedUser.age,
          gender: updatedUser.gender,
          phone: updatedUser.phone
        });
        alert('');
        setShowProfileModal(false);
      } else {
        alert('');
      }
    } catch (err) {
      console.error(err);
      alert('');
    }
  };

  const tabs = [
    { id: 'diary', label: '', icon: Calendar },
    { id: 'questionnaire', label: '', icon: FileText },
    { id: 'stats', label: '', icon: BarChart3 },
  ];

  return (
    <div className="min-h-screen bg-background">
      <Header user={user} onLogout={onLogout} onNavigate={onNavigate} />

      <div className="max-w-full mx-auto px-4 py-8">
        { /* Welcome Section */ }
      </div>
    </div>
  );
}

```

```

    <div className="mb-8 flex flex-col md:flex-row md:items-center md:justify-between
gap-4 border-b border-border pb-6">
    <div>
        <h1 className="text-text-primary mb-2">{user.name}</h1>
        <div className="text-text-secondary flex flex-wrap items-center gap-3
text-sm">
            <span>{currentUser?.age}& (
                <span className="w-1.5 h-1.5 bg-gray-300 rounded-full"></span>
                <span className="text-primary font-medium">{currentUser.age}</span>
            )
            {currentUser?.gender & currentUser?.gender !== 'M' & (
                <span className="w-1.5 h-1.5 bg-gray-300 rounded-full"></span>
                <span className="text-primary
font-medium">{currentUser.gender}</span>
            )
            {currentUser?.phone & (
                <span className="w-1.5 h-1.5 bg-gray-300 rounded-full"></span>
                <span className="text-primary
font-medium">{currentUser.phone}</span>
            )
        }
    </div>
    <button
        onClick={() => setShowProfileModal(true)}
        className="px-5 py-2 bg-primary hover:bg-primary-dark text-white rounded-lg
transition-all shadow-md font-medium text-sm self-start md:self-auto hover:shadow-lg"
        >
        </button>
    </div>

    { /* Tabs */ }
    <div className="flex gap-4 mb-6 border-b border-border overflow-x-auto">
        {tabs.map((tab) => {
            const Icon = tab.icon;
            return (
                <button
                    key={tab.id}
                    onClick={() => setActiveTab(tab.id as TabType)}
                    className={`flex items-center gap-2 px-6 py-3 transition-all
whitespace-nowrap $
                    {activeTab === tab.id
                        ? 'border-b-2 border-primary text-primary font-medium'
                        : 'text-text-secondary hover:text-primary'}
                    `
                >
                    <Icon className="w-5 h-5" />
                    {tab.label}
                </button>
            );
        })}
    </div>

    { /* Content */ }
    <div className="mt-6">
        {activeTab === 'diary' & <HeadacheDiary />}
        {activeTab === 'questionnaire' & <QuestionnaireList />}
        {activeTab === 'stats' & <PatientStats />}
    </div>
</div>

{ /* Profile Edit Modal */ }
{showProfileModal & (
    <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50
p-4 animate-fade-in">
        <div className="bg-white rounded-xl shadow-2xl max-w-md w-full border
border-border p-6 relative">
            <h3 className="text-text-primary mb-2 text-xl font-bold"></h3>
            <p className="text-text-secondary text-sm
mb-6">
                <form onSubmit={handleSaveProfile} className="space-y-4">
                    <div>
                        <label className="block text-sm font-medium text-text-primary
mb-1">

```


■ File: src\components\PatientRecordModal.tsx

```
import { useState, useEffect } from 'react';
import { X, Calendar, Activity, Pill, FileText, TrendingDown, AlertCircle } from
    'lucide-react';
import { API_BASE_URL } from '../config';

interface Patient {
    id: string;
    name: string;
    age: number;
    gender: string;
    lastVisit: string;
    midasScore: number;
    headacheDays: number;
    riskLevel: 'low' | 'medium' | 'high';
}

interface DiaryEntry {
    id: string;
    date: string;
    time: string;
    intensity: number;
    symptoms: string[];
    medication: string;
    notes: string;
}

interface QuestionnaireScore {
    name: string;
    score: number;
    maxScore: number;
    date: string;
    interpretation: string;
}

interface PatientRecordModalProps {
    patient: Patient;
    onClose: () => void;
}

export function PatientRecordModal({ patient, onClose }: PatientRecordModalProps) {
    const [diaries, setDiaries] = useState<DiaryEntry[]>([]);
    const [questionnaires, setQuestionnaires] = useState<QuestionnaireScore[]>([]);
    const [loading, setLoading] = useState<boolean>(true);

    useEffect(() => {
        const fetchPatientData = async () => {
            try {
                setLoading(true);
                // 1. Fetch diaries
                const diariesRes = await fetch(`${API_BASE_URL}/api/diaries/user/${patient.id}`);
                if (diariesRes.ok) {
                    const diariesData = await diariesRes.json();
                    setDiaries(diariesData);
                }

                // 2. Fetch questionnaires
                const questionnairesRes = await fetch(`${API_BASE_URL}/api/questionnaires/patient/${
                    patient.id
                }`);
                if (questionnairesRes.ok) {
                    const rawQData = await questionnairesRes.json();

                    // Map to QuestionnaireScore interface
                    const mappedScores: QuestionnaireScore[] = rawQData.map((q: any) => {
                        const dateObj = new Date(q.completedAt);
                        const dateStr = isNaN(dateObj.getTime()) ? '███' :
                            dateObj.toISOString().split('T')[0];

                        const qId = q.questionnaireId.toLowerCase();
                        let name = q.questionnaireId.toUpperCase();
                        let maxScore = 100;
                        let interpretation = '███';
                        const score = q.score;

                        if (qId === 'midas') {
                            maxScore = 27;
                            if (score >= 20) interpretation = '████ (Grade IV)';
                            else if (score >= 10) interpretation = '████ (Grade III)';
                            else if (score >= 5) interpretation = '████ (Grade II)';
                            else interpretation = '██████ (Grade I)';
```

```

    } else if (qId === 'psqi') {
      maxScore = 21;
      if (score > 8) interpretation = '■■■■■■';
      else if (score > 5) interpretation = '■■■■■■';
      else interpretation = '■■■■■■';
    } else if (qId === 'hads-anxiety' || qId === 'hads_anxiety' ||
qId.includes('anxiety')) {
      name = 'HADS-■■';
      maxScore = 21;
      if (score > 10) interpretation = '■■■■■■';
      else if (score > 7) interpretation = '■■■■';
      else interpretation = '■■';
    } else if (qId === 'hads-depression' || qId === 'hads_depression' ||
qId.includes('depression')) {
      name = 'HADS-■■';
      maxScore = 21;
      if (score > 10) interpretation = '■■■■■■';
      else if (score > 7) interpretation = '■■■■';
      else interpretation = '■■';
    } else if (qId === 'pss') {
      maxScore = 40;
      if (score > 26) interpretation = '■■■■■■';
      else if (score > 13) interpretation = '■■■■■■';
      else interpretation = '■■■■■■';
    } else if (qId === 'bdi') {
      maxScore = 63;
      if (score > 29) interpretation = '■■■■■■';
      else if (score > 19) interpretation = '■■■■';
      else if (score > 13) interpretation = '■■■■';
      else interpretation = '■■■■■■';
    } else if (qId === 'fss') {
      maxScore = 63;
      if (score >= 36) interpretation = '■■■■';
      else interpretation = '■■■■';
    } else if (qId === 'allodynia') {
      maxScore = 24;
      if (score > 8) interpretation = '■■■■■■';
      else if (score > 5) interpretation = '■■■■■■';
      else if (score > 2) interpretation = '■■■■■■';
      else interpretation = '■■■■';
    } else if (qId === 'wpi') {
      maxScore = 19;
      if (score > 7) interpretation = '■■■■■■■■';
      else interpretation = '■■';
    }
  }

  return {
    name,
    score,
    maxScore,
    date: dateStr,
    interpretation
  };
});

setQuestionnaires(mappedScores);
}
} catch (err) {
  console.error(err);
} finally {
  setLoading(false);
}
};

fetchPatientData();
}, [patient.id]);

// Calculate dynamic summaries
const currentMonthStr = new Date().toISOString().slice(0, 7); // yyyy-MM
const headacheDaysThisMonth = diaries.filter(d => d.date &&
  d.date.startsWith(currentMonthStr)).length;

const avgPainIntensity = diaries.length > 0
  ? (diaries.reduce((sum, d) => sum + d.intensity, 0) / diaries.length).toFixed(1)
  : "0.0";

return (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-xl shadow-2xl max-w-4xl w-full max-h-[90vh]
overflow-y-auto">
      { /* Header */ }
    <div className="sticky top-0 bg-white border-b border-border p-6 flex
items-center justify-between">

```


■ **File:** src\components\PatientStats.tsx

```
import { useState, useEffect } from 'react';
import { AreaChart, Area, BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer, PieChart, Pie, Cell } from 'recharts';
import { TrendingUp, Calendar, Activity, Pill, AlertTriangle, AlertCircle } from 'lucide-react';
import { useAuthStore } from '../store/useAuthStore';
import { API_BASE_URL } from '../config';

// ██████████

const COLORS = ['#3b82f6', '#06b6d4', '#10b981', '#f59e0b', '#ec4899', '#8b5cf6'];

interface DiaryEntry {
    id?: string;
    userId?: string;
    date: string;
    time: string;
    intensity: number;
    symptoms: string[];
    medication: string;
    notes: string;
}

interface QuestionnaireResponse {
    id?: string;
    patientId: string;
    questionnaireId: string;
    score: number;
    details: string;
    completedAt: string;
}

export function PatientStats() {
    const currentUser = useAuthStore((state) => state.currentUser);
    const [diaries, setDiaries] = useState<DiaryEntry[]>([]);
    const [questionnaires, setQuestionnaires] = useState<QuestionnaireResponse[]>([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<string | null>(null);

    const fetchData = async () => {
        if (!currentUser?.id) return;
        try {
            setLoading(true);
            setError(null);

            // Fetch diaries
            const diariesRes = await fetch(`${API_BASE_URL}/api/diaries/user/${currentUser.id}`);
            ;
            let diariesData: DiaryEntry[] = [];
            if (diariesRes.ok) {
                diariesData = await diariesRes.json();
                setDiaries(diariesData);
            } else {
                throw new Error('██████████');
            }

            // Fetch questionnaires
            const questionnairesRes = await fetch(`${API_BASE_URL}/api/questionnaires/patient/${currentUser.id}`);
            if (questionnairesRes.ok) {
                const questionnairesData = await questionnairesRes.json();
                setQuestionnaires(questionnairesData);
            } else {
                throw new Error('██████████');
            }
        } catch (err: any) {
            console.error(err);
            setError('████████████████████ Spring Boot ██████████');
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        fetchData();
    }, [currentUser]);

    // 1. ██████████
    const currentMonthStr = new Date().toISOString().slice(0, 7); // yyyy-MM
```



```

        <Activity className="w-6 h-6" />;
    </div>;
</div>;
<div className="text-text-secondary text-sm">■■■■■■</div>;
<h3 className="text-danger mt-1 text-2xl font-bold">{avgIntensity}</span>
className="text-sm font-normal text-text-secondary">/ 10</span></h3>;
</div>;

<div className="bg-white rounded-xl shadow-sm p-6 border border-border
hover:shadow-md transition-shadow flex flex-col items-center text-center">;
    <div className="mb-2">;
        <div className="w-12 h-12 bg-green-50 text-green-500 rounded-xl flex
items-center justify-center">;
            <TrendingUp className="w-6 h-6" />;
        </div>;
    </div>;
    <div className="text-text-secondary text-sm">■■■■■■</div>;
    <h3 className="text-success mt-1 text-2xl font-bold">{avgDuration}</span>
className="text-sm font-normal text-text-secondary">■■</span></h3>;
    </div>;

<div className="bg-white rounded-xl shadow-sm p-6 border border-border
hover:shadow-md transition-shadow flex flex-col items-center text-center">;
    <div className="mb-2">;
        <div className="w-12 h-12 bg-orange-50 text-orange-500 rounded-xl flex
items-center justify-center">;
            <Pill className="w-6 h-6" />;
        </div>;
    </div>;
    <div className="text-text-secondary text-sm">■■■■■■</div>;
    <h3 className="text-warning mt-1 text-2xl font-bold">{medicationCountThisMonth}</span>
className="text-sm font-normal text-text-secondary">■■</span></h3>;
    </div>;
</div>;

{ /* Charts Grid */ }
<div className="grid grid-cols-1 lg:grid-cols-2 gap-6">;
{ /* Intensity Trend - AreaChart with glowing gradient */ }
    <div className="bg-white rounded-xl shadow-md p-6 border border-border">;
        <h3 className="text-text-primary mb-6 font-semibold text-center">■■■■■■</h3>;
        {chartHeadacheData.length === 0 ? (
            <div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary">;
                <AlertCircle className="w-12 h-12 text-gray-300 mb-2" />;
                <p>■■■■■■</p>;
            </div>;
        ) : (
            <ResponsiveContainer width="100%" height={300}>;
                <AreaChart data={chartHeadacheData} margin={{ top: 20, right: 30, left: 0,
bottom: 15 }}>;
                    <defs>;
                        <linearGradient id="colorIntensity" x1="0" y1="0" x2="0" y2="1">;
                            <stop offset="5%" stopColor="#3b82f6" stopOpacity={0.25}>;
                            <stop offset="95%" stopColor="#3b82f6" stopOpacity={0.01}>;
                        </linearGradient>;
                    </defs>;
                    <CartesianGrid strokeDasharray="4 4" stroke="#f1f5f9" vertical={false} />;
                    <XAxis dataKey="date" stroke="#94a3b8" tickLine={false} axisLine={false}
dy={12} />;
                    <YAxis stroke="#94a3b8" domain={[0, 10]} tickLine={false}
axisLine={false} dx={-12} />;
                    <Tooltip contentStyle={customTooltipStyle} />;
                    <Legend iconType="circle" />;
                    <Area type="monotone" dataKey="intensity" stroke="#3b82f6"
strokeWidth={2.5} fillOpacity={1} fill="url(#colorIntensity)" name="■■■■■■"
activeDot={{ r: 6, strokeWidth: 0 }} />;
                    </AreaChart>;
                </ResponsiveContainer>;
            </>
        ) : (
            <ResponsiveContainer width="100%" height={300}>;
                <BarChart with custom gradient and rounded top corners */>
                <div className="bg-white rounded-xl shadow-md p-6 border border-border">;
                    <h3 className="text-text-primary mb-6 font-semibold text-center">■■■■■■</h3>;
                    {chartHeadacheData.length === 0 ? (
                        <div className="flex flex-col items-center justify-center h-[300px]
text-text-secondary">;
                            <AlertCircle className="w-12 h-12 text-gray-300 mb-2" />;
                            <p>■■■■■■</p>;
                        </div>;
                    ) : (
                        <ResponsiveContainer width="100%" height={300}>;

```



```

const entry = chartMidasData.find(d => d.score === value);
return [`${value} (Grade ${entry?.grade})`, name];
}
return [value, name];
}}
/ >
< Legend iconType="circle" / >
< Area type="monotone" dataKey="score" stroke="#10b981" strokeWidth={2.5}
fillOpacity={1} fill="url(#colorMidas)" name="MIDAS" activeDot={{ r: 6,
strokeWidth: 0 }} / >
< /AreaChart >
< /ResponsiveContainer >
)}
< /div >
< /div >

{/* Insights */}
< div className="bg-gradient-to-br from-blue-50/70 via-cyan-50/30 to-blue-50/50
rounded-2xl p-6 border border-blue-100 shadow-sm" >
  < h3 className="text-text-primary mb-4 font-semibold flex items-center
justify-center gap-2" >
    < Activity className="w-5 h-5 text-primary" / >
    ■■■■■■■■
  < /h3 >
  < div className="space-y-4" >
    {insightsList.map((insight, idx) => (
      < div key={idx} className="flex items-start gap-3" >
        < div className="w-2 h-2 bg-primary rounded-full mt-2 flex-shrink-0" > < /div >
        < p className="text-text-primary leading-relaxed text-sm" > {insight} < /p >
      < /div >
    ))}
  < /div >
< /div >
< /div >
);
}

```

■ File: src\components\PrescriptionModal.tsx

```
import { useState } from 'react';
import { X, Pill, Plus, Trash2, FileText } from 'lucide-react';

interface Patient {
  id: string;
  name: string;
  age: number;
  gender: string;
}

interface PrescriptionModalProps {
  patient: Patient;
  onClose: () => void;
  onSubmit: (prescription: PrescriptionData) => void;
}

interface Medication {
  id: string;
  name: string;
  dosage: string;
  frequency: string;
  duration: string;
  notes: string;
}

interface PrescriptionData {
  medications: Medication[];
  diagnosis: string;
  instructions: string;
  nextVisit: string;
}

const commonMedications = [
  { name: '■■■■■■■ (Panadol)', defaultDosage: '500mg', defaultFrequency: '■■■■■4-6■■■1■' },
  { name: '■■■■■■■ (Panadol Extra)', defaultDosage: '500mg + 65mg ■■■', defaultFrequency: '■■■■■4-6■■■1■' },
  { name: 'EVE Quick ■■■ (EVE)', defaultDosage: '150mg Ibuprofen', defaultFrequency: '■■■■■(■■2■)' },
  { name: '■■■■■■■■■ (Bufferin)', defaultDosage: '500mg', defaultFrequency: '■■■■■4-6■■■1■' },
  { name: '■■■■■■■ (Snooz)', defaultDosage: '500mg', defaultFrequency: '■4-6■■■1■(■■1-2■)' },
];

export function PrescriptionModal({ patient, onClose, onSubmit }: PrescriptionModalProps) {
  const [medications, setMedications] = useState<Medication[]>([]);
  const [diagnosis, setDiagnosis] = useState('■■■');
  const [instructions, setInstructions] = useState('');
  const [nextVisit, setNextVisit] = useState('');
  const [showAddMed, setShowAddMed] = useState(false);
  const [newMed, setNewMed] = useState({
    name: '',
    dosage: '',
    frequency: '',
    duration: '7■',
    notes: '',
  });

  const handleAddMedication = () => {
    if (newMed.name && newMed.dosage && newMed.frequency) {
      const medication: Medication = {
        id: Date.now().toString(),
        ...newMed,
      };
      setMedications([...medications, medication]);
      setNewMed({
        name: '',
        dosage: '',
        frequency: '',
        duration: '7■',
        notes: '',
      });
      setShowAddMed(false);
    }
  };

  const handleRemoveMedication = (id: string) => {
    setMedications(medications.filter(med => med.id !== id));
  };
}
```

```

};

const handleQuickAdd = (commonMed: typeof commonMedications[0]) => {
  setNewMed({
    name: commonMed.name,
    dosage: commonMed.defaultDosage,
    frequency: commonMed.defaultFrequency,
    duration: '7',
    notes: '',
  });
  setShowAddMed(true);
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  onSubmit({
    medications,
    diagnosis,
    instructions,
    nextVisit,
  });
};

return (
  <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50 p-4">
    <div className="bg-white rounded-xl shadow-2xl max-w-3xl w-full max-h-[90vh]
      overflow-y-auto">
      { /* Header */ }
      <div className="sticky top-0 bg-white border-b border-border p-6 flex
        items-center justify-between">
        <div>
          <h2 className="text-text-primary">
          <p className="text-text-secondary">
            {patient.name} · {patient.age} · {patient.gender}
          </p>
        </div>
        <button
          onClick={onClose}
          className="p-2 hover:bg-gray-100 rounded-lg transition-colors"
        >
          <X className="w-6 h-6 text-text-secondary" />
        </button>
      </div>

      { /* Content */ }
      <form onSubmit={handleSubmit} className="p-6 space-y-6">
        { /* Diagnosis */ }
        <div>
          <label className="block mb-2">
          <input
            type="text"
            value={diagnosis}
            onChange={(e) => setDiagnosis(e.target.value)}
            className="w-full px-4 py-3 border border-border rounded-lg
            focus:outline-none focus:ring-2 focus:ring-primary"
            placeholder=""
            required
          />
        </div>

        { /* Medications */ }
        <div>
          <div className="flex items-center justify-between mb-4">
            <label className="flex items-center gap-2">
              <Pill className="w-5 h-5" />
            </label>
            <button
              type="button"
              onClick={() => setShowAddMed(true)}
              className="flex items-center gap-2 px-4 py-2 bg-primary
              hover:bg-primary-dark text-white rounded-lg transition-all"
            >
              <Plus className="w-4 h-4" />
            </button>
          </div>

          { /* Common Medications Quick Add */ }
          { !showAddMed & & medications.length === 0 & & (
            <div className="mb-4">
              <p className="text-text-secondary mb-2">

```

```

    <div className="grid grid-cols-2 md:grid-cols-3 gap-2">
      {commonMedications.map((med, idx) => {
        <button
          key={idx}
          type="button"
          onClick={() => handleQuickAdd(med)}
          className="px-3 py-2 border border-border rounded-lg
hover:border-primary hover:bg-medical-blue transition-all text-left"
        >
          <div className="text-text-primary">{med.name}</div>
          <div className="text-text-secondary
text-sm">{med.defaultDosage}</div>
        </button>
      })}
    </div>
  </div>
)

/* Add Medication Form */
{showAddMed & & (
  <div className="bg-medical-blue rounded-lg p-4 mb-4 space-y-4">
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div>
        <label className="block mb-2 text-sm">Name</label>
        <input
          type="text"
          value={newMed.name}
          onChange={(e) => setNewMed({ ...newMed, name: e.target.value })}
          className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          placeholder="Name"
        />
      </div>
      <div>
        <label className="block mb-2 text-sm">Dosage</label>
        <input
          type="text"
          value={newMed.dosage}
          onChange={(e) => setNewMed({ ...newMed, dosage: e.target.value })}
          className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          placeholder="500mg"
        />
      </div>
      <div>
        <label className="block mb-2 text-sm">Frequency</label>
        <input
          type="text"
          value={newMed.frequency}
          onChange={(e) => setNewMed({ ...newMed, frequency: e.target.value
        ))}
        <div className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
        <div>
          <label className="block mb-2 text-sm">Duration</label>
          <input
            type="text"
            value={newMed.duration}
            onChange={(e) => setNewMed({ ...newMed, duration: e.target.value })}
            className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
            placeholder="7"
          />
        </div>
      </div>
      <div>
        <label className="block mb-2 text-sm">Notes</label>
        <input
          type="text"
          value={newMed.notes}
          onChange={(e) => setNewMed({ ...newMed, notes: e.target.value })}
          className="w-full px-3 py-2 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          placeholder="Notes"
        />
      </div>
    </div>
    <div className="flex gap-2">
      <button
        type="button"

```



```

    /&gt;
    &lt;/div&gt;

    { /* Next Visit */ }
    &lt;div&gt;
        &lt;label className="block mb-2"&gt;■■■■■■&lt;/label&gt;
        &lt;input
            type="date"
            value={nextVisit}
            onChange={(e) =&gt; setNextVisit(e.target.value)}
            className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
        /&gt;
        &lt;/div&gt;

        { /* Buttons */ }
        &lt;div className="flex gap-3"&gt;
            &lt;button
                type="button"
                onClick={onClose}
                className="flex-1 px-6 py-3 border border-border hover:bg-gray-50
rounded-lg transition-all"
            /&gt;
            ■■
            &lt;/button&gt;
            &lt;button
                type="submit"
                className="flex-1 px-6 py-3 bg-primary hover:bg-primary-dark text-white
rounded-lg transition-all shadow-md"
            /&gt;
            ■■■■■■■■
            &lt;/button&gt;
        &lt;/div&gt;
    &lt;/form&gt;
&lt;/div&gt;
);
}

```


■ File: src\components\QuestionnaireList.tsx

```
import { useState, useEffect } from 'react';
import { FileText, ChevronRight, CheckCircle, Circle } from 'lucide-react';
import { useAuthStore } from '../store/useAuthStore';
import { API_BASE_URL } from '../config';
import { MIDASQuestionnaire } from '../questionnaires/MIDASQuestionnaire';
import { HADSQuestionnaire } from '../questionnaires/HADSQuestionnaire';
import { BDIQuestionnaire } from '../questionnaires/BDIQuestionnaire';
import { PSQIQuestionnaire } from '../questionnaires/PSQIQuestionnaire';
import { FSSQuestionnaire } from '../questionnaires/FSSQuestionnaire';
import { WPIQuestionnaire } from '../questionnaires/WPIQuestionnaire';
import { AllodyniaQuestionnaire } from '../questionnaires/AllodyniaQuestionnaire';
import { PSSQuestionnaire } from '../questionnaires/PSSQuestionnaire';

interface Questionnaire {
  id: string;
  name: string;
  fullName: string;
  description: string;
  completed: boolean;
  lastCompleted?: string;
}

export function QuestionnaireList() {
  const currentUser = useAuthStore((state) => state.currentUser);
  const [selectedQuestionnaire, setSelectedQuestionnaire] = useState<string | null>(null);
  const [questionnaires, setQuestionnaires] = useState<Questionnaire[]>([
    {
      id: 'midas',
      name: 'MIDAS',
      fullName: 'Migraine Disability Assessment',
      description: '■■■■■■■■■■',
      completed: false,
    },
    {
      id: 'hads',
      name: 'HADS',
      fullName: 'Hospital Anxiety and Depression Scale',
      description: '■■■■■■■■■■',
      completed: false,
    },
    {
      id: 'bdi',
      name: 'BDI',
      fullName: 'Beck Depression Inventory',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'psqi',
      name: 'PSQI',
      fullName: 'Pittsburgh Sleep Quality Index',
      description: '■■■■■■■■■■',
      completed: false,
    },
    {
      id: 'fss',
      name: 'FSS',
      fullName: 'Fatigue Severity Scale',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'wpi',
      name: 'WPI',
      fullName: 'Widespread Pain Index',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'allodynia',
      name: 'Allodynia',
      fullName: 'Allodynia Questionnaire',
      description: '■■■■■■■■',
      completed: false,
    },
    {
      id: 'pss',
      name: 'PSS',
      fullName: 'Perceived Stress Scale',
    }
  ])
```

```

        description: '■■■■■■■',
        completed: false,
    },
]);

useEffect(() => {
    if (currentUser?.id) {
        fetch(`${API_BASE_URL}/api/questionnaires/patient/${currentUser.id}`)
            .then(res => res.json())
            .then((responses: any[]) => {
                setQuestionnaires(prev => prev.map(q => {
                    const qs = responses.filter(r => r.questionnaireId === q.id);
                    if (qs.length > 0) {
                        // Find the latest response by date simply or take first
                        const lastResponse = qs[qs.length - 1];
                        return { ...q, completed: true, lastCompleted:
                            lastResponse.completedAt.split('T')[0] };
                    }
                    return q;
                }));
            })
            .catch(err => console.error("Failed to load questionnaires", err));
    }
}, [currentUser]);

const handleComplete = async (questionnaireId: string, score: number = 0,
    targetListId?: string) => {
    if (!currentUser) return;

    const displayId = targetListId || questionnaireId;
    try {
        await fetch(`${API_BASE_URL}/api/questionnaires`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                patientId: currentUser.id,
                questionnaireId: questionnaireId,
                score: score,
                details: 'Completed via UI'
            })
        });

        setQuestionnaires((prev) => {
            prev.map((q) => {
                q.id === displayId
                    ? { ...q, completed: true, lastCompleted: new
                        Date().toISOString().split('T')[0] }
                    : q
            })
        });
    } catch (err) {
        console.error("Save questionnaire error", err);
    }

    setSelectedQuestionnaire(null);
};

if (selectedQuestionnaire === 'midas') {
    return <MIDASQuestionnaire onBack={() => setSelectedQuestionnaire(null)}
        onComplete={(score) => handleComplete('midas', score)} />;
}
if (selectedQuestionnaire === 'hads') {
    return (
        <HADSQuestionnaire
            onBack={() => setSelectedQuestionnaire(null)}
            onComplete={async (anxiety, depression) => {
                await handleComplete('hads-anxiety', anxiety, 'hads');
                await handleComplete('hads-depression', depression, 'hads');
            }}
        />
    );
}
if (selectedQuestionnaire === 'bdi') {
    return <BDIQuestionnaire onBack={() => setSelectedQuestionnaire(null)}
        onComplete={(score) => handleComplete('bdi', score)} />;
}
if (selectedQuestionnaire === 'psqi') {
    return <PSQIQuestionnaire onBack={() => setSelectedQuestionnaire(null)}
        onComplete={(score) => handleComplete('psqi', score)} />;
}
if (selectedQuestionnaire === 'fss') {
    return <FSSQuestionnaire onBack={() => setSelectedQuestionnaire(null)}

```



```
        }}  
        &lt;/div>  
        &lt;/button>  
    )}  
    &lt;/div>  
    &lt;/div>  
};  
}
```

■ File: src/components/figma/ImgWithFallback.tsx

```
import React, { useState } from 'react'

const ERROR_IMG_SRC =
  ''

export function ImgWithFallback(props: React.ImgHTMLAttributes<HTMLImageElement> & {
  const [didError, setDidError] = useState(false)

  const handleError = () => {
    setDidError(true)
  }

  const { src, alt, style, className, ...rest } = props

  return didError ? (
    <div
      className={`inline-block bg-gray-100 text-center align-middle ${className ?? ''}`}
      style={style}
    >
      <div className="flex items-center justify-center w-full h-full">
        <img src={ERROR_IMG_SRC} alt="Error loading image" {...rest}
          data-original-url={src} />
      </div>
    </div>
  ) : (
    <img src={src} alt={alt} className={className} style={style} {...rest}
      onError={handleError} />
  )
}
```

■ File: src\components\questionnaires\AllodyniaQuestionnaire.tsx

[illegible]

```

        <label className="block text-text-primary">
          {index + 1}. {question}
        </label>
        <div className="grid grid-cols-2 gap-3">
          {options.map((option, optionIndex) => (
            <button
              key={optionIndex}
              type="button"
              onClick={() => {
                const newAnswers = [...answers];
                newAnswers[index] = optionIndex;
                setAnswers(newAnswers);
              }}
              className={`p-4 rounded-lg border-2 transition-all ${
                answers[index] === optionIndex
                  ? 'bg-primary text-white border-primary'
                  : 'bg-white text-text-secondary border-border
hover:border-primary'
              }`}
            >
              {option}
            </button>
          ))}
        </div>
      </div>
    </div>
  </div>
  <div>
    { /* Score Display */ }
    <div className="bg-gray-50 p-6 rounded-lg border-2 border-border">
      <div className="flex items-center justify-between">
        <span className="text-text-secondary">■■■■</span>
        <span className="text-primary">{getScore()}/12</span>
      </div>
      <p className="text-text-secondary mt-2">0■■■ | 1-2■■■ | 3-6■■■ | 7-12■■■</p>
    </div>

    <div className="flex gap-3 justify-end">
      <button
        type="button"
        onClick={onBack}
        className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
        >
        ■■
      </button>
      <button
        type="submit"
        className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
        >
        <Save className="w-5 h-5" />
        ■■■■
      </button>
    </div>
  </div>
</div>
</div>
);
}

```

■ File: src\components\questionnaires\BDIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface BDIQuestionnaireProps {
  onBack: () => void;
  onComplete: (score: number) => void;
}

export function BDIQuestionnaire({ onBack, onComplete }: BDIQuestionnaireProps) {
  const [answers, setAnswers] = useState<number[]>(Array(10).fill(-1));

  const questions = [
    {
      text: '■',
      options: ['■■■■■■', '■■■■■■', '■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■',
      options: ['■■■■■■■■■■', '■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■',
      options: ['■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■',
      options: ['■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
    {
      text: '■■■■■',
      options: ['■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■', '■■■■■■■■■■■■■■■■■■'],
    },
  ];

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (answers.some((a) => a === -1)) {
      alert('■■■■■■■■■■');
      return;
    }

    const total = answers.reduce((sum, val) => sum + val, 0);
    let level = '';
    if (total <= 13) level = '■■■■■■■■';
    else if (total <= 19) level = '■■■■■■';
    else if (total <= 28) level = '■■■■■';
    else level = '■■■■';

    alert(`BDI ■■■■■■\n\n■■■■■${total}\n■■■■■${level}\n\n■■■■■■■■■■■■■■■■■■`);
    onComplete(total);
  };

  const getScore = () => {
    return answers.reduce((sum, val) => sum + (val >= 0 ? val : 0), 0);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-4">
```


}

■ File: src\components\questionnaires\FSSQuestionnaire.tsx

```

import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface FSSQuestionnaireProps {
  onBack: () => void;
  onComplete: (score: number) => void;
}

export function FSSQuestionnaire({ onBack, onComplete }: FSSQuestionnaireProps) {
  const [answers, setAnswers] = useState<number[]>(Array(9).fill(-1));

  const questions = [
    '■■■■■■',
    '■■■■■■',
    '■■■■',
    '■■■■■■■■■■',
    '■■■■■■■■',
    '■■■■■■■■■■■■',
    '■■■■■■■■■■■■ and ■■■',
    '■■■■■■■■■■■■■■■■■■',
    '■■■■■■■■■■■■■■■■■■■■',
  ];

  const options = ['■■■■■■', '■■■', '■■■■■■', '■■■', '■■■■■■', '■■■', '■■■■■■'];

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (answers.some((a) => a === -1)) {
      alert('■■■■■■■■');
      return;
    }

    const total = answers.reduce((sum, val) => sum + (val + 1), 0);
    const average = (total / 9).toFixed(1);
    const level = parseFloat(average) >= 4 ? '■■■■' : '■■■■';

    alert(`FSS ■■■■■\n\n■■■■■${average}\n■■■${level}\n\n■■■■■■■■■■■■■■`);
    onComplete(total);
  };

  const getAverage = () => {
    const validAnswers = answers.filter((a) => a >= 0);
    if (validAnswers.length === 0) return 0;
    const total = validAnswers.reduce((sum, val) => sum + (val + 1), 0);
    return (total / validAnswers.length).toFixed(1);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-4">
        <button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
          hover:bg-gray-50 transition-colors"
          >
          <ArrowLeft className="w-5 h-5" />
          ■■
        </button>
      </div>
      <div>
        <h2 className="text-text-primary">FSS ■■■■■■</h2>
        <p className="text-text-secondary">Fatigue Severity Scale</p>
      </div>

      <div className="bg-white rounded-xl shadow-lg p-6 border border-border">
        <form onSubmit={handleSubmit} className="space-y-6">
          <div className="bg-medical-blue p-4 rounded-lg">
            <p className="text-text-primary">■■■■■■■■■■■■■■■■■■■■1-7■■■</p>
          </div>

          {questions.map((question, index) => (
            <div key={index} className="space-y-3 pb-4 border-b border-border
            last:border-b-0">
              <label className="block text-text-primary">
                {index + 1}. {question}
              </label>
              <div className="grid grid-cols-7 gap-2">
                {options.map((option, optionIndex) => (
                  <button

```

```

        key={optionIndex}
        type="button"
        onClick={() => {
          const newAnswers = [...answers];
          newAnswers[index] = optionIndex;
          setAnswers(newAnswers);
        }}
        className={`p-3 rounded-lg border-2 transition-all text-center ${
          answers[index] === optionIndex
            ? 'bg-primary text-white border-primary'
            : 'bg-white text-text-secondary border-border
hover:border-primary'
        }`}
        title={option}
        < >
        {optionIndex + 1}
        < /button >
      )}
    < /div >
    < div className="flex justify-between text-text-secondary" >
      < span >■■■■■< /span >
      < span >■■■■■< /span >
    < /div >
  < /div >
))}

{/* Score Display */}
< div className="bg-gray-50 p-6 rounded-lg border-2 border-border" >
  < div className="flex items-center justify-between" >
    < span className="text-text-secondary" >■■■■■< /span >
    < span className="text-primary" >{getAverage()}/7< /span >
  < /div >
  < p className="text-text-secondary mt-2" >■■■■ ≥ 4 ■■■■■■< /p >
< /div >

< div className="flex gap-3 justify-end" >
  < button
    type="button"
    onClick={onBack}
    className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
    >
    ■■
  < /button >
  < button
    type="submit"
    className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
    >
    < Save className="w-5 h-5" / >
    ■■■■
  < /button >
< /div >
< /form >
< /div >
< /div >
);
}

```

■ File: src\components\questionnaires\HADSQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface HADSQuestionnaireProps {
  onBack: () => void;
  onComplete: (anxietyScore: number, depressionScore: number) => void;
}

export function HADSQuestionnaire({ onBack, onComplete }: HADSQuestionnaireProps) {
  const [answers, setAnswers] = useState<number[]>(Array(14).fill(-1));

  const questions = [
    { text: 'I feel nervous', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel depressed', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel lonely', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a burden on others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a failure', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to myself', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to myself', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to myself', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to myself', type: 'A', options: ['None', 'A little', 'A lot', 'Very much'] },
    { text: 'I feel like I am a disappointment to others', type: 'D', options: ['None', 'A little', 'A lot', 'Very much'] },
  ];

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (answers.some((a) => a === -1)) {
      alert('Please answer all questions');
      return;
    }

    const anxietyScore = answers
      .filter((_, idx) => questions[idx].type === 'A')
      .reduce((sum, val) => sum + val, 0);

    const depressionScore = answers
      .filter((_, idx) => questions[idx].type === 'D')
      .reduce((sum, val) => sum + val, 0);

    alert(
      `HADS Anxiety Score (A) is ${anxietyScore} and Depression Score (D) is ${depressionScore}`
    );
    onComplete(anxietyScore, depressionScore);
  };

  const getAnxietyScore = () => {
    return answers
      .filter((_, idx) => questions[idx].type === 'A')
      .reduce((sum, val) => sum + (val >= 0 ? val : 0), 0);
  };

  const getDepressionScore = () => {
    return answers
      .filter((_, idx) => questions[idx].type === 'D')
      .reduce((sum, val) => sum + (val >= 0 ? val : 0), 0);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-4">
        <button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-gray-200 rounded-lg
            hover:bg-gray-50 transition-colors">
          <ArrowLeft className="w-5 h-5" />
          Back
        </button>
      </div>
      <div>
        <h2 className="text-text-primary">HADS Questionnaire</h2>
        <p className="text-text-secondary">Hospital Anxiety and Depression Scale</p>
      </div>
    </div>
  );
}
```


■ File: src\components\questionnaires\MIDASQuestionnaire.tsx

[illegible]

```

        setAnswers(newAnswers);
    }}
    className="w-full md:w-48 px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
    placeholder="■■■"
    required
    /&gt;
    &lt;/div&gt;
    )}}

    { /* Score Display */
    &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
    &lt;div className="flex items-center justify-between mb-4"&gt;
    &lt;h4 className="text-text-primary"&gt;■■&lt;/h4&gt;
    &lt;span className="text-primary"&gt;{answers.reduce((sum, val) => sum + (val ===
'' ? 0 : val), 0)} ■&lt;/span&gt;
    &lt;/div&gt;
    &lt;div className="flex items-center justify-between"&gt;
    &lt;span className="text-text-secondary"&gt;■■&lt;/span&gt;
    &lt;span className={` ${grade.color}`}&gt;
        Grade {grade.grade} - {grade.level}
    &lt;/span&gt;
    &lt;/div&gt;
    &lt;/div&gt;

    &lt;div className="flex gap-3 justify-end"&gt;
    &lt;button
        type="button"
        onClick={onBack}
        className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
        &gt;
        ■■
    &lt;/button&gt;
    &lt;button
        type="submit"
        className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
        &gt;
        &lt;Save className="w-5 h-5" /&gt;
        ■■■■
    &lt;/button&gt;
    &lt;/div&gt;
    &lt;/form&gt;
    &lt;/div&gt;
    &lt;/div&gt;
    );
}

```


■ File: src\components\questionnaires\PSQIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface PSQIQuestionnaireProps {
  onBack: () => void;
  onComplete: (score: number) => void;
}

export function PSQIQuestionnaire({ onBack, onComplete }: PSQIQuestionnaireProps) {
  const [bedTime, setBedTime] = useState('23:00');
  const [fallAsleepTime, setFallAsleepTime] = useState('30');
  const [wakeTime, setWakeTime] = useState('07:00');
  const [sleepHours, setSleepHours] = useState('7');
  const [answers, setAnswers] = useState<number[]>(Array(7).fill(-1));

  const questions = [
    {
      text: 'How long does it take you to fall asleep?',
      options: ['< 30', '30-45', '46-60', '61-90'],
    },
    {
      text: 'Do you have trouble falling asleep?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
    {
      text: 'Do you wake up too early in the morning?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
    {
      text: 'Do you have trouble staying asleep?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
    {
      text: 'Do you feel tired or fatigued during the day?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
    {
      text: 'Do you feel restless or can\'t relax during the day?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
    {
      text: 'Do you feel depressed or hopeless during the day?',
      options: ['No', 'Yes', 'Sometimes', 'Often'],
    },
  ];

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!bedTime || !fallAsleepTime || !wakeTime || !sleepHours) {
      alert('Please fill in all fields');
      return;
    }
    if (answers.some((a) => a === -1)) {
      alert('Please answer all questions');
      return;
    }

    const total = answers.reduce((sum, val) => sum + val, 0);
    const quality = total > 5 ? 'Good' : 'Poor';

    alert(`PSQI Score: ${total} / 30. Sleep Quality: ${quality}`);
    onComplete(total);
  };

  const getScore = () => {
    return answers.reduce((sum, val) => sum + (val >= 0 ? val : 0), 0);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-4">
        <button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-gray-200 rounded-lg
            hover:bg-gray-50 transition-colors">
          <ArrowLeft className="w-5 h-5"/>
          Back
        </button>
      </div>
    </div>
  );
}
```

```

    </div>
    <h2 className="text-text-primary">PSQI </h2>
    <p className="text-text-secondary">Pittsburgh Sleep Quality Index</p>
  </div>
</div>

<div className="bg-white rounded-xl shadow-lg p-6 border border-border">
  <form onSubmit={handleSubmit} className="space-y-6">
    <div className="bg-medical-blue p-4 rounded-lg">
      <p className="text-text-primary"></p>
    </div>

    { /* Basic sleep info */ }
    <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
      <div>
        <label className="block mb-2"></label>
        <input
          type="time"
          value={bedTime}
          onChange={(e) => setBedTime(e.target.value)}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
        />
      </div>
      <div>
        <label className="block mb-2"></label>
        <input
          type="number"
          value={fallAsleepTime}
          onChange={(e) => setFallAsleepTime(e.target.value)}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
        />
      </div>
      <div>
        <label className="block mb-2"></label>
        <input
          type="time"
          value={wakeTime}
          onChange={(e) => setWakeTime(e.target.value)}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
        />
      </div>
      <div>
        <label className="block mb-2"></label>
        <input
          type="number"
          value={sleepHours}
          onChange={(e) => setSleepHours(e.target.value)}
          className="w-full px-4 py-3 border border-border rounded-lg
focus:outline-none focus:ring-2 focus:ring-primary"
          step="0.5"
        />
      </div>
    </div>

    {questions.map((question, index) => (
      <div key={index} className="space-y-3 pb-4 border-b border-border
last:border-b-0">
        <label className="block text-text-primary">{question.text}</label>
        <div className="grid grid-cols-1 md:grid-cols-2 gap-2">
          {question.options.map((option, optionIndex) => (
            <button
              key={optionIndex}
              type="button"
              onClick={() => {
                const newAnswers = [...answers];
                newAnswers[index] = optionIndex;
                setAnswers(newAnswers);
              }}
              className={`p-3 rounded-lg border-2 transition-all text-left ${
                answers[index] === optionIndex
                  ? 'bg-primary text-white border-primary'
                  : 'bg-white text-text-secondary border-border
hover:border-primary'
              }`}
            >{option}</button>
          ))}
        </div>
      </div>
    ))}
  </div>

```

```

        </div>
    )}

    { /* Score Display */
    <div className="bg-gray-50 p-6 rounded-lg border-2 border-border">
        <div className="flex items-center justify-between">
            <span className="text-text-secondary">■■</span>
            <span className="text-primary">{getScore()}/21</span>
        </div>
        <p className="text-text-secondary mt-2">■■ {'>'} 5 ■■■■■■■■</p>
    </div>

    <div className="flex gap-3 justify-end">
        <button
            type="button"
            onClick={onBack}
            className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
            >
            ■■
        </button>
        <button
            type="submit"
            className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
            >
            <Save className="w-5 h-5" />
            ■■■■
        </button>
    </div>
    </form>
    </div>
    </div>
    );
}

```

■ File: src\components\questionnaires\PSSQuestionnaire.tsx

[illegible]

```

    </div>
    <h2 className="text-text-primary">PSS <div></div></h2>
    <p className="text-text-secondary">Perceived Stress Scale</p>
  </div>
</div>

<div className="bg-white rounded-xl shadow-lg p-6 border border-border">
  <form onSubmit={handleSubmit} className="space-y-6">
    <div className="bg-medical-blue p-4 rounded-lg">
      <p className="text-text-primary">
        <div></div>
      </p>
    </div>

    {questions.map((question, index) => (
      <div key={index} className="space-y-3 pb-4 border-b border-border
last:border-b-0">
        <label className="block text-text-primary">
          {index + 1}. {question}
        </label>
        <div className="grid grid-cols-5 gap-2">
          {options.map((option, optionIndex) => (
            <button
              key={optionIndex}
              type="button"
              onClick={() => {
                const newAnswers = [...answers];
                newAnswers[index] = optionIndex;
                setAnswers(newAnswers);
              }}
              className={`p-3 rounded-lg border-2 transition-all text-center ${
                answers[index] === optionIndex
                  ? 'bg-primary text-white border-primary'
                  : 'bg-white text-text-secondary border-border
hover:border-primary'
              }`}
            </button>
            <div className="hidden md:block">{option}</div>
            <div className="md:hidden">{optionIndex}</div>
          </div>
        </div>
      </div>
    ))}

    <div className="flex justify-between text-text-secondary md:hidden">
      <span><div></div></span>
      <span><div></div></span>
    </div>
  </form>
</div>

<div className="flex gap-3 justify-end">
  <button
    type="button"
    onClick={onBack}
    className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
  >
    <div></div>
  </button>
  <button
    type="submit"
    className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
  >
    <div></div>
    <div></div>
  </button>
</div>
</div>
);
}

```

■ File: src\components\questionnaires\WPIQuestionnaire.tsx

```
import { useState } from 'react';
import { ArrowLeft, Save } from 'lucide-react';

interface WPIQuestionnaireProps {
  onBack: () => void;
  onComplete: (score: number) => void;
}

export function WPIQuestionnaire({ onBack, onComplete }: WPIQuestionnaireProps) {
  const [selectedAreas, setSelectedAreas] = useState<string[]>([]);

  const bodyAreas = [
    { id: 'shoulder-left', label: '■' },
    { id: 'shoulder-right', label: '■' },
    { id: 'arm-left', label: '■' },
    { id: 'arm-right', label: '■' },
    { id: 'forearm-left', label: '■' },
    { id: 'forearm-right', label: '■' },
    { id: 'hip-left', label: '■' },
    { id: 'hip-right', label: '■' },
    { id: 'thigh-left', label: '■' },
    { id: 'thigh-right', label: '■' },
    { id: 'calf-left', label: '■' },
    { id: 'calf-right', label: '■' },
    { id: 'jaw-left', label: '■' },
    { id: 'jaw-right', label: '■' },
    { id: 'chest', label: '■' },
    { id: 'abdomen', label: '■' },
    { id: 'upper-back', label: '■' },
    { id: 'lower-back', label: '■' },
    { id: 'neck', label: '■' },
  ];

  const toggleArea = (areaId: string) => {
    setSelectedAreas((prev) => {
      prev.includes(areaId) ? prev.filter((id) => id !== areaId) : [...prev, areaId]
    });
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    const score = selectedAreas.length;
    alert(`WPI ██████\n\n██████████${score}/19\n\n██████████`);
    onComplete(score);
  };

  return (
    <div className="space-y-6">
      <div className="flex items-center gap-4">
        <button
          onClick={onBack}
          className="flex items-center gap-2 px-4 py-2 border border-border rounded-lg
          hover:bg-gray-50 transition-colors"
        >
          <ArrowLeft className="w-5 h-5" />
          ■
        </button>
      </div>
      <div>
        <h2 className="text-text-primary">WPI ██████</h2>
        <p className="text-text-secondary">Widespread Pain Index</p>
      </div>
      <div className="bg-white rounded-xl shadow-lg p-6 border border-border">
        <form onSubmit={handleSubmit} className="space-y-6">
          <div className="bg-medical-blue p-4 rounded-lg">
            <p className="text-text-primary">
              ████████████████████
            </p>
          </div>
          <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-3">
            {bodyAreas.map((area) => (
              <button
                key={area.id}
                type="button"
                onClick={() => toggleArea(area.id)}
                className={`p-4 rounded-lg border-2 transition-all ${
                  selectedAreas.includes(area.id)
                }`}
              >
                {area.label}
              </button>
            ))}
          </div>
        </form>
      </div>
    </div>
  );
}
```

```

        ? 'bg-primary text-white border-primary'
        : 'bg-white text-text-secondary border-border hover:border-primary'
    `}}
    &gt;
    {area.label}
    &lt;/button&gt;
  )}}
  &lt;/div&gt;

  { /* Score Display */ }
  &lt;div className="bg-gray-50 p-6 rounded-lg border-2 border-border"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;span className="text-text-secondary"&gt;■■■■■■&lt;/span&gt;
      &lt;span className="text-primary"&gt;{selectedAreas.length}/19&lt;/span&gt;
    &lt;/div&gt;
  &lt;/div&gt;

  &lt;div className="flex gap-3 justify-end"&gt;
    &lt;button
      type="button"
      onClick={onBack}
      className="px-6 py-2 border border-border rounded-lg hover:bg-gray-50
transition-colors"
    &gt;
      ■■
    &lt;/button&gt;
    &lt;button
      type="submit"
      className="flex items-center gap-2 px-6 py-2 bg-primary
hover:bg-primary-dark text-white rounded-lg transition-all shadow-md"
    &gt;
      &lt;Save className="w-5 h-5" /&gt;
      ■■■■
    &lt;/button&gt;
  &lt;/div&gt;
  &lt;/form&gt;
&lt;/div&gt;
&lt;/div&gt;
  );
}

```

■ File: src\components\ui\accordion.tsx

```
"use client";

import * as React from "react";
import * as AccordionPrimitive from "@radix-ui/react-accordion@1.2.3";
import { ChevronDownIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Accordion({
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Root> & {
  return <AccordionPrimitive.Root data-slot="accordion" {...props} />;
}) {

function AccordionItem({
  className,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Item> & {
  return (
    <AccordionPrimitive.Item
      data-slot="accordion-item"
      className={cn("border-b last:border-b-0", className)}
      {...props}
    />
  );
}

function AccordionTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Trigger> & {
  return (
    <AccordionPrimitive.Header className="flex" &gt;
      <AccordionPrimitive.Trigger
        data-slot="accordion-trigger"
        className={cn(
          "focus-visible:border-ring focus-visible:ring-ring/50 flex flex-1 items-start
          justify-between gap-4 rounded-md py-4 text-left text-sm font-medium transition-all
          outline-none hover:underline focus-visible:ring-[3px] disabled:pointer-events-none
          disabled:opacity-50 [&[data-state=open]&];svg):rotate-180",
          className,
        )}
        {...props}
      &gt;
        {children}
        <ChevronDownIcon className="text-muted-foreground pointer-events-none size-4
          shrink-0 translate-y-0.5 transition-transform duration-200" />
      </AccordionPrimitive.Trigger>
    </AccordionPrimitive.Header>
  );
}

function AccordionContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Content> & {
  return (
    <AccordionPrimitive.Content
      data-slot="accordion-content"
      className="data-[state=closed]:animate-accordion-up
      data-[state=open]:animate-accordion-down overflow-hidden text-sm"
      {...props}
    &gt;
      <div className={cn("pt-0 pb-4", className)}&gt;{children}</div>
    </AccordionPrimitive.Content>
  );
}

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent };
```


■ File: src/components/ui/alert-dialog.tsx

```
"use client";

import * as React from "react";
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog@1.1.6";

import { cn } from "../utils";
import { buttonVariants } from "../button";

function AlertDialog({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Root> & {
  return <AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} />;
}) {

function AlertDialogTrigger({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Trigger> & {
  return (
    <AlertDialogPrimitive.Trigger data-slot="alert-dialog-trigger" {...props} />
  );
}) {

function AlertDialogPortal({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Portal> & {
  return (
    <AlertDialogPrimitive.Portal data-slot="alert-dialog-portal" {...props} />
  );
}) {

function AlertDialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Overlay> & {
  return (
    <AlertDialogPrimitive.Overlay
      data-slot="alert-dialog-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
        bg-black/50",
        className,
      )}
      {...props}
    />
  );
}) {

function AlertDialogContent({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Content> & {
  return (
    <AlertDialogPortal>
      <AlertDialogOverlay />
      <AlertDialogPrimitive.Content
        data-slot="alert-dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
          data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 fixed top-[50%]
          left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)] translate-x-[-50%]
          translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200 sm:max-w-lg",
          className,
        )}
        {...props}
      />
    </AlertDialogPortal>
  );
}) {

function AlertDialogHeader({
  className,
  ...props
}: React.ComponentProps<"div"> & {
  return (
    <div
      data-slot="alert-dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
    >
```

```

        {...props}
      /&gt;
    );
  }

function AlertDialogFooter({
  className,
  ...props
}: React.ComponentProps<div>) {
  return (
    &lt;div
      data-slot="alert-dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Title>) {
  return (
    &lt;AlertDialogPrimitive.Title
      data-slot="alert-dialog-title"
      className={cn("text-lg font-semibold", className)}
      {...props}
    /&gt;
  );
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Description>) {
  return (
    &lt;AlertDialogPrimitive.Description
      data-slot="alert-dialog-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

function AlertDialogAction({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action>) {
  return (
    &lt;AlertDialogPrimitive.Action
      className={cn(buttonVariants(), className)}
      {...props}
    /&gt;
  );
}

function AlertDialogCancel({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Cancel>) {
  return (
    &lt;AlertDialogPrimitive.Cancel
      className={cn(buttonVariants({ variant: "outline" }), className)}
      {...props}
    /&gt;
  );
}

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
  AlertDialogAction,

```

```
    AlertDialogCancel,  
};
```

■ File: src/components/ui/alert.tsx

```
import * as React from "react";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "../utils";

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm grid",
  {
    has: [
      "&svg":grid-cols-[calc(var(--spacing)*4)_1fr] grid-cols-[0_1fr]
      has:[&svg]:gap-x-3 gap-y-0.5 items-start [&svg]:size-4 [&svg]:translate-y-0.5
      [&svg]:text-current",
    ],
    variants: {
      variant: {
        default: "bg-card text-card-foreground",
        destructive:
          "text-destructive bg-card [&svg]:text-current",
        *:data-[slot=alert-description]:text-destructive/90",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

function Alert({
  className,
  variant,
  ...props
}: React.ComponentProps<"div"& & VariantProps<typeof alertVariants> & ) {
  return (
    &lt;div
      data-slot="alert"
      role="alert"
      className={cn(alertVariants({ variant }), className)}
      {...props}
    /&gt;
  );
}

function AlertTitle({ className, ...props }: React.ComponentProps<"div"& & ) {
  return (
    &lt;div
      data-slot="alert-title"
      className={cn(
        "col-start-2 line-clamp-1 min-h-4 font-medium tracking-tight",
        className,
      )}
      {...props}
    /&gt;
  );
}

function AlertDescription({
  className,
  ...props
}: React.ComponentProps<"div"& & ) {
  return (
    &lt;div
      data-slot="alert-description"
      className={cn(
        "text-muted-foreground col-start-2 grid justify-items-start gap-1 text-sm",
        [&p]:leading-relaxed",
        className,
      )}
      {...props}
    /&gt;
  );
}

export { Alert, AlertTitle, AlertDescription };
```

■ File: src\components\ui\aspect-ratio.tsx

```
"use client";

import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio@1.1.2";

function AspectRatio({
  ...props
}: React.ComponentProps<typeof AspectRatioPrimitive.Root> & {
  return <AspectRatioPrimitive.Root data-slot="aspect-ratio" {...props} />;
}) {

export { AspectRatio };
```

■ File: src/components/ui/avatar.tsx

```
"use client";

import * as React from "react";
import * as AvatarPrimitive from "@radix-ui/react-avatar@1.1.3";

import { cn } from "../../utils";

function Avatar({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Root>) {
  return (
    <AvatarPrimitive.Root
      data-slot="avatar"
      className={cn(
        "relative flex size-10 shrink-0 overflow-hidden rounded-full",
        className,
      )}
      {...props}
    />
  );
}

function AvatarImage({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Image>) {
  return (
    <AvatarPrimitive.Image
      data-slot="avatar-image"
      className={cn("aspect-square size-full", className)}
      {...props}
    />
  );
}

function AvatarFallback({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Fallback>) {
  return (
    <AvatarPrimitive.Fallback
      data-slot="avatar-fallback"
      className={cn(
        "bg-muted flex size-full items-center justify-center rounded-full",
        className,
      )}
      {...props}
    />
  );
}

export { Avatar, AvatarImage, AvatarFallback };
```

■ File: src/components/ui/badge.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "../../utils";

const badgeVariants = cva(
  "inline-flex items-center justify-center rounded-md border px-2 py-0.5 text-xs font-medium w-fit whitespace-nowrap shrink-0 [&&svg]:size-3 gap-1 [&&svg]:pointer-events-none focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px] aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive transition-[color,box-shadow] overflow-hidden",
  {
    variants: {
      variant: {
        default: "border-transparent bg-primary text-primary-foreground [&&]:hover:bg-primary/90",
        secondary: "border-transparent bg-secondary text-secondary-foreground [&&]:hover:bg-secondary/90",
        destructive: "border-transparent bg-destructive text-white [&&]:hover:bg-destructive/90 focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40 dark:bg-destructive/60",
        outline: "text-foreground [&&]:hover:bg-accent [&&]:hover:text-accent-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  },
);

function Badge({
  className,
  variant,
  asChild = false,
  ...props
}: React.ComponentProps<"span"> & VariantProps<typeof badgeVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "span";

  return (
    <Comp
      data-slot="badge"
      className={cn(badgeVariants({ variant }), className)}
      {...props}
    />
  );
}

export { Badge, badgeVariants };
```

■ File: src/components/ui/breadcrumb.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { ChevronRight, MoreHorizontal } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Breadcrumb({ ...props }: React.ComponentProps<"nav">) {
  return <nav aria-label="breadcrumb" data-slot="breadcrumb" {...props} />;
}

function BreadcrumbList({ className, ...props }: React.ComponentProps<"ol">) {
  return (
    <ol
      data-slot="breadcrumb-list"
      className={cn(
        "text-muted-foreground flex flex-wrap items-center gap-1.5 text-sm break-words",
        sm:gap-2.5,
        className,
      )}
      {...props}
    />
  );
}

function BreadcrumbItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-item"
      className={cn("inline-flex items-center gap-1.5", className)}
      {...props}
    />
  );
}

function BreadcrumbLink({
  asChild,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean;
}) {
  const Comp = asChild ? Slot : "a";

  return (
    <Comp
      data-slot="breadcrumb-link"
      className={cn("hover:text-foreground transition-colors", className)}
      {...props}
    />
  );
}

function BreadcrumbPage({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-page"
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn("text-foreground font-normal", className)}
      {...props}
    />
  );
}

function BreadcrumbSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-separator"
      role="presentation"
      aria-hidden="true"
      className={cn("&#x2013; size-3.5", className)}
      {...props}
    />
    {children ?? <ChevronRight />}
  );
}
```



```

        </li>;
    );
}

function BreadcrumbEllipsis({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-ellipsis"
      role="presentation"
      aria-hidden="true"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    >
      <MoreHorizontal className="size-4" />
      <span className="sr-only">More</span>
    </span>
  );
}

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
};

```

■ File: src/components/ui/button.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "../../utils";

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-medium transition-all disabled:pointer-events-none disabled:opacity-50 [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4 shrink-0 [&_svg]:shrink-0 outline-none focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px] aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        destructive: "bg-destructive text-white hover:bg-destructive/90 focus-visible:ring-destructive/20 dark:focus-visible:ring-destructive/40 dark:bg-destructive/60",
        outline: "border bg-background text-foreground hover:bg-accent hover:text-accent-foreground dark:bg-input/30 dark:border-input dark:hover:bg-input/50",
        secondary: "bg-secondary text-secondary-foreground hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground dark:hover:bg-accent/50",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2 has-[:svg]:px-3",
        sm: "h-8 rounded-md gap-1.5 px-3 has-[:svg]:px-2.5",
        lg: "h-10 rounded-md px-6 has-[:svg]:px-4",
        icon: "size-9 rounded-md",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function Button({
  className,
  variant,
  size,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & VariantProps<typeof buttonVariants> & {
  asChild?: boolean;
}) {
  const Comp = asChild ? Slot : "button";

  return (
    <Comp
      data-slot="button"
      className={cn(buttonVariants({ variant, size, className }))}
      {...props}
    />
  );
}

export { Button, buttonVariants };
```

■ File: src\components\ui\calendar.tsx

```
"use client";

import * as React from "react";
import { ChevronLeft, ChevronRight } from "lucide-react@0.487.0";
import { DayPicker } from "react-day-picker@8.10.1";

import { cn } from "../utils";
import { buttonVariants } from "../button";

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  ...props
}: React.ComponentProps<typeof DayPicker>) {
  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn("p-3", className)}
      classNames={{
        months: "flex flex-col sm:flex-row gap-2",
        month: "flex flex-col gap-4",
        caption: "flex justify-center pt-1 relative items-center w-full",
        caption_label: "text-sm font-medium",
        nav: "flex items-center gap-1",
        nav_button: cn(
          buttonVariants({ variant: "outline" }),
          "size-7 bg-transparent p-0 opacity-50 hover:opacity-100",
        ),
        nav_button_previous: "absolute left-1",
        nav_button_next: "absolute right-1",
        table: "w-full border-collapse space-x-1",
        head_row: "flex",
        head_cell:
          "text-muted-foreground rounded-md w-8 font-normal text-[0.8rem]",
        row: "flex w-full mt-2",
        cell: cn(
          "relative p-0 text-center text-sm focus-within:z-20",
          [&:has([aria-selected])]:bg-accent [&:has([aria-selected].day-range-end)]:rounded-r-md",
          props.mode === "range"
            ? "[&:has(&:day-range-end)]:rounded-r-md"
            : "[&:has(&:day-range-start)]:rounded-l-md first:[&:has([aria-selected])]:rounded-l-md last:[&:has([aria-selected])]:rounded-r-md"
            : "[&:has([aria-selected])]:rounded-md",
        ),
        day: cn(
          buttonVariants({ variant: "ghost" }),
          "size-8 p-0 font-normal aria-selected:opacity-100",
        ),
        day_range_start:
          "day-range-start aria-selected:bg-primary aria-selected:text-primary-foreground",
        day_range_end:
          "day-range-end aria-selected:bg-primary aria-selected:text-primary-foreground",
        day_selected:
          "bg-primary text-primary-foreground hover:bg-primary",
        day_today: "bg-accent text-accent-foreground",
        day_outside:
          "day-outside text-muted-foreground aria-selected:text-muted-foreground",
        day_disabled: "text-muted-foreground opacity-50",
        day_range_middle:
          "aria-selected:bg-accent aria-selected:text-accent-foreground",
        day_hidden: "invisible",
        ...classNames,
      }}
      components={{
        IconLeft: ({ className, ...props }) => (
          <ChevronLeft className={cn("size-4", className)} {...props} />
        ),
        IconRight: ({ className, ...props }) => (
          <ChevronRight className={cn("size-4", className)} {...props} />
        ),
      }}
      {...props}
    />
  );
}
```

```
export { Calendar };
```

■ File: src/components/ui/card.tsx

```
import * as React from "react";

import { cn } from "../../utils";

function Card({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="card"
      className={cn(
        "bg-card text-card-foreground flex flex-col gap-6 rounded-xl border",
        className,
      )}
      {...props}
    />
  );
}

function CardHeader({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="card-header"
      className={cn(
        "container/card-header grid auto-rows-min grid-rows-[auto_auto] items-start gap-1.5 px-6 pt-6 has-data-[slot=card-action]:grid-cols-[1fr_auto] [.border-b]:pb-6",
        className,
      )}
      {...props}
    />
  );
}

function CardTitle({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <h4
      data-slot="card-title"
      className={cn("leading-none", className)}
      {...props}
    />
  );
}

function CardDescription({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <p
      data-slot="card-description"
      className={cn("text-muted-foreground", className)}
      {...props}
    />
  );
}

function CardAction({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className,
      )}
      {...props}
    />
  );
}

function CardContent({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="card-content"
      className={cn("px-6 [&:last-child]:pb-6", className)}
      {...props}
    />
  );
}

function CardFooter({ className, ...props }: React.ComponentProps<"div">): React.ComponentProps<"div"> {
  return (
    <div
      data-slot="card-footer"
      className={cn("flex items-center px-6 pb-6 [.border-t]:pt-6", className)}
    />
  );
}
```

```
        {...props}
      /&gt;
    );
  }

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
  CardContent,
};
```

■ File: src/components/ui/carousel.tsx

```
"use client";

import * as React from "react";
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from "embla-carousel-react@8.6.0";
import { ArrowLeft, ArrowRight } from "lucide-react@0.487.0";

import { cn } from "../../utils";
import { Button } from "../../button";

type CarouselApi = UseEmblaCarouselType[1];
type UseCarouselParameters = Parameters<typeof useEmblaCarousel>;
type CarouselOptions = UseCarouselParameters[0];
type CarouselPlugin = UseCarouselParameters[1];

type CarouselProps = {
  opts?: CarouselOptions;
  plugins?: CarouselPlugin;
  orientation?: "horizontal" | "vertical";
  setApi?: (api: CarouselApi) => void;
};

type CarouselContextProps = {
  carouselRef: ReturnType<typeof useEmblaCarousel>[0];
  api: ReturnType<typeof useEmblaCarousel>[1];
  scrollPrev: () => void;
  scrollNext: () => void;
  canScrollPrev: boolean;
  canScrollNext: boolean;
} & CarouselProps;

const CarouselContext = React.createContext<CarouselContextProps | null>(null);

function useCarousel() {
  const context = React.useContext(CarouselContext);

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />");
  }

  return context;
}

function Carousel({
  orientation = "horizontal",
  opts,
  setApi,
  plugins,
  className,
  children,
  ...props
}: React.ComponentProps<"div"> & CarouselProps) {
  const [carouselRef, api] = useEmblaCarousel(
    {
      ...opts,
      axis: orientation === "horizontal" ? "x" : "y",
    },
    plugins,
  );
  const [canScrollPrev, setCanScrollPrev] = React.useState(false);
  const [canScrollNext, setCanScrollNext] = React.useState(false);

  const onSelect = React.useCallback((api: CarouselApi) => {
    if (!api) return;
    setCanScrollPrev(api.canScrollPrev());
    setCanScrollNext(api.canScrollNext());
  }, []);

  const scrollPrev = React.useCallback(() => {
    api?.scrollPrev();
  }, [api]);

  const scrollNext = React.useCallback(() => {
    api?.scrollNext();
  }, [api]);

  const handleKeyDown = React.useCallback(
    (event: React.KeyboardEvent<HTMLDivElement>) => {

```

```

    if (event.key === "ArrowLeft") {
      event.preventDefault();
      scrollPrev();
    } else if (event.key === "ArrowRight") {
      event.preventDefault();
      scrollNext();
    }
  },
  [scrollPrev, scrollNext],
);

React.useEffect(() => {
  if (!api || !setApi) return;
  setApi(api);
}, [api, setApi]);

React.useEffect(() => {
  if (!api) return;
  onSelect(api);
  api.on("reInit", onSelect);
  api.on("select", onSelect);

  return () => {
    api?.off("select", onSelect);
  };
}, [api, onSelect]);

return (
  <CarouselContext.Provider
    value={{
      carouselRef,
      api: api,
      opts,
      orientation:
        orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
      scrollPrev,
      scrollNext,
      canScrollPrev,
      canScrollNext,
    }}
  >
    <div
      onKeyDownCapture={handleKeyDown}
      className={cn("relative", className)}
      role="region"
      aria-roledescription="carousel"
      data-slot="carousel"
      {...props}
    >
      <div>
        {children}
      </div>
    </CarouselContext.Provider>
  >
);
}

function CarouselContent({ className, ...props }: React.ComponentProps<"div">) {
  const { carouselRef, orientation } = useCarousel();

  return (
    <div
      ref={carouselRef}
      className="overflow-hidden"
      data-slot="carousel-content"
    >
      <div
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className,
        )}
        {...props}
      />
    </div>
  );
}

function CarouselItem({ className, ...props }: React.ComponentProps<"div">) {
  const { orientation } = useCarousel();

  return (
    <div
      role="group"

```



```

        aria-roledescription="slide"
        data-slot="carousel-item"
        className={cn(
            "min-w-0 shrink-0 grow-0 basis-full",
            orientation === "horizontal" ? "pl-4" : "pt-4",
            className,
        )}
        {...props}
    />
);
}

function CarouselPrevious({
    className,
    variant = "outline",
    size = "icon",
    ...props
}: React.ComponentProps<typeof Button>) {
    const { orientation, scrollPrev, canScrollPrev } = useCarousel();

    return (
        <Button
            data-slot="carousel-previous"
            variant={variant}
            size={size}
            className={cn(
                "absolute size-8 rounded-full",
                orientation === "horizontal"
                    ? "top-1/2 -left-12 -translate-y-1/2"
                    : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
                className,
            )}
            disabled={!canScrollPrev}
            onClick={scrollPrev}
            {...props}
        >
            <ArrowLeft />
            <span className="sr-only">Previous slide</span>
        </Button>
    );
}

function CarouselNext({
    className,
    variant = "outline",
    size = "icon",
    ...props
}: React.ComponentProps<typeof Button>) {
    const { orientation, scrollNext, canScrollNext } = useCarousel();

    return (
        <Button
            data-slot="carousel-next"
            variant={variant}
            size={size}
            className={cn(
                "absolute size-8 rounded-full",
                orientation === "horizontal"
                    ? "top-1/2 -right-12 -translate-y-1/2"
                    : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",
                className,
            )}
            disabled={!canScrollNext}
            onClick={scrollNext}
            {...props}
        >
            <ArrowRight />
            <span className="sr-only">Next slide</span>
        </Button>
    );
}

export {
    type CarouselApi,
    Carousel,
    CarouselContent,
    CarouselItem,
    CarouselPrevious,
    CarouselNext,
};

```

■ File: src/components/ui/chart.tsx

```
"use client";

import * as React from "react";
import * as RechartsPrimitive from "recharts@2.15.2";

import { cn } from "../../utils";

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const;

export type ChartConfig = {
  [k in string]: {
    label?: React.ReactNode;
    icon?: React.ComponentType;
  } & (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record<keyof typeof THEMES, string>; }
  );
};

type ChartContextProps = {
  config: ChartConfig;
};

const ChartContext = React.createContext<ChartContextProps | null>(null);

function useChart() {
  const context = React.useContext(ChartContext);

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />");
  }

  return context;
}

function ChartContainer({
  id,
  className,
  children,
  config,
  ...props
}: React.ComponentProps<"div"> & {
  config: ChartConfig;
  children: React.ComponentProps<
    typeof RechartsPrimitive.ResponsiveContainer
  >["children"];
}) {
  const uniqueId = React.useId();
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`;

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-slot="chart"
        data-chart={chartId}
        className={cn(
          "[&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground",
          [&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50",
          [&_.recharts-curve.recharts-tooltip-cursor]:stroke-border",
          [&_.recharts-polar-grid_[stroke='#ccc']]:stroke-border",
          [&_.recharts-radial-bar-background-sector]:fill-muted",
          [&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted",
          [&_.recharts-reference_line_[stroke='#ccc']]:stroke-border flex aspect-video",
          justify-center text-xs [&_.recharts-dot[stroke='#fff']]:stroke-transparent",
          [&_.recharts-layer]:outline-hidden [&_.recharts-sector]:outline-hidden",
          [&_.recharts-sector[stroke='#fff']]:stroke-transparent",
          [&_.recharts-surface]:outline-hidden",
          className,
        )}
        {...props}
      >
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>
  );
}
```

```

}

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(
    ([, config]) => config.theme || config.color,
  );

  if (!colorConfig.length) {
    return null;
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
            ([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) => {
    const color =
      itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ||
      itemConfig.color;
    return color ? `  --color-${key}: ${color};` : null;
  })
  .join("\n")}
`,
    )
    .join("\n"),
      }
    />
  );
};

```

```

const ChartTooltip = RechartsPrimitive.Tooltip;

function ChartTooltipContent({
  active,
  payload,
  className,
  indicator = "dot",
  hideLabel = false,
  hideIndicator = false,
  label,
  labelFormatter,
  labelClassName,
  formatter,
  color,
  nameKey,
  labelKey,
}: React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
  React.ComponentProps<"div"> & {
  hideLabel?: boolean;
  hideIndicator?: boolean;
  indicator?: "line" | "dot" | "dashed";
  nameKey?: string;
  labelKey?: string;
}) {
  const { config } = useChart();

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null;
    }

    const [item] = payload;
    const key = `${labelKey || item?.dataKey || item?.name || "value"}`;
    const itemConfig = getPayloadConfigFromPayload(config, item, key);
    const value =
      !labelKey && typeof label === "string"
        ? config[label as keyof typeof config]?.label || label
        : itemConfig?.label;

    if (labelFormatter) {
      return (
        <div className={cn("font-medium", labelClassName)}>
          {labelFormatter(value, payload)}
        </div>
      );
    }
  });
}

```

```

    if (!value) {
      return null;
    }

    return <div className={cn("font-medium", labelClassName)}>{value}</div>;
  }, [
    label,
    labelFormatter,
    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
  ]);

  if (!active || !payload?.length) {
    return null;
  }

  const nestLabel = payload.length === 1 && indicator !== "dot";

  return (
    <div
      className={cn(
        "border-border/50 bg-background grid min-w-[8rem] items-start gap-1.5 rounded-lg
        border px-2.5 py-1.5 text-xs shadow-xl",
        className,
      )}
    >
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload.map((item, index) => {
          const key = `${nameKey || item.name || item.dataKey || "value"}`;
          const itemConfig = getPayloadConfigFromPayload(config, item, key);
          const indicatorColor = color || item.payload.fill || item.color;

          return (
            <div
              key={item.dataKey}
              className={cn(
                "[&&svg]:text-muted-foreground flex w-full flex-wrap items-stretch gap-2
                [&&svg]:h-2.5 [&&svg]:w-2.5",
                indicator === "dot" && "items-center",
              )}
            >
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <&&
                {itemConfig?.icon ? (
                  <itemConfig.icon />
                ) : (
                  !hideIndicator && (
                    <div
                      className={cn(
                        "shrink-0 rounded-[2px] border(--color-border)
                        bg(--color-bg)",
                      )}
                    {
                      "h-2.5 w-2.5": indicator === "dot",
                      "w-1": indicator === "line",
                      "w-0 border-[1.5px] border-dashed bg-transparent":
                        indicator === "dashed",
                      "my-0.5": nestLabel && indicator === "dashed",
                    }
                  )}
                  style={
                    {
                      "--color-bg": indicatorColor,
                      "--color-border": indicatorColor,
                    } as React.CSSProperties
                  }
                />
              )}
            </div>
          )}
        )}
      </div>
      <div className={cn(
        "flex flex-1 justify-between leading-none",
        nestLabel ? "items-end" : "items-center",
      )}
    >
      <div className="grid gap-1.5">
        {nestLabel ? tooltipLabel : null}

```

```

        <span className="text-muted-foreground">
          {itemConfig?.label || item.name}
        </span>
      </div>
      {item.value & & (
        <span className="text-foreground font-mono font-medium
tabular-nums">
          {item.value.toLocaleString()}
        </span>
      )}
    </div>
  </div>
)
);
}}}
</div>
</div>
);
}

const ChartLegend = RechartsPrimitive.Legend;

function ChartLegendContent({
  className,
  hideIcon = false,
  payload,
  verticalAlign = "bottom",
  nameKey,
}: React.ComponentProps<"div"> &
Pick<RechartsPrimitive.LegendProps, "payload" | "verticalAlign"> & {
  hideIcon?: boolean;
  nameKey?: string;
}) {
  const { config } = useChart();

  if (!payload?.length) {
    return null;
  }

  return (
    <div
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className,
      )}
    >
      {payload.map((item) => {
        const key = `${nameKey} || item.dataKey || "value"`;
        const itemConfig = getPayloadConfigFromPayload(config, item, key);

        return (
          <div
            key={item.value}
            className={cn(
              "[&svg]:text-muted-foreground flex items-center gap-1.5 [&svg]:h-3
[&svg]:w-3",
            )}
          >
            {itemConfig?.icon & !hideIcon ? (
              <itemConfig.icon />
            ) : (
              <div
                className="h-2 w-2 shrink-0 rounded-[2px]"
                style={{
                  backgroundColor: item.color,
                }}
              />
            )}
            {itemConfig?.label}
          </div>
        );
      })}
    </div>
  );
}

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
  config: ChartConfig,
  payload: unknown,

```

```

    key: string,
  ) {
    if (typeof payload !== "object" || payload === null) {
      return undefined;
    }

    const payloadPayload =
      "payload" in payload &&&
      typeof payload.payload === "object" &&&
      payload.payload !== null
      ? payload.payload
      : undefined;

    let configLabelKey: string = key;

    if (
      key in payload &&&
      typeof payload[key as keyof typeof payload] === "string"
    ) {
      configLabelKey = payload[key as keyof typeof payload] as string;
    } else if (
      payloadPayload &&&
      key in payloadPayload &&&
      typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
    ) {
      configLabelKey = payloadPayload[
        key as keyof typeof payloadPayload
      ] as string;
    }

    return configLabelKey in config
      ? config[configLabelKey]
      : config[key as keyof typeof config];
  }

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
};

```

■ File: src/components/ui/lcheckbox.tsx

```
"use client";

import * as React from "react";
import * as CheckboxPrimitive from "@radix-ui/react-checkbox@1.1.4";
import { CheckIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Checkbox({
  className,
  ...props
}: React.ComponentProps<typeof CheckboxPrimitive.Root>) {
  return (
    <CheckboxPrimitive.Root
      data-slot="checkbox"
      className={cn(
        "peer border bg-input-background dark:bg-input/30 data-[state=checked]:bg-primary
        data-[state=checked]:text-primary-foreground dark:data-[state=checked]:bg-primary
        data-[state=checked]:border-primary focus-visible:border-ring
        focus-visible:ring-ring/50 aria-invalid:ring-destructive/20
        dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive size-4
        shrink-0 rounded-[4px] border shadow-xs transition-shadow outline-none
        focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    >
      <CheckboxPrimitive.Indicator
        data-slot="checkbox-indicator"
        className="flex items-center justify-center text-current transition-none"
      >
        <CheckIcon className="size-3.5" />
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  );
}

export { Checkbox };
```

■ File: src\components\ui\collapsible.tsx

```
"use client";

import * as CollapsiblePrimitive from "@radix-ui/react-collapsible@1.1.3";

function Collapsible({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.Root> & {
  return <CollapsiblePrimitive.Root data-slot="collapsible" {...props} /&gt;;
}) {

  function CollapsibleTrigger({
    ...props
  }: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleTrigger>) {
    return (
      <CollapsiblePrimitive.CollapsibleTrigger
        data-slot="collapsible-trigger"
        {...props}
      /&gt;
    );
  }

  function CollapsibleContent({
    ...props
  }: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleContent>) {
    return (
      <CollapsiblePrimitive.CollapsibleContent
        data-slot="collapsible-content"
        {...props}
      /&gt;
    );
  }

  export { Collapsible, CollapsibleTrigger, CollapsibleContent };
```


■ File: src\components\ui\command.tsx

```
"use client";

import * as React from "react";
import { Command as CommandPrimitive } from "cmdk@1.1.1";
import { SearchIcon } from "lucide-react@0.487.0";

import { cn } from "../utils";
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "../dialog";

function Command({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive> & {
  return (
    <CommandPrimitive
      data-slot="command"
      className={cn(
        "bg-popover text-popover-foreground flex h-full w-full flex-col overflow-hidden
        rounded-md",
        className,
      )}
      {...props}
    />
  );
});

function CommandDialog({
  title = "Command Palette",
  description = "Search for a command to run...",
  children,
  ...props
}: React.ComponentProps<typeof Dialog> & {
  title?: string;
  description?: string;
}) {
  return (
    <Dialog {...props}>
      <DialogHeader className="sr-only">
        <DialogTitle>{title}</DialogTitle>
        <DialogDescription>{description}</DialogDescription>
      </DialogHeader>
      <DialogContent className="overflow-hidden p-0">
        <Command className="[&_[:cmdk-group-heading]]:text-muted-foreground
        **:data-[slot=command-input-wrapper]:h-12 [&_[:cmdk-group-heading]]:px-2
        [&_[:cmdk-group-heading]]:font-medium [&_[:cmdk-group]]:px-2
        [&_[:cmdk-group]]:not([hidden])_~[:cmdk-group]:pt-0 [&_[:cmdk-input-wrapper]]:h-5
        [&_[:cmdk-input-wrapper]]_svg:w-5 [&_[:cmdk-input]]:h-12 [&_[:cmdk-item]]:px-2
        [&_[:cmdk-item]]:py-3 [&_[:cmdk-item]]_svg:h-5 [&_[:cmdk-item]]_svg:w-5">
          {children}
        </Command>
      </DialogContent>
    </Dialog>
  );
}

function CommandInput({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Input> & {
  return (
    <div
      data-slot="command-input-wrapper"
      className="flex h-9 items-center gap-2 border-b px-3"
    >
      <SearchIcon className="size-4 shrink-0 opacity-50" />
      <CommandPrimitive.Input
        data-slot="command-input"
        className={cn(
          "placeholder:text-muted-foreground flex h-10 w-full rounded-md bg-transparent
          py-3 text-sm outline-hidden disabled:cursor-not-allowed disabled:opacity-50",
          className,
        )}
        {...props}
      />
    </div>
  );
}
```

```

    /&gt;
    &lt;/div&gt;
  );
}

function CommandList({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.List&gt;) {
  return (
    &lt;CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        "max-h-[300px] scroll-py-1 overflow-x-hidden overflow-y-auto",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CommandEmpty({
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Empty&gt;) {
  return (
    &lt;CommandPrimitive.Empty
      data-slot="command-empty"
      className="py-6 text-center text-sm"
      {...props}
    /&gt;
  );
}

function CommandGroup({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Group&gt;) {
  return (
    &lt;CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        "text-foreground [&_&#9494;command-group-heading]:text-muted-foreground overflow-hidden
p-1 [&_&#9494;command-group-heading]:px-2 [&_&#9494;command-group-heading]:py-1.5
&_&#9494;command-group-heading]:text-xs [&_&#9494;command-group-heading]:font-medium",
        className,
      )}
      {...props}
    /&gt;
  );
}

function CommandSeparator({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Separator&gt;) {
  return (
    &lt;CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn("bg-border -mx-1 h-px", className)}
      {...props}
    /&gt;
  );
}

function CommandItem({
  className,
  ...props
}: React.ComponentProps&lt;typeof CommandPrimitive.Item&gt;) {
  return (
    &lt;CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground
&_&#9494;svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
data-[disabled=true]:pointer-events-none data-[disabled=true]:opacity-50
&_&#9494;svg]:pointer-events-none [&_&#9494;svg]:shrink-0 [&_&#9494;svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    /&gt;
  );
}

```

```

}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="command-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    />
  );
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
};

```

■ File: src\components\ui\context-menu.tsx

```

use client";

import * as React from "react";
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu@2.2.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "../utils";

function ContextMenu({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Root> & {
  return <ContextMenuPrimitive.Root data-slot="context-menu" {...props} /&gt;;
}) {}

function ContextMenuTrigger({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Trigger> & {
  return (
    <ContextMenuPrimitive.Trigger data-slot="context-menu-trigger" {...props} /&gt;;
  );
}) {}

function ContextMenuGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Group> & {
  return (
    <ContextMenuPrimitive.Group data-slot="context-menu-group" {...props} /&gt;;
  );
}) {}

function ContextMenuPortal({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Portal> & {
  return (
    <ContextMenuPrimitive.Portal data-slot="context-menu-portal" {...props} /&gt;;
  );
}) {}

function ContextMenuSub({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Sub> & {
  return <ContextMenuPrimitive.Sub data-slot="context-menu-sub" {...props} /&gt;;
}) {}

function ContextMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioGroup> & {
  return (
    <ContextMenuPrimitive.RadioGroup
      data-slot="context-menu-radio-group"
      {...props}
    /&gt;;
  );
}) {}

function ContextMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.SubTrigger> & {
  inset?: boolean;
}) {
  return (
    <ContextMenuPrimitive.SubTrigger
      data-slot="context-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
        px-2 py-1.5 text-sm outline-hidden select-none data-[inset]:pl-8
        [&_&svg]:pointer-events-none [&_&svg]:shrink-0 [&_&svg]:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    &gt;
      {children}
      <ChevronRightIcon className="ml-auto" /&gt;
    </ContextMenuPrimitive.SubTrigger>
  );
}

```

```

    );
}

function ContextMenuSubContent({
  className,
  ...props
}): React.ComponentProps<typeof ContextMenuPrimitive.SubContent> {
  return (
    <ContextMenuPrimitive.SubContent
      data-slot="context-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
        data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin(--radix-context-menu-content-transform-origin) overflow-hidden rounded-md
        border p-1 shadow-lg",
        className,
      )}
      {...props}
    />
  );
}

function ContextMenuContent({
  className,
  ...props
}): React.ComponentProps<typeof ContextMenuPrimitive.Content> {
  return (
    <ContextMenuPrimitive.Portal>
      <ContextMenuPrimitive.Content
        data-slot="context-menu-content"
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
          data-[state=closed]:animate-out data-[state=closed]:fade-out-0
          data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50
          max-h-(--radix-context-menu-content-available-height) min-w-[8rem]
          origin(--radix-context-menu-content-transform-origin) overflow-x-hidden
          overflow-y-auto rounded-md border p-1 shadow-md",
          className,
        )}
        {...props}
      />
    </ContextMenuPrimitive.Portal>
  );
}

function ContextMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}): React.ComponentProps<typeof ContextMenuPrimitive.Item> & {
  inset?: boolean;
  variant?: "default" | "destructive";
} {
  return (
    <ContextMenuPrimitive.Item
      data-slot="context-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
        data-[variant=destructive]:text-destructive
        data-[variant=destructive]:focus:bg-destructive/10
        dark:data-[variant=destructive]:focus:bg-destructive/20
        data-[variant=destructive]:focus:text-destructive
        data-[variant=destructive]:*[svg]:!text-destructive
        [&_svg:not([class*='text-']):text-muted-foreground relative flex cursor-default
        items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-']):size-4",
        className,
      )}
      {...props}
    />
  );
}

```

```

}

function ContextMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.CheckboxItem> & {
  checked?: boolean
}) {
  return (
    <ContextMenuPrimitive.CheckboxItem
      data-slot="context-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
        items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
      justify-center">
        <ContextMenuPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.CheckboxItem>
  );
}

function ContextMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioItem> & {
  checked?: boolean
}) {
  return (
    <ContextMenuPrimitive.RadioItem
      data-slot="context-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
        items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
      justify-center">
        <ContextMenuPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.RadioItem>
  );
}

function ContextMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.Label
      data-slot="context-menu-label"
      data-inset={inset}
      className={cn(
        "text-foreground px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    >
      {children}
    </ContextMenuPrimitive.Label>
  );
}

function ContextMenuSeparator({
  className,
  ...props
}) {
  return (
    <ContextMenuPrimitive.Separator
      data-slot="context-menu-separator"
      className={cn(
        "border-t border-t-border px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    >
      {children}
    </ContextMenuPrimitive.Separator>
  );
}

```

```

    ...props
  }: React.ComponentProps<typeof ContextMenuPrimitive.Separator> ) {
    return (
      <ContextMenuPrimitive.Separator
        data-slot="context-menu-separator"
        className={cn("bg-border -mx-1 my-1 h-px", className)}
        {...props}
      />
    );
  }

function ContextMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="context-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    />
  );
}

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
};

```

■ File: src/components/ui/dialog.tsx

```
"use client";

import * as React from "react";
import * as DialogPrimitive from "@radix-ui/react-dialog@1.1.6";
import { XIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Dialog({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Root> & {
  return <DialogPrimitive.Root data-slot="dialog" {...props} />;
}) {

function DialogTrigger({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Trigger> & {
  return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />;
}) {

function DialogPortal({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Portal> & {
  return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />;
}) {

function DialogClose({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Close> & {
  return <DialogPrimitive.Close data-slot="dialog-close" {...props} />;
}) {

function DialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Overlay> & {
  return (
    <DialogPrimitive.Overlay
      data-slot="dialog-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
        bg-black/50",
        className,
      )}
      {...props}
    />
  );
}

function DialogContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Content> & {
  return (
    <DialogPortal data-slot="dialog-portal">
      <DialogOverlay />
      <DialogPrimitive.Content
        data-slot="dialog-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
          data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 fixed top-[50%]
          left-[50%] z-50 grid w-full max-w-[calc(100%-2rem)] translate-x-[-50%]
          translate-y-[-50%] gap-4 rounded-lg border p-6 shadow-lg duration-200 sm:max-w-lg",
          className,
        )}
        {...props}
      >
        {children}
        <DialogPrimitive.Close className="ring-offset-background focus:ring-ring
        data-[state=open]:bg-accent data-[state=open]:text-muted-foreground absolute top-4
        right-4 rounded-xs opacity-70 transition-opacity hover:opacity-100 focus:ring-2
        focus:ring-offset-2 focus:outline-hidden disabled:pointer-events-none
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-'])]:size-4">
          <XIcon />
          <span className="sr-only">Close</span>
        </DialogPrimitive.Close>
      </DialogPrimitive.Content>
    </DialogPortal>
  );
}
```



```

        <DialogPrimitive.Content>
    </DialogPortal>
  );
}

function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-2 text-center sm:text-left", className)}
      {...props}
    />
  );
}

function DialogFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-2 sm:flex-row sm:justify-end",
        className,
      )}
      {...props}
    />
  );
}

function DialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Title>) {
  return (
    <DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn("text-lg leading-none font-variant", className)}
      {...props}
    />
  );
}

function DialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Description>) {
  return (
    <DialogPrimitive.Description
      data-slot="dialog-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    />
  );
}

export {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogOverlay,
  DialogPortal,
  DialogTitle,
  DialogTrigger,
};

```

■ File: src/components/ui/drawer.tsx

```
"use client";

import * as React from "react";
import { Drawer as DrawerPrimitive } from "vaul@1.1.2";

import { cn } from "../../utils";

function Drawer({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root> & {
  return <DrawerPrimitive.Root data-slot="drawer" {...props} />;
}) {

  function DrawerTrigger({
    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Trigger>) {
    return <DrawerPrimitive.Trigger data-slot="drawer-trigger" {...props} />;
  }

  function DrawerPortal({
    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Portal>) {
    return <DrawerPrimitive.Portal data-slot="drawer-portal" {...props} />;
  }

  function DrawerClose({
    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Close>) {
    return <DrawerPrimitive.Close data-slot="drawer-close" {...props} />;
  }

  function DrawerOverlay({
    className,
    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Overlay>) {
    return (
      <DrawerPrimitive.Overlay
        data-slot="drawer-overlay"
        className={cn(
          "data-[state=open]:animate-in data-[state=closed]:animate-out
          data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
          bg-black/50",
          className,
        )}
        {...props}
      />
    );
  }

  function DrawerContent({
    className,
    children,
    ...props
  }: React.ComponentProps<typeof DrawerPrimitive.Content>) {
    return (
      <DrawerPortal data-slot="drawer-portal">
        <DrawerOverlay />
        <DrawerPrimitive.Content
          data-slot="drawer-content"
          className={cn(
            "group/drawer-content bg-background fixed z-50 flex h-auto flex-col",
            "data-[vaul-drawer-direction=top]:inset-x-0
            data-[vaul-drawer-direction=top]:top-0 data-[vaul-drawer-direction=top]:mb-24
            data-[vaul-drawer-direction=top]:max-h-[80vh]
            data-[vaul-drawer-direction=top]:rounded-b-lg
            data-[vaul-drawer-direction=top]:border-b",
            "data-[vaul-drawer-direction=bottom]:inset-x-0
            data-[vaul-drawer-direction=bottom]:bottom-0
            data-[vaul-drawer-direction=bottom]:mt-24
            data-[vaul-drawer-direction=bottom]:max-h-[80vh]
            data-[vaul-drawer-direction=bottom]:rounded-t-lg
            data-[vaul-drawer-direction=bottom]:border-t",
            "data-[vaul-drawer-direction=right]:inset-y-0
            data-[vaul-drawer-direction=right]:right-0 data-[vaul-drawer-direction=right]:w-3/4
            data-[vaul-drawer-direction=right]:border-l
            data-[vaul-drawer-direction=right]:sm:max-w-sm",
            "data-[vaul-drawer-direction=left]:inset-y-0
            data-[vaul-drawer-direction=left]:left-0 data-[vaul-drawer-direction=left]:w-3/4
            data-[vaul-drawer-direction=left]:border-r
          )}
        />
      </DrawerPortal>
    );
  }
}
```

```

    data-[vault-drawer-direction=left]:sm:max-w-sm",
    className,
  )}
  {...props}
  &gt;
  &lt;div className="bg-muted mx-auto mt-4 hidden h-2 w-[100px] shrink-0 rounded-full
group-data-[vault-drawer-direction=bottom]/drawer-content:block" /&gt;
    {children}
  &lt;/DrawerPrimitive.Content&gt;
  &lt;/DrawerPortal&gt;
);
}

function DrawerHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="drawer-header"
      className={cn("flex flex-col gap-1.5 p-4", className)}
      {...props}
    /&gt;
  );
}

function DrawerFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="drawer-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    /&gt;
  );
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Title&gt;) {
  return (
    &lt;DrawerPrimitive.Title
      data-slot="drawer-title"
      className={cn("text-foreground font-semibold", className)}
      {...props}
    /&gt;
  );
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof DrawerPrimitive.Description&gt;) {
  return (
    &lt;DrawerPrimitive.Description
      data-slot="drawer-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
};

```

■ File: src/components/ui/dropdown-menu.tsx

```
"use client";

import * as React from "react";
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu@2.1.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function DropdownMenu({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Root>) {
  return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />;
}

function DropdownMenuPortal({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Portal> & {
  return (
    <DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />;
  );
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger>) {
  return (
    <DropdownMenuPrimitive.Trigger
      data-slot="dropdown-menu-trigger"
      {...props}
    />;
  );
}

function DropdownMenuContent({
  className,
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Content> & {
  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        data-slot="dropdown-menu-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
          data-[state=closed]:animate-out data-[state=closed]:fade-out-0
          data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
          data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50
          max-h-(--radix-dropdown-menu-content-available-height) min-w-[8rem]
          origin-(--radix-dropdown-menu-content-transform-origin) overflow-x-hidden
          overflow-y-auto rounded-md border p-1 shadow-md",
          className,
        )}
        {...props}
      />;
    </DropdownMenuPrimitive.Portal>
  );
}

function DropdownMenuGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Group>) {
  return (
    <DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />;
  );
}

function DropdownMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
  inset?: boolean;
  variant?: "default" | "destructive";
}) {
  return (
```

```

    <DropdownMenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
        data-[variant=destructive]:text-destructive
        data-[variant=destructive]:focus:bg-destructive/10
        dark:data-[variant=destructive]:focus:bg-destructive/20
        data-[variant=destructive]:focus:text-destructive
        data-[variant=destructive]:*[svg]:!text-destructive
        [&_svg:not([class*='text-'])]:text-muted-foreground relative flex cursor-default
        items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    />
  );
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem>) {
  return (
    <DropdownMenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
        items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
      justify-center">
        <DropdownMenuPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.CheckboxItem>
  );
}

function DropdownMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioGroup>) {
  return (
    <DropdownMenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    />
  );
}

function DropdownMenuRadioItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioItem>) {
  return (
    <DropdownMenuPrimitive.RadioItem
      data-slot="dropdown-menu-radio-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
        items-center gap-2 rounded-sm py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
        data-[disabled]:pointer-events-none data-[disabled]:opacity-50
        [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className,
      )}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
      justify-center">

```

```

        <DropdownMenuPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
    <children>
    </DropdownMenuPrimitive.RadioItem>
  );
}

function DropdownMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Label> & {
  inset?: boolean;
}) {
  return (
    <DropdownMenuPrimitive.Label
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    />
  );
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>) {
  return (
    <DropdownMenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    />
  );
}

function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    />
  );
}

function DropdownMenuSub({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Sub>) {
  return <DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} />;
}

function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubTrigger> & {
  inset?: boolean;
}) {
  return (
    <DropdownMenuPrimitive.SubTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
        px-2 py-1.5 text-sm outline-hidden select-none data-[inset]:pl-8",
        className,
      )}
    >
      {children}
    </DropdownMenuPrimitive.SubTrigger>
  );
}

```

```

    {...props}
    &gt;
    {children}
    &lt;ChevronRightIcon className="ml-auto size-4" /&gt;
    &lt;/DropdownMenuPrimitive.SubTrigger&gt;
  );
}

function DropdownMenuSubContent({
  className,
  ...props
}: React.ComponentProps&lt;typeof DropdownMenuPrimitive.SubContent&gt;) {
  return (
    &lt;DropdownMenuPrimitive.SubContent
      data-slot="dropdown-menu-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
        data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin(--radix-dropdown-menu-content-transform-origin) overflow-hidden rounded-md
        border p-1 shadow-lg",
        className,
      )}
      {...props}
    /&gt;
  );
}

export {
  DropdownMenu,
  DropdownMenuPortal,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuLabel,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioGroup,
  DropdownMenuRadioItem,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuSub,
  DropdownMenuSubTrigger,
  DropdownMenuSubContent,
};

```

■ File: src/components/ui/form.tsx

```
"use client";

import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label@2.1.2";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import {
  Controller,
  FormProvider,
  useFormContext,
  useFormState,
  type ControllerProps,
  type FieldPath,
  type FieldValues,
} from "react-hook-form@7.55.0";

import { cn } from "../../utils";
import { Label } from "../../label";

const Form = FormProvider;

type FormFieldContextValue<T> = {
  fieldValues: T;
  fieldName: string;
  fieldPath: string;
};

const FormFieldContext = React.createContext<FormFieldContextValue<T>>({
  fieldValues: {} as T,
  fieldName: "",
  fieldPath: "",
});

const FormField = <T>({
  fieldValues: T;
  fieldName: string;
  fieldPath: string;
  ...props
}: ControllerProps<T>, T, string) => {
  return (
    <FormFieldContext.Provider value={{ fieldValues, fieldName, fieldPath }}>
      <Controller {...props} />
    </FormFieldContext.Provider>
  );
};

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext);
  const itemContext = React.useContext(FormItemContext);
  const { getFieldState } = useFormContext();
  const formState = useFormState({ name: fieldContext.name });
  const fieldState = getFieldState(fieldContext.name, formState);

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>");
  }

  const { id } = itemContext;

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  };
};

type FormItemContextValue = {
  id: string;
};

const FormItemContext = React.createContext<FormItemContextValue>({
  id: "" as string,
});

function FormItem({ className, ...props }: React.ComponentProps<"div">) {
  const id = React.useId();

  return (
    <FormItemContext.Provider value={{ id }}>
      <div className={cn(className)} ...props />
    </FormItemContext.Provider>
  );
}
```



```

    <FormItemContext.Provider value={{ id }}>
      <div
        data-slot="form-item"
        className={cn("grid gap-2", className)}
        {...props}
      />
    </FormItemContext.Provider>
  );
}

function FormLabel({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  const { error, formItemId } = useFormField();

  return (
    <Label
      data-slot="form-label"
      data-error={!!error}
      className={cn("data-[error=true]:text-destructive", className)}
      htmlFor={formItemId}
      {...props}
    />
  );
}

function FormControl({ ...props }: React.ComponentProps<typeof Slot>) {
  const { error, formItemId, formDescriptionId, formMessageId } =
    useFormField();

  return (
    <Slot
      data-slot="form-control"
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props}
    />
  );
}

function FormDescription({ className, ...props }: React.ComponentProps<"p">) {
  const { formDescriptionId } = useFormField();

  return (
    <p
      data-slot="form-description"
      id={formDescriptionId}
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    />
  );
}

function FormMessage({ className, ...props }: React.ComponentProps<"p">) {
  const { error, formMessageId } = useFormField();
  const body = error ? String(error?.message ?? "") : props.children;

  if (!body) {
    return null;
  }

  return (
    <p
      data-slot="form-message"
      id={formMessageId}
      className={cn("text-destructive text-sm", className)}
      {...props}
    >
      {body}
    </p>
  );
}

export {
  useFormField,
  Form,

```

```
FormItem,  
FormLabel,  
FormControl,  
FormDescription,  
FormMessage,  
FormField,  
};
```

■ File: src/components/ui/hover-card.tsx

```
"use client";

import * as React from "react";
import * as HoverCardPrimitive from "@radix-ui/react-hover-card@1.1.6";

import { cn } from "../../utils";

function HoverCard({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Root> & {
  return <HoverCardPrimitive.Root data-slot="hover-card" {...props} /&gt;;
}) {

function HoverCardTrigger({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Trigger> & {
  return (
    <HoverCardPrimitive.Trigger data-slot="hover-card-trigger" {...props} /&gt;
  );
}) {

function HoverCardContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Content> & {
  return (
    <HoverCardPrimitive.Portal data-slot="hover-card-portal"&gt;
      <HoverCardPrimitive.Content
        data-slot="hover-card-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2 z-50 w-64
origin-(--radix-hover-card-content-transform-origin) rounded-md border p-4 shadow-md
outline-hidden",
          className,
        )}
        {...props}
      /&gt;
    </HoverCardPrimitive.Portal>
  );
}) {

export { HoverCard, HoverCardTrigger, HoverCardContent };
```

■ File: src/components/ui/input-otp.tsx

```
"use client";

import * as React from "react";
import { OTPInput, OTPInputContext } from "input-otp@1.4.2";
import { MinusIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function InputOTP({
  className,
  containerClassName,
  ...props
}: React.ComponentProps<typeof OTPInput> & {
  containerClassName?: string;
}) {
  return (
    <OTPInput
      data-slot="input-otp"
      containerClassName={cn(
        "flex items-center gap-2 has-disabled:opacity-50",
        containerClassName,
      )}
      className={cn("disabled:cursor-not-allowed", className)}
      {...props}
    />
  );
}

function InputOTPGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-otp-group"
      className={cn("flex items-center gap-1", className)}
      {...props}
    />
  );
}

function InputOTPSlot({
  index,
  className,
  ...props
}: React.ComponentProps<"div"> & {
  index: number;
}) {
  const inputOTPContext = React.useContext(OTPInputContext);
  const { char, hasFakeCaret, isActive } = inputOTPContext?.slots[index] ?? {};

  return (
    <div
      data-slot="input-otp-slot"
      data-active={isActive}
      className={cn(
        "data-[active=true]:border-ring data-[active=true]:ring-ring/50
        data-[active=true]:aria-invalid:ring-destructive/20
        dark:data-[active=true]:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive data-[active=true]:aria-invalid:border-destructive
        dark:bg-input/30 border-input relative flex h-9 w-9 items-center justify-center
        border-y border-r text-sm bg-input-background transition-all outline-none
        first:rounded-l-md first:border-l last:rounded-r-md data-[active=true]:z-10
        data-[active=true]:ring-[3px]",
        className,
      )}
      {...props}
    >
      {char}
      {hasFakeCaret & & (
        <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="animate-caret-blink bg-foreground h-4 w-px duration-1000" />
        </div>
      )}
    </div>
  );
}

function InputOTPSeparator({ ...props }: React.ComponentProps<"div">) {
  return (
    <div data-slot="input-otp-separator" role="separator" {...props}>>
  );
}
```

```
        &lt;MinusIcon /&gt;
    &lt;/div>
);
}

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator };
```

■ File: src/components/ui/input.tsx

```
import * as React from "react";

import { cn } from "../../utils";

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        "file:text-foreground placeholder:text-muted-foreground selection:bg-primary
        selection:text-primary-foreground dark:bg-input/30 border-input flex h-9 w-full
        min-w-0 rounded-md border px-3 py-1 text-base bg-input-background
        transition-[color,box-shadow] outline-none file:inline-flex file:h-7 file:border-0
        file:bg-transparent file:text-sm file:font-medium disabled:pointer-events-none
        disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        "focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]",
        "aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive",
        className,
      )}
      {...props}
    />
  );
}

export { Input };
```

■ File: src\components\ui\label.tsx

```
"use client";

import * as React from "react";
import * as LabelPrimitive from "@radix-ui/react-label@2.1.2";

import { cn } from "../utils";

function Label({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn(
        "flex items-center gap-2 text-sm leading-none font-medium select-none
        group-data-[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50
        peer-disabled:cursor-not-allowed peer-disabled:opacity-50",
        className,
      )}
      {...props}
    />
  );
}

export { Label };
```

■ File: src/components/ui/menuubar.tsx

```
"use client";

import * as React from "react";
import * as MenubarPrimitive from "@radix-ui/react-menubar@1.1.6";
import { CheckIcon, ChevronRightIcon, CircleIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Menubar({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Root>) {
  return (
    <MenubarPrimitive.Root
      data-slot="menubar"
      className={cn(
        "bg-background flex h-9 items-center gap-1 rounded-md border p-1 shadow-xs",
        className,
      )}
      {...props}
    />
  );
}

function MenubarMenu({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Menu>) {
  return <MenubarPrimitive.Menu data-slot="menubar-menu" {...props} />;
}

function MenubarGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Group>) {
  return <MenubarPrimitive.Group data-slot="menubar-group" {...props} />;
}

function MenubarPortal({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Portal>) {
  return <MenubarPrimitive.Portal data-slot="menubar-portal" {...props} />;
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioGroup>) {
  return (
    <MenubarPrimitive.RadioGroup data-slot="menubar-radio-group" {...props} />
  );
}

function MenubarTrigger({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Trigger>) {
  return (
    <MenubarPrimitive.Trigger
      data-slot="menubar-trigger"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex items-center rounded-sm px-2 py-1
        text-sm font-medium outline-hidden select-none",
        className,
      )}
      {...props}
    />
  );
}

function MenubarContent({
  className,
  align = "start",
  alignOffset = -4,
  sideOffset = 8,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Content>) {
  return (
    <MenubarPortal>
      <MenubarPrimitive.Content
        data-slot="menubar-content"

```



```

    align={align}
    alignOffset={alignOffset}
    sideOffset={sideOffset}
    className={cn(
      "bg-popover text-popover-foreground data-[state=open]:animate-in
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2 z-50
min-w-[12rem] origin-(--radix-menubar-content-transform-origin) overflow-hidden
rounded-md border p-1 shadow-md",
      className,
    )}
    {...props}
  />
</MenubarPortal>
);
}

function MenubarItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Item> & {
  inset?: boolean;
  variant?: "default" | "destructive";
}) {
  return (
    <MenubarPrimitive.Item
      data-slot="menubar-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
data-[variant=destructive]:text-destructive
data-[variant=destructive]:focus:bg-destructive/10
dark:data-[variant=destructive]:focus:bg-destructive/20
data-[variant=destructive]:focus:text-destructive
data-[variant=destructive]:*[svg]:!text-destructive
[&_svg]:not([class*='text-']):text-muted-foreground relative flex cursor-default
items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none
data-[disabled]:pointer-events-none data-[disabled]:opacity-50 data-[inset]:pl-8
[&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-']):size-4",
        className,
      )}
      {...props}
    />
  );
}

function MenubarCheckboxItem({
  className,
  children,
  checked,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.CheckboxItem>) {
  return (
    <MenubarPrimitive.CheckboxItem
      data-slot="menubar-checkbox-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
items-center gap-2 rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
data-[disabled]:pointer-events-none data-[disabled]:opacity-50
[&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-']):size-4",
        className,
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
justify-center">
        <MenubarPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.CheckboxItem>
  );
}

function MenubarRadioItem({

```

```

    className,
    children,
    ...props
  }: React.ComponentProps<typeof MenubarPrimitive.RadioItem>) {
    return (
      <MenubarPrimitive.RadioItem
        data-slot="menubar-radio-item"
        className={cn(
          "focus:bg-accent focus:text-accent-foreground relative flex cursor-default
            items-center gap-2 rounded-xs py-1.5 pr-2 pl-8 text-sm outline-hidden select-none
            data-[disabled]:pointer-events-none data-[disabled]:opacity-50
            [&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-']):size-4",
          className,
        )}
        {...props}
      />
      <span className="pointer-events-none absolute left-2 flex size-3.5 items-center
        justify-center">
        <MenubarPrimitive.ItemIndicator>
          <CircleIcon className="size-2 fill-current" />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.RadioItem>
  );
}

function MenubarLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Label> & {
  inset?: boolean;
}) {
  return (
    <MenubarPrimitive.Label
      data-slot="menubar-label"
      data-inset={inset}
      className={cn(
        "px-2 py-1.5 text-sm font-medium data-[inset]:pl-8",
        className,
      )}
      {...props}
    />
  );
}

function MenubarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Separator>) {
  return (
    <MenubarPrimitive.Separator
      data-slot="menubar-separator"
      className={cn("bg-border -mx-1 my-1 h-px", className)}
      {...props}
    />
  );
}

function MenubarShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="menubar-shortcut"
      className={cn(
        "text-muted-foreground ml-auto text-xs tracking-widest",
        className,
      )}
      {...props}
    />
  );
}

function MenubarSub({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Sub>) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
}

```

```

function MenubarSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubTrigger> & {
  inset?: boolean;
}) {
  return (
    <MenubarPrimitive.SubTrigger
      data-slot="menubar-sub-trigger"
      data-inset={inset}
      className={cn(
        "focus:bg-accent focus:text-accent-foreground data-[state=open]:bg-accent
        data-[state=open]:text-accent-foreground flex cursor-default items-center rounded-sm
        px-2 py-1.5 text-sm outline-none select-none data-[inset]:pl-8",
        className,
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto h-4 w-4" />
    </MenubarPrimitive.SubTrigger>
  );
}

function MenubarSubContent({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubContent> & {
}) {
  return (
    <MenubarPrimitive.SubContent
      data-slot="menubar-sub-content"
      className={cn(
        "bg-popover text-popover-foreground data-[state=open]:animate-in
        data-[state=closed]:animate-out data-[state=closed]:fade-out-0
        data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
        data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
        data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
        data-[side=top]:slide-in-from-bottom-2 z-50 min-w-[8rem]
        origin-(--radix-menubar-content-transform-origin) overflow-hidden rounded-md border
        p-1 shadow-lg",
        className,
      )}
      {...props}
    >
      {children}
    </MenubarPrimitive.SubContent>
  );
}

export {
  Menubar,
  MenubarPortal,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarGroup,
  MenubarSeparator,
  MenubarLabel,
  MenubarItem,
  MenubarShortcut,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarSub,
  MenubarSubTrigger,
  MenubarSubContent,
};

```

■ File: src/components/ui/navigation-menu.tsx

```
import * as React from "react";
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu@1.2.5";
import { cva } from "class-variance-authority@0.7.1";
import { ChevronDownIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function NavigationMenu({
  className,
  children,
  viewport = true,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Root> & {
  viewport?: boolean;
}) {
  return (
    <NavigationMenuPrimitive.Root
      data-slot="navigation-menu"
      data-viewport={viewport}
      className={cn(
        "group/navigation-menu relative flex max-w-max flex-1 items-center
        justify-center",
        className,
      )}
      {...props}
    >
      {children}
      <NavigationMenuViewport />
    </NavigationMenuPrimitive.Root>
  );
}

function NavigationMenuList({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.List>) {
  return (
    <NavigationMenuPrimitive.List
      data-slot="navigation-menu-list"
      className={cn(
        "group flex flex-1 list-none items-center justify-center gap-1",
        className,
      )}
      {...props}
    >
      />
  );
}

function NavigationMenuItem({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Item>) {
  return (
    <NavigationMenuPrimitive.Item
      data-slot="navigation-menu-item"
      className={cn("relative", className)}
      {...props}
    >
      />
  );
}

const navigationMenuTriggerStyle = cva(
  "group inline-flex h-9 w-max items-center justify-center rounded-md bg-background px-4
  py-2 text-sm font-medium hover:bg-accent hover:text-accent-foreground
  focus:bg-accent focus:text-accent-foreground disabled:pointer-events-none
  disabled:opacity-50 data-[state=open]:hover:bg-accent
  data-[state=open]:text-accent-foreground data-[state=open]:focus:bg-accent
  data-[state=open]:bg-accent/50 focus-visible:ring-ring/50 outline-none
  transition-[color,box-shadow] focus-visible:ring-[3px] focus-visible:outline-1",
);

function NavigationMenuTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Trigger>) {
  return (
    <NavigationMenuPrimitive.Trigger
      data-slot="navigation-menu-trigger"
    >
      {children}
    </>
  );
}
```

```

        className={cn(navigationMenuTriggerStyle(), "group", className)}
        {...props}
      >
        {children}{" "}
        <ChevronDownIcon
          className="relative top-[1px] ml-1 size-3 transition duration-300
group-data-[state=open]:rotate-180"
          aria-hidden="true"
        />
      </NavigationMenuPrimitive.Trigger>
    );
  }

function NavigationMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Content>) {
  return (
    <NavigationMenuPrimitive.Content
      data-slot="navigation-menu-content"
      className={cn(
        "data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out
data-[motion^=from-]:fade-in data-[motion^=to-]:fade-out
data-[motion=from-end]:slide-in-from-right-52
data-[motion=from-start]:slide-in-from-left-52
data-[motion=to-end]:slide-out-to-right-52
data-[motion=to-start]:slide-out-to-left-52 top-0 left-0 w-full p-2 pr-2.5
md:absolute md:w-auto",
        "group-data-[viewport=false]/navigation-menu:bg-popover
group-data-[viewport=false]/navigation-menu:text-popover-foreground
group-data-[viewport=false]/navigation-menu:data-[state=open]:animate-in
group-data-[viewport=false]/navigation-menu:data-[state=closed]:animate-out
group-data-[viewport=false]/navigation-menu:data-[state=closed]:zoom-out-95
group-data-[viewport=false]/navigation-menu:data-[state=open]:zoom-in-95
group-data-[viewport=false]/navigation-menu:data-[state=open]:fade-in-0
group-data-[viewport=false]/navigation-menu:data-[state=closed]:fade-out-0
group-data-[viewport=false]/navigation-menu:top-full
group-data-[viewport=false]/navigation-menu:mt-1.5
group-data-[viewport=false]/navigation-menu:overflow-hidden
group-data-[viewport=false]/navigation-menu:rounded-md
group-data-[viewport=false]/navigation-menu:border
group-data-[viewport=false]/navigation-menu:shadow
group-data-[viewport=false]/navigation-menu:duration-200
**data-[slot=navigation-menu-link]:focus:ring-0
**data-[slot=navigation-menu-link]:focus:outline-none",
        className,
      )}
      {...props}
    />
  );
}

function NavigationMenuViewport({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Viewport>) {
  return (
    <div
      className={cn(
        "absolute top-full left-0 isolate z-50 flex justify-center",
      )}
    >
      <NavigationMenuPrimitive.Viewport
        data-slot="navigation-menu-viewport"
        className={cn(
          "origin-top-center bg-popover text-popover-foreground
data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 relative mt-1.5
h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden rounded-md
border shadow md:w-[var(--radix-navigation-menu-viewport-width)]",
          className,
        )}
        {...props}
      />
    </div>
  );
}

function NavigationMenuLink({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Link>) {

```

```

return (
  <NavigationMenuPrimitive.Link
    data-slot="navigation-menu-link"
    className={cn(
      "data-[active=true]:focus:bg-accent data-[active=true]:hover:bg-accent
      data-[active=true]:bg-accent/50 data-[active=true]:text-accent-foreground
      hover:bg-accent hover:text-accent-foreground focus:bg-accent
      focus:text-accent-foreground focus-visible:ring-ring/50
      [&_svg:not([class*='text-']):text-muted-foreground flex flex-col gap-1 rounded-sm
      p-2 text-sm transition-all outline-none focus-visible:ring-[3px]
      focus-visible:outline-1 [&_svg:not([class*='size-']):size-4",
      className,
    )}
    {...props}
  />
);
}

function NavigationMenuIndicator({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Indicator>) {
  return (
    <NavigationMenuPrimitive.Indicator
      data-slot="navigation-menu-indicator"
      className={cn(
        "data-[state=visible]:animate-in data-[state=hidden]:animate-out
        data-[state=hidden]:fade-out data-[state=visible]:fade-in top-full z-[1] flex h-1.5
        items-end justify-center overflow-hidden",
        className,
      )}
      {...props}
    >
    <div className="bg-border relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm
      shadow-md" />
    </NavigationMenuPrimitive.Indicator>
  );
}

export {
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
  navigationMenuTriggerStyle,
};

```

■ File: src/components/ui/pagination.tsx

```
import * as React from "react";
import {
  ChevronLeftIcon,
  ChevronRightIcon,
  MoreHorizontalIcon,
} from "lucide-react@0.487.0";

import { cn } from "../utils";
import { Button, buttonVariants } from "../button";

function Pagination({ className, ...props }: React.ComponentProps<"nav">) {
  return (
    <nav
      role="navigation"
      aria-label="pagination"
      data-slot="pagination"
      className={cn("mx-auto flex w-full justify-center", className)}
      {...props}
    />
  );
}

function PaginationContent({
  className,
  ...props
}: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="pagination-content"
      className={cn("flex flex-row items-center gap-1", className)}
      {...props}
    />
  );
}

function PaginationItem({ ...props }: React.ComponentProps<"li">) {
  return <li data-slot="pagination-item" {...props} />;
}

type PaginationLinkProps = {
  isActive?: boolean;
} & Pick<React.ComponentProps<typeof Button>;, "size"> &
  React.ComponentProps<"a">;

function PaginationLink({
  className,
  isActive,
  size = "icon",
  ...props
}: PaginationLinkProps) {
  return (
    <a
      aria-current={isActive ? "page" : undefined}
      data-slot="pagination-link"
      data-active={isActive}
      className={cn(
        buttonVariants({
          variant: isActive ? "outline" : "ghost",
          size,
        }),
        className,
      )}
      {...props}
    />
  );
}

function PaginationPrevious({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink>) {
  return (
    <PaginationLink
      aria-label="Go to previous page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pl-2.5", className)}
      {...props}
    />
    <ChevronLeftIcon />
  );
}
```

```

        <span className="hidden sm:block">Previous</span>
      </PaginationLink>
    );
  }

function PaginationNext({
  className,
  ...props
}: React.ComponentProps<typeof PaginationLink>) {
  return (
    <PaginationLink
      aria-label="Go to next page"
      size="default"
      className={cn("gap-1 px-2.5 sm:pr-2.5", className)}
      {...props}
    >
      <span className="hidden sm:block">Next</span>
      <ChevronRightIcon />
    </PaginationLink>
  );
}

function PaginationEllipsis({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      aria-hidden
      data-slot="pagination-ellipsis"
      className={cn("flex size-9 items-center justify-center", className)}
      {...props}
    >
      <MoreHorizontalIcon className="size-4" />
      <span className="sr-only">More pages</span>
    </span>
  );
}

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
};

```


■ File: src/components/ui/popover.tsx

```
"use client";

import * as React from "react";
import * as PopoverPrimitive from "@radix-ui/react-popover@1.1.6";

import { cn } from "../../utils";

function Popover({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Root> & {
  return <PopoverPrimitive.Root data-slot="popover" {...props} />;
}) {

  function PopoverTrigger({
    ...props
  }: React.ComponentProps<typeof PopoverPrimitive.Trigger> & {
    return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />;
  }) {

    function PopoverContent({
      className,
      align = "center",
      sideOffset = 4,
      ...props
    }: React.ComponentProps<typeof PopoverPrimitive.Content> & {
      return (
        <PopoverPrimitive.Portal>
          <PopoverPrimitive.Content
            data-slot="popover-content"
            align={align}
            sideOffset={sideOffset}
            className={cn(
              "bg-popover text-popover-foreground data-[state=open]:animate-in
              data-[state=closed]:animate-out data-[state=closed]:fade-out-0
              data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
              data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
              data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
              data-[side=top]:slide-in-from-bottom-2 z-50 w-72
              origin-(-radix-popover-content-transform-origin) rounded-md border p-4 shadow-md
              outline-hidden",
              className,
            )}
            {...props}
          />
        </PopoverPrimitive.Portal>
      );
    }

    function PopoverAnchor({
      ...props
    }: React.ComponentProps<typeof PopoverPrimitive.Anchor> & {
      return <PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} />;
    }

    export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor };
```

■ File: src/components/ui/progress.tsx

```
"use client";

import * as React from "react";
import * as ProgressPrimitive from "@radix-ui/react-progress@1.1.2";

import { cn } from "../utils";

function Progress({
  className,
  value,
  ...props
}: React.ComponentProps<typeof ProgressPrimitive.Root>) {
  return (
    <ProgressPrimitive.Root
      data-slot="progress"
      className={cn(
        "bg-primary/20 relative h-2 w-full overflow-hidden rounded-full",
        className,
      )}
      {...props}
    >
      <ProgressPrimitive.Indicator
        data-slot="progress-indicator"
        className="bg-primary h-full w-full flex-1 transition-all"
        style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
      />
    </ProgressPrimitive.Root>
  );
}

export { Progress };
```

■ File: src/components/ui/radio-group.tsx

```
"use client";

import * as React from "react";
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group@1.2.3";
import { CircleIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function RadioGroup({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Root>) {
  return (
    <RadioGroupPrimitive.Root
      data-slot="radio-group"
      className={cn("grid gap-3", className)}
      {...props}
    />
  );
}

function RadioGroupItem({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Item>) {
  return (
    <RadioGroupPrimitive.Item
      data-slot="radio-group-item"
      className={cn(
        "border-input text-primary focus-visible:border-ring focus-visible:ring-ring/50
        aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive dark:bg-input/30 aspect-square size-4 shrink-0
        rounded-full border shadow-xs transition-[color,box-shadow] outline-none
        focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    >
      <RadioGroupPrimitive.Indicator
        data-slot="radio-group-indicator"
        className="relative flex items-center justify-center"
      >
        <CircleIcon className="fill-primary absolute top-1/2 left-1/2 size-2
        -translate-x-1/2 -translate-y-1/2" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  );
}

export { RadioGroup, RadioGroupItem };
```

■ File: src\components\uilresizable.tsx

```
"use client";

import * as React from "react";
import { GripVerticalIcon } from "lucide-react@0.487.0";
import * as ResizablePrimitive from "react-resizable-panels@2.1.7";

import { cn } from "../utils";

function ResizablePanelGroup({
  className,
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.PanelGroup>) {
  return (
    <ResizablePrimitive.PanelGroup
      data-slot="resizable-panel-group"
      className={cn(
        "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
        className,
      )}
      {...props}
    />
  );
}

function ResizablePanel({
  ...props
}: React.ComponentProps<typeof ResizablePrimitive.Panel> & {
  return <ResizablePrimitive.Panel data-slot="resizable-panel" {...props} />;
}) {
  return (
    <ResizablePrimitive.PanelResizeHandle
      data-slot="resizable-handle"
      className={cn(
        "bg-border focus-visible:ring-ring relative flex w-px items-center justify-center
        after:absolute after:inset-y-0 after:left-1/2 after:w-1 after:-translate-x-1/2
        focus-visible:ring-1 focus-visible:ring-offset-1 focus-visible:outline-hidden
        data-[panel-group-direction=vertical]:h-px
        data-[panel-group-direction=vertical]:w-full
        data-[panel-group-direction=vertical]:after:left-0
        data-[panel-group-direction=vertical]:after:h-1
        data-[panel-group-direction=vertical]:after:w-full
        data-[panel-group-direction=vertical]:after:-translate-y-1/2
        data-[panel-group-direction=vertical]:after:translate-x-0
        [&[data-panel-group-direction=vertical]]&div:rotate-90",
        className,
      )}
      {...props}
    >
      {withHandle & & (
        <div className="bg-border z-10 flex h-4 w-3 items-center justify-center
        rounded-xs border">
          <GripVerticalIcon className="size-2.5" />
        </div>
      )}
    </ResizablePrimitive.PanelResizeHandle>
  );
}

export { ResizablePanelGroup, ResizablePanel, ResizableHandle };
```

■ File: src\components\ui\scroll-area.tsx

```
"use client";

import * as React from "react";
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area@1.2.3";

import { cn } from "../utils";

function ScrollArea({
  className,
  children,
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.Root> & {
  return (
    <ScrollAreaPrimitive.Root
      data-slot="scroll-area"
      className={cn("relative", className)}
      {...props}
    >
      <ScrollAreaPrimitive.Viewport
        data-slot="scroll-area-viewport"
        className="focus-visible:ring-ring/50 size-full rounded-[inherit]
        transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
        focus-visible:outline-1"
        >
          {children}
        </ScrollAreaPrimitive.Viewport>
        <ScrollBar />
        <ScrollAreaPrimitive.Corner />
      </ScrollAreaPrimitive.Root>
    </>
  );
}

function ScrollBar({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.ScrollAreaScrollbar> & {
  return (
    <ScrollAreaPrimitive.ScrollAreaScrollbar
      data-slot="scroll-area-scrollbar"
      orientation={orientation}
      className={cn(
        "flex touch-none p-px transition-colors select-none",
        orientation === "vertical" & &
        "h-full w-2.5 border-l border-l-transparent",
        orientation === "horizontal" & &
        "h-2.5 flex-col border-t border-t-transparent",
        className,
      )}
      {...props}
    >
      <ScrollAreaPrimitive.ScrollAreaThumb
        data-slot="scroll-area-thumb"
        className="bg-border relative flex-1 rounded-full"
        />
      </ScrollAreaPrimitive.ScrollAreaScrollbar>
    </>
  );
}

export { ScrollArea, ScrollBar };
```

■ File: src/components/ui/select.tsx

```
"use client";

import * as React from "react";
import * as SelectPrimitive from "@radix-ui/react-select@2.1.6";
import {
  CheckIcon,
  ChevronDownIcon,
  ChevronUpIcon,
} from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Select({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Root> & {
  return <SelectPrimitive.Root data-slot="select" {...props} />;
}) {

function SelectGroup({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Group> & {
  return <SelectPrimitive.Group data-slot="select-group" {...props} />;
}) {

function SelectValue({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Value> & {
  return <SelectPrimitive.Value data-slot="select-value" {...props} />;
}) {

function SelectTrigger({
  className,
  size = "default",
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Trigger> & {
  size?: "sm" | "default";
}) {
  return (
    <SelectPrimitive.Trigger
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        "border-input data-[placeholder]:text-muted-foreground
        [&_svg:not([class*='text-'])]:text-muted-foreground focus-visible:border-ring
        focus-visible:ring-ring/50 aria-invalid:ring-destructive/20
        dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
        dark:bg-input/30 dark:hover:bg-input/50 flex w-full items-center justify-between
        gap-2 rounded-md border bg-input-background px-3 py-2 text-sm whitespace-nowrap
        transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
        disabled:cursor-not-allowed disabled:opacity-50 data-[size=default]:h-9
        data-[size=sm]:h-8 *:data-[slot=select-value]:line-clamp-1
        *:data-[slot=select-value]:flex *:data-[slot=select-value]:items-center
        *:data-[slot=select-value]:gap-2 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg:not([class*='size-'])]:size-4",
        className,
      )
    >{...props}</>
    <children>
      <SelectPrimitive.Icon asChild>
        <ChevronDownIcon className="size-4 opacity-50" />
      </SelectPrimitive.Icon>
    </SelectPrimitive.Trigger>
  );
}

function SelectContent({
  className,
  children,
  position = "popper",
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Content> & {
  return (
    <SelectPrimitive.Portal>
      <SelectPrimitive.Content
        data-slot="select-content"
        className={cn(
          "bg-popover text-popover-foreground data-[state=open]:animate-in

```

```

data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2 relative z-50
max-h-(--radix-select-content-available-height) min-w-[8rem]
origin(--radix-select-content-transform-origin) overflow-x-hidden overflow-y-auto
rounded-md border shadow-md",
  position === "popper" &&&
  "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1
data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
  className,
)}
position={position}
{...props}
</>
<SelectScrollUpButton />
<SelectPrimitive.Viewport
  className={cn(
    "p-1",
    position === "popper" &&&
    "h-[var(--radix-select-trigger-height)] w-full
min-w-[var(--radix-select-trigger-width)] scroll-my-1",
  )}
  </>
  {children}
  </SelectPrimitive.Viewport>
  <SelectScrollDownButton />
  </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
);
}

function SelectLabel({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Label> & {
  return (
    <SelectPrimitive.Label
      data-slot="select-label"
      className={cn("text-muted-foreground px-2 py-1.5 text-xs", className)}
      {...props}
    />
  );
}

function SelectItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Item> & {
  return (
    <SelectPrimitive.Item
      data-slot="select-item"
      className={cn(
        "focus:bg-accent focus:text-accent-foreground
[&_svg:not([class*='text-'])]:text-muted-foreground relative flex w-full
cursor-default items-center gap-2 rounded-sm py-1.5 pr-8 pl-2 text-sm outline-hidden
select-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50
[&_svg]:pointer-events-none [&_svg]:shrink-0 [&_svg]:not([class*='size-']):size-4
*:span]:last:flex *:span]:last:items-center *:span]:last:gap-2",
        className,
      )}
      {...props}
    >
      <span className="absolute right-2 flex size-3.5 items-center justify-center">
        <SelectPrimitive.ItemIndicator>
          <CheckIcon className="size-4" />
        </SelectPrimitive.ItemIndicator>
      </span>
      <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
    </SelectPrimitive.Item>
  );
}

function SelectSeparator({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Separator> & {
  return (
    <SelectPrimitive.Separator
      data-slot="select-separator"

```

```

        className={cn("bg-border pointer-events-none -mx-1 my-1 h-px", className)}
        {...props}
      /&gt;
    );
  }

function SelectScrollUpButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollUpButton> &gt;) {
  return (
    &lt;SelectPrimitive.ScrollUpButton
      data-slot="select-scroll-up-button"
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className,
      )}
      {...props}
    &gt;
      &lt;ChevronUpIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollUpButton&gt;
  );
}

function SelectScrollDownButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollDownButton> &gt;) {
  return (
    &lt;SelectPrimitive.ScrollDownButton
      data-slot="select-scroll-down-button"
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className,
      )}
      {...props}
    &gt;
      &lt;ChevronDownIcon className="size-4" /&gt;
    &lt;/SelectPrimitive.ScrollDownButton&gt;
  );
}

export {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectScrollDownButton,
  SelectScrollUpButton,
  SelectSeparator,
  SelectTrigger,
  SelectValue,
};

```


■ File: src/components/ui/separator.tsx

```
"use client";

import * as React from "react";
import * as SeparatorPrimitive from "@radix-ui/react-separator@1.1.2";

import { cn } from "../utils";

function Separator({
  className,
  orientation = "horizontal",
  decorative = true,
  ...props
}: React.ComponentProps<typeof SeparatorPrimitive.Root>) {
  return (
    <SeparatorPrimitive.Root
      data-slot="separator-root"
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "bg-border shrink-0 data-[orientation=horizontal]:h-px
        data-[orientation=horizontal]:w-full data-[orientation=vertical]:h-full
        data-[orientation=vertical]:w-px",
        className,
      )}
      {...props}
    />
  );
}

export { Separator };
```

■ File: src/components/ui/sheet.tsx

```
"use client";

import * as React from "react";
import * as SheetPrimitive from "@radix-ui/react-dialog@1.1.6";
import { XIcon } from "lucide-react@0.487.0";

import { cn } from "../../utils";

function Sheet({ ...props }: React.ComponentProps<SheetPrimitive.Root>) {
  return <SheetPrimitive.Root data-slot="sheet" {...props} />;
}

function SheetTrigger({
  ...props
}: React.ComponentProps<SheetPrimitive.Trigger>) {
  return <SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} />;
}

function SheetClose({
  ...props
}: React.ComponentProps<SheetPrimitive.Close>) {
  return <SheetPrimitive.Close data-slot="sheet-close" {...props} />;
}

function SheetPortal({
  ...props
}: React.ComponentProps<SheetPrimitive.Portal>) {
  return <SheetPrimitive.Portal data-slot="sheet-portal" {...props} />;
}

function SheetOverlay({
  className,
  ...props
}: React.ComponentProps<SheetPrimitive.Overlay>) {
  return (
    <SheetPrimitive.Overlay
      data-slot="sheet-overlay"
      className={cn(
        "data-[state=open]:animate-in data-[state=closed]:animate-out
        data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 fixed inset-0 z-50
        bg-black/50",
        className,
      )}
      {...props}
    />
  );
}

function SheetContent({
  className,
  children,
  side = "right",
  ...props
}: React.ComponentProps<SheetPrimitive.Content> & {
  side?: "top" | "right" | "bottom" | "left";
}) {
  return (
    <SheetPortal>
      <SheetOverlay />
      <SheetPrimitive.Content
        data-slot="sheet-content"
        className={cn(
          "bg-background data-[state=open]:animate-in data-[state=closed]:animate-out
          fixed z-50 flex flex-col gap-4 shadow-lg transition ease-in-out
          data-[state=closed]:duration-300 data-[state=open]:duration-500",
          side === "right" &&
            "data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right
            inset-y-0 right-0 h-full w-3/4 border-l sm:max-w-sm",
          side === "left" &&
            "data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left
            inset-y-0 left-0 h-full w-3/4 border-r sm:max-w-sm",
          side === "top" &&
            "data-[state=closed]:slide-out-to-top data-[state=open]:slide-in-from-top
            inset-x-0 top-0 h-auto border-b",
          side === "bottom" &&
            "data-[state=closed]:slide-out-to-bottom
            data-[state=open]:slide-in-from-bottom inset-x-0 bottom-0 h-auto border-t",
          className,
        )}
      >
        {children}
      </SheetPrimitive.Content>
    </SheetPortal>
  );
}
```

```

        {...props}
      &gt;
      {children}
      &lt;SheetPrimitive.Close className="ring-offset-background focus:ring-ring
data-[state=open]:bg-secondary absolute top-4 right-4 rounded-xs opacity-70
transition-opacity hover:opacity-100 focus:ring-2 focus:ring-offset-2
focus:outline-hidden disabled:pointer-events-none"&gt;
        &lt;XIcon className="size-4" /&gt;
        &lt;span className="sr-only"&gt;Close&lt;/span&gt;
      &lt;/SheetPrimitive.Close&gt;
    &lt;/SheetPrimitive.Content&gt;
  &lt;/SheetPortal&gt;
);
}

function SheetHeader({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sheet-header"
      className={cn("flex flex-col gap-1.5 p-4", className)}
      {...props}
    /&gt;
  );
}

function SheetFooter({ className, ...props }: React.ComponentProps&lt;"div"&gt;) {
  return (
    &lt;div
      data-slot="sheet-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    /&gt;
  );
}

function SheetTitle({
  className,
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Title&gt;) {
  return (
    &lt;SheetPrimitive.Title
      data-slot="sheet-title"
      className={cn("text-foreground font-semibold", className)}
      {...props}
    /&gt;
  );
}

function SheetDescription({
  className,
  ...props
}: React.ComponentProps&lt;typeof SheetPrimitive.Description&gt;) {
  return (
    &lt;SheetPrimitive.Description
      data-slot="sheet-description"
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    /&gt;
  );
}

export {
  Sheet,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
};

```

■ File: src/components/ui/sidebar.tsx

```
"use client";

import * as React from "react";
import { Slot } from "@radix-ui/react-slot@1.1.2";
import { VariantProps, cva } from "class-variance-authority@0.7.1";
import { PanelLeftIcon } from "lucide-react@0.487.0";

import { useIsMobile } from "../use-mobile";
import { cn } from "../utils";
import { Button } from "../button";
import { Input } from "../input";
import { Separator } from "../separator";
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from "../sheet";
import { Skeleton } from "../skeleton";
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "../tooltip";

const SIDEBAR_COOKIE_NAME = "sidebar_state";
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;
const SIDEBAR_WIDTH = "16rem";
const SIDEBAR_WIDTH_MOBILE = "18rem";
const SIDEBAR_WIDTH_ICON = "3rem";
const SIDEBAR_KEYBOARD_SHORTCUT = "b";

type SidebarContextProps = {
  state: "expanded" | "collapsed";
  open: boolean;
  setOpen: (open: boolean) => void;
  openMobile: boolean;
  setOpenMobile: (open: boolean) => void;
  isMobile: boolean;
  toggleSidebar: () => void;
};

const SidebarContext = React.createContext<SidebarContextProps | null>(null);

function useSidebar() {
  const context = React.useContext(SidebarContext);
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.");
  }

  return context;
}

function SidebarProvider({
  defaultOpen = true,
  open: openProp,
  onOpenChange: setOpenProp,
  className,
  style,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  defaultOpen?: boolean;
  open?: boolean;
  onOpenChange?: (open: boolean) => void;
}) {
  const isMobile = useIsMobile();
  const [openMobile, setOpenMobile] = React.useState(false);

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen);
  const open = openProp ?? _open;
  const setOpen = React.useCallback(
    (value: boolean | ((value: boolean) => boolean)) => {
      const openState = typeof value === "function" ? value(open) : value;
      if (setOpenProp) {
        setOpenProp(openState);
      }
    },
    [setOpenProp, open]
  );
```

```

        setOpenProp(openState);
    } else {
        _setOpen(openState);
    }

    // This sets the cookie to keep the sidebar state.
    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/;
    max-age=${SIDEBAR_COOKIE_MAX_AGE}`;
  },
  [setOpenProp, open],
);

// Helper to toggle the sidebar.
const toggleSidebar = React.useCallback(() => {
  return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open);
}, [isMobile, setOpen, setOpenMobile]);

// Adds a keyboard shortcut to toggle the sidebar.
React.useEffect(() => {
  const handleKeyDown = (event: KeyboardEvent) => {
    if (
      event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
      (event.metaKey || event.ctrlKey)
    ) {
      event.preventDefault();
      toggleSidebar();
    }
  };

  window.addEventListener("keydown", handleKeyDown);
  return () => window.removeEventListener("keydown", handleKeyDown);
}, [toggleSidebar]);

// We add a state so that we can do data-state="expanded" or "collapsed".
// This makes it easier to style the sidebar with Tailwind classes.
const state = open ? "expanded" : "collapsed";

const contextValue = React.useMemo<SidebarContextProps>(() => ({
  state,
  open,
  setOpen,
  isMobile,
  openMobile,
  setOpenMobile,
  toggleSidebar,
})),
[state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
);

return (
  <SidebarContext.Provider value={contextValue}>
    <TooltipProvider delayDuration={0}>
      <div
        data-slot="sidebar-wrapper"
        style={
          {
            "--sidebar-width": SIDEBAR_WIDTH,
            "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
            ...style,
          } as React.CSSProperties
        }
        className={cn(
          "group/sidebar-wrapper has-data-[variant=inset]:bg-sidebar flex min-h-svh
w-full",
          className,
        )}
        {...props}
      >
        {children}
      </div>
    </TooltipProvider>
  </SidebarContext.Provider>
);
}

function Sidebar({
  side = "left",
  variant = "sidebar",
  collapsible = "offcanvas",
  className,
  children,

```

```

    ...props
  }; React.ComponentProps<div> & & {
    side?: "left" | "right";
    variant?: "sidebar" | "floating" | "inset";
    collapsible?: "offcanvas" | "icon" | "none";
  }) {
    const { isMobile, state, openMobile, setOpenMobile } = useSidebar();

    if (collapsible === "none") {
      return (
        <div
          data-slot="sidebar"
          className={cn(
            "bg-sidebar text-sidebar-foreground flex h-full w-(--sidebar-width) flex-col",
            className,
          )}
          {...props}
        >
          {children}
        </div>
      );
    }

    if (isMobile) {
      return (
        <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
          <SheetContent
            data-sidebar="sidebar"
            data-slot="sidebar"
            data-mobile="true"
            className="bg-sidebar text-sidebar-foreground w-(--sidebar-width) p-0"
            [& & button]:hidden
            style={{
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            }} as React.CSSProperties
            side={side}
          >
            <SheetHeader className="sr-only">
              <SheetTitle>Sidebar</SheetTitle>
              <SheetDescription>Displays the mobile sidebar.</SheetDescription>
            </SheetHeader>
            <div className="flex h-full w-full flex-col">{children}</div>
          </SheetContent>
        </Sheet>
      );
    }

    return (
      <div
        className="group peer text-sidebar-foreground hidden md:block"
        data-state={state}
        data-collapsible={state === "collapsed" ? collapsible : ""}
        data-variant={variant}
        data-side={side}
        data-slot="sidebar"
      >
        {
          /* This is what handles the sidebar gap on desktop */
          <div
            data-slot="sidebar-gap"
            className={cn(
              "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200 ease-linear",
              "group-data-[collapsible=offcanvas]:w-0",
              "group-data-[side=right]:rotate-180",
              variant === "floating" || variant === "inset"
                ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4)))]"
                : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)",
            )}
          >
            <div
              data-slot="sidebar-container"
              className={cn(
                "fixed inset-y-0 z-10 hidden h-svh w-(--sidebar-width) transition-[left,right,width] duration-200 ease-linear md:flex",
                side === "left"
                  ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
                  : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
              )}
            >

```

```

    // Adjust the padding for floating and inset variants.
    variant === "floating" || variant === "inset"
      ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4))+2px)]"
      : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)
    group-data-[side=left]:border-r group-data-[side=right]:border-l",
    className,
  )}
  {...props}
  >
    <div
      data-sidebar="sidebar"
      data-slot="sidebar-inner"
      className="bg-sidebar group-data-[variant=floating]:border-sidebar-border flex
h-full w-full flex-col group-data-[variant=floating]:rounded-lg
group-data-[variant=floating]:border group-data-[variant=floating]:shadow-sm"
      >
        {children}
      </div>
    </div>
  </div>
);
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps<typeof Button> & {
  const { toggleSidebar } = useSidebar();

  return (
    <Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon"
      className={cn("size-7", className)}
      onClick={(event) => {
        onClick?.(event);
        toggleSidebar();
      }}
      {...props}
    >
      <PanelLeftIcon />
      <span className="sr-only">Toggle Sidebar</span>
    </Button>
  );
}

function SidebarRail({ className, ...props }: React.ComponentProps<"button"> & {
  const { toggleSidebar } = useSidebar();

  return (
    <button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "hover:after:bg-sidebar-border absolute inset-y-0 z-20 hidden w-4
-translate-x-1/2 transition-all ease-linear group-data-[side=left]:right-4
group-data-[side=right]:left-0 after:absolute after:inset-y-0 after:left-1/2
after:w-[2px] sm:flex",
        "in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]&]:cursor-e-resize",
        "[[data-side=right][data-state=collapsed]&]:cursor-w-resize",
        "hover:group-data-[collapsible=offcanvas]:bg-sidebar",
        group-data-[collapsible=offcanvas]:translate-x-0",
        group-data-[collapsible=offcanvas]:after:left-full",
        "[[data-side=left][data-collapsible=offcanvas]&]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]&]:-left-2",
        className,
      )}
      {...props}
    >
  );
}

function SidebarInset({ className, ...props }: React.ComponentProps<"main"> & {

```

```

return (
  <main
    data-slot="sidebar-inset"
    className={cn(
      "bg-background relative flex w-full flex-1 flex-col",
      "md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0",
      "md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow-sm",
      "md:peer-data-[variant=inset]:peer-data-[state=collapsed]:ml-2",
      className,
    )}
    {...props}
  />
);
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps<typeof Input> & { type: typeof Input }) {
  return (
    <Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn("bg-background h-8 w-full shadow-none", className)}
      {...props}
    />
  );
}

function SidebarHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  );
}

function SidebarFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  );
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator> & { type: typeof Separator }) {
  return (
    <Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn("bg-sidebar-border mx-2 w-auto", className)}
      {...props}
    />
  );
}

function SidebarContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto",
        "group-data-[collapsible=icon]:overflow-hidden",
        className,
      )}
      {...props}
    />
  );
}

function SidebarGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (

```



```

    <div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    />
  );
}

function SidebarGroupLabel({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"div"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "div";

  return (
    <Comp
      data-slot="sidebar-group-label"
      data-sidebar="group-label"
      className={cn(
        "text-sidebar-foreground/70 ring-sidebar-ring flex h-8 shrink-0 items-center
        rounded-md px-2 text-xs font-medium outline-hidden transition-[margin,opacity]
        duration-200 ease-linear focus-visible:ring-2 [&svg]:size-4 [&svg]:shrink-0",
        "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
        className,
      )}
      {...props}
    />
  );
}

function SidebarGroupAction({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot : "button";

  return (
    <Comp
      data-slot="sidebar-group-action"
      data-sidebar="group-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
        hover:text-sidebar-accent-foreground absolute top-3.5 right-3 flex aspect-square w-5
        items-center justify-center rounded-md p-0 outline-hidden transition-transform
        focus-visible:ring-2 [&svg]:size-4 [&svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  );
}

function SidebarGroupContent({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group-content"
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
      {...props}
    />
  );
}

function SidebarMenu({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu"
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-1", className)}
      {...props}
    />
  );
}

```

```

function SidebarMenuItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    />
  );
}

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2
  text-left text-sm outline-hidden ring-sidebar-ring transition-[width,height,padding]
  hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-visible:ring-2
  active:bg-sidebar-accent active:text-sidebar-accent-foreground
  disabled:pointer-events-none disabled:opacity-50
  group-has-data-[sidebar=menu-action]/menu-item:pr-8
  aria-disabled:pointer-events-none aria-disabled:opacity-50
  data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
  data-[active=true]:text-sidebar-accent-foreground
  data-[state=open]:hover:bg-sidebar-accent
  data-[state=open]:hover:text-sidebar-accent-foreground
  group-data-[collapsible=icon]:size-8! group-data-[collapsible=icon]:p-2!
  [&&span:last-child]:truncate [&&svg]:size-4 [&&svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_1px_hsl(var(--sidebar-border))]"
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:p-0!",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = "default",
  size = "default",
  tooltip,
  className,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean;
  isActive?: boolean;
  tooltip?: string | React.ComponentProps<typeof TooltipContent>;
} & { VariantProps<typeof sidebarMenuButtonVariants> }) {
  const Comp = asChild ? Slot : "button";
  const { isMobile, state } = useSidebar();

  const button = (
    <Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    />
  );

  if (!tooltip) {
    return button;
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    };
  }

```

```

}

return (
  <Tooltip>
    <TooltipTrigger asChild>{button}</TooltipTrigger>
    <TooltipContent
      side="right"
      align="center"
      hidden={state !== "collapsed" || isMobile}
    >{...tooltip}</>
  </Tooltip>
);
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean;
  showOnHover?: boolean;
}) {
  const Comp = asChild ? Slot : "button";

  return (
    <Comp
      data-slot="sidebar-menu-action"
      data-sidebar="menu-action"
      className={cn(
        "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
        hover:text-sidebar-accent-foreground peer-hover/menu-button:text-sidebar-accent-foreg
        round absolute top-1.5 right-1 flex aspect-square w-5 items-center justify-center
        rounded-md p-0 outline-hidden transition-transform focus-visible:ring-2
        [&#x26;#x27;svg]:size-4 [&#x26;#x27;svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 md:after:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &#x26;#x27;
        "peer-data-[active=true]/menu-button:text-sidebar-accent-foreground
        group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100
        data-[state=open]:opacity-100 md:opacity-0",
        className,
      )}>
      {...props}
    </>
  );
}

function SidebarMenuBadge({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        "text-sidebar-foreground pointer-events-none absolute right-1 flex h-5 min-w-5
        items-center justify-center rounded-md px-1 text-xs font-medium tabular-nums
        select-none",
        "peer-hover/menu-button:text-sidebar-accent-foreground
        peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}>
      {...props}
    </>
  );
}

function SidebarMenuSkeleton({
  className,
  showIcon = false,
  ...props

```

```

}: React.ComponentProps<"div"> & {
  showIcon?: boolean;
}) {
  // Random width between 50 to 90%.
  const width = React.useMemo(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, []);

  return (
    <div
      data-slot="sidebar-menu-skeleton"
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    >
      {showIcon & & {
        <Skeleton
          className="size-4 rounded-md"
          data-sidebar="menu-skeleton-icon"
        />
      }}
      <Skeleton
        className="h-4 max-w(--skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      />
    </div>
  );
}

function SidebarMenuSub({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu-sub"
      data-sidebar="menu-sub"
      className={cn(
        "border-sidebar-border mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l
        px-2.5 py-0.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    />
  );
}

function SidebarMenuSubItem({
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-sub-item"
      data-sidebar="menu-sub-item"
      className={cn("group/menu-sub-item relative", className)}
      {...props}
    />
  );
}

function SidebarMenuSubButton({
  asChild = false,
  size = "md",
  isActive = false,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean;
  size?: "sm" | "md";
  isActive?: boolean;
}) {
  const Comp = asChild ? Slot : "a";

  return (
    <Comp
      data-slot="sidebar-menu-sub-button"
      data-sidebar="menu-sub-button"
      data-size={size}
    >

```

```

    data-active={isActive}
    className={cn(
      "text-sidebar-foreground ring-sidebar-ring hover:bg-sidebar-accent
      hover:text-sidebar-accent-foreground active:bg-sidebar-accent
      active:text-sidebar-accent-foreground [&svg]:text-sidebar-accent-foreground flex
      h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2
      outline-hidden focus-visible:ring-2 disabled:pointer-events-none disabled:opacity-50
      aria-disabled:pointer-events-none aria-disabled:opacity-50
      [&span]:last-child:truncate [&svg]:size-4 [&svg]:shrink-0",
      "data-[active=true]:bg-sidebar-accent
      data-[active=true]:text-sidebar-accent-foreground",
      size === "sm" && "text-xs",
      size === "md" && "text-sm",
      "group-data-[collapsible=icon]:hidden",
      className,
    )}
    {...props}
  /&gt;
}
);

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
};

```

■ File: src\components\ui\skeleton.tsx

```
import { cn } from "../utils";

function Skeleton({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="skeleton"
      className={cn("bg-accent animate-pulse rounded-md", className)}
      {...props}
    />
  );
}

export { Skeleton };
```

■ File: src/components/ui/slider.tsx

```
"use client";

import * as React from "react";
import * as SliderPrimitive from "@radix-ui/react-slider@1.2.3";

import { cn } from "../../utils";

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps<typeof SliderPrimitive.Root>) {
  const _values = React.useMemo(
    () => {
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
    [value, defaultValue, min, max],
  );

  return (
    <SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        "relative flex w-full touch-none items-center select-none",
        data-[disabled]:opacity-50 data-[orientation=vertical]:h-full
        data-[orientation=vertical]:min-h-44 data-[orientation=vertical]:w-auto
        data-[orientation=vertical]:flex-col",
        className,
      )}
      {...props}
    >
      <SliderPrimitive.Track
        data-slot="slider-track"
        className={cn(
          "bg-muted relative grow overflow-hidden rounded-full",
          data-[orientation=horizontal]:h-4 data-[orientation=horizontal]:w-full
          data-[orientation=vertical]:h-full data-[orientation=vertical]:w-1.5",
        )}
      >
        <SliderPrimitive.Range
          data-slot="slider-range"
          className={cn(
            "bg-primary absolute data-[orientation=horizontal]:h-full
            data-[orientation=vertical]:w-full",
          )}
        />
        </SliderPrimitive.Track>
        {Array.from({ length: _values.length }, (_, index) => (
          <SliderPrimitive.Thumb
            data-slot="slider-thumb"
            key={index}
            className="border-primary bg-background ring-ring/50 block size-4 shrink-0
            rounded-full border shadow-sm transition-[color,box-shadow] hover:ring-4
            focus-visible:ring-4 focus-visible:outline-hidden disabled:pointer-events-none
            disabled:opacity-50"
          />
        ))}
      </SliderPrimitive.Root>
    );
}

export { Slider };
```

■ File: src/components/ui/sonner.tsx

```
"use client";

import { useTheme } from "next-themes@0.4.6";
import { Toaster as Sonner, ToasterProps } from "sonner@2.0.3";

const Toaster = ({ ...props }: ToasterProps) => {
  const { theme = "system" } = useTheme();

  return (
    <Sonner
      theme={theme as ToasterProps["theme"]}
      className="toaster group"
      style={{
        "--normal-bg": "var(--popover)",
        "--normal-text": "var(--popover-foreground)",
        "--normal-border": "var(--border)",
      } as React.CSSProperties
      {...props}
    />
  );
};

export { Toaster };
```


■ File: src/components/ui/switch.tsx

```
"use client";

import * as React from "react";
import * as SwitchPrimitive from "@radix-ui/react-switch@1.1.3";

import { cn } from "../utils";

function Switch({
  className,
  ...props
}: React.ComponentProps<typeof SwitchPrimitive.Root> & {
  return (
    <SwitchPrimitive.Root
      data-slot="switch"
      className={cn(
        "peer data-[state=checked]:bg-primary data-[state=unchecked]:bg-switch-background
        focus-visible:border-ring focus-visible:ring-ring/50
        dark:data-[state=unchecked]:bg-input/80 inline-flex h-[1.15rem] w-8 shrink-0
        items-center rounded-full border border-transparent transition-all outline-none
        focus-visible:ring-[3px] disabled:cursor-not-allowed disabled:opacity-50",
        className,
      )}
      {...props}
    >
      <SwitchPrimitive.Thumb
        data-slot="switch-thumb"
        className={cn(
          "bg-card dark:data-[state=unchecked]:bg-card-foreground
          dark:data-[state=checked]:bg-primary-foreground pointer-events-none block size-4
          rounded-full ring-0 transition-transform
          data-[state=checked]:translate-x-[calc(100%-2px)]
          data-[state=unchecked]:translate-x-0",
        )}
        />
      </SwitchPrimitive.Root>
    >
  );
}

export { Switch };
```

■ File: src\components\ui\table.tsx

```

use client";

import * as React from "react";

import { cn } from "../utils";

function Table({ className, ...props }: React.ComponentProps<"table">) {
  return (
    <div
      data-slot="table-container"
      className="relative w-full overflow-x-auto"
    >
      <table
        data-slot="table"
        className={cn("w-full caption-bottom text-sm", className)}
        {...props}
      />
    </div>
  );
}

function TableHeader({ className, ...props }: React.ComponentProps<"thead">) {
  return (
    <thead
      data-slot="table-header"
      className={cn("[&_tr]:border-b", className)}
      {...props}
    />
  );
}

function TableBody({ className, ...props }: React.ComponentProps<"tbody">) {
  return (
    <tbody
      data-slot="table-body"
      className={cn("[&_tr:last-child]:border-0", className)}
      {...props}
    />
  );
}

function TableFooter({ className, ...props }: React.ComponentProps<"tfoot">) {
  return (
    <tfoot
      data-slot="table-footer"
      className={cn(
        "bg-muted/50 border-t font-medium [&_tr:last]:border-b-0",
        className,
      )}
      {...props}
    />
  );
}

function TableRow({ className, ...props }: React.ComponentProps<"tr">) {
  return (
    <tr
      data-slot="table-row"
      className={cn(
        "hover:bg-muted/50 data-[state=selected]:bg-muted border-b transition-colors",
        className,
      )}
      {...props}
    />
  );
}

function TableHead({ className, ...props }: React.ComponentProps<"th">) {
  return (
    <th
      data-slot="table-head"
      className={cn(
        "text-foreground h-10 px-2 text-left align-middle font-medium whitespace-nowrap",
        [&_has([role=checkbox]):pr-0 [&_tr]:pr-0 [&_tr]:pr-0 [&_tr]:pr-0 [&_tr]:pr-0 [&_tr]:pr-0",
        className,
      )}
      {...props}
    />
  );
}

```

```

}

function TableCell({ className, ...props }: React.ComponentProps<"td">) {
  return (
    <td
      data-slot="table-cell"
      className={cn(
        "p-2 align-middle whitespace-nowrap [&:has([role=checkbox])]:pr-0
        [&:&[role=checkbox]]:translate-y-[2px]",
        className,
      )}
      {...props}
    />
  );
}

function TableCaption({
  className,
  ...props
}: React.ComponentProps<"caption">) {
  return (
    <caption
      data-slot="table-caption"
      className={cn("text-muted-foreground mt-4 text-sm", className)}
      {...props}
    />
  );
}

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
};

```

■ File: src/components/uitabs.tsx

```
"use client";

import * as React from "react";
import * as TabsPrimitive from "@radix-ui/react-tabs@1.1.3";

import { cn } from "../utils";

function Tabs({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Root> & {
  return (
    <TabsPrimitive.Root
      data-slot="tabs"
      className={cn("flex flex-col gap-2", className)}
      {...props}
    />
  );
}

function TabsList({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.List> & {
  return (
    <TabsPrimitive.List
      data-slot="tabs-list"
      className={cn(
        "bg-muted text-muted-foreground inline-flex h-9 w-fit items-center justify-center
        rounded-xl p-[3px] flex",
        className,
      )}
      {...props}
    />
  );
}

function TabsTrigger({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Trigger> & {
  return (
    <TabsPrimitive.Trigger
      data-slot="tabs-trigger"
      className={cn(
        "data-[state=active]:bg-card dark:data-[state=active]:text-foreground
        focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:outline-ring
        dark:data-[state=active]:border-input dark:data-[state=active]:bg-input/30
        text-foreground dark:text-muted-foreground inline-flex h-[calc(100%-1px)] flex-1
        items-center justify-center gap-1.5 rounded-xl border border-transparent px-2 py-1
        text-sm font-medium whitespace-nowrap transition-[color,box-shadow]
        focus-visible:ring-[3px] focus-visible:outline-1 disabled:pointer-events-none
        disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:shrink-0
        [&_svg]:not([class*='size-']):size-4",
        className,
      )}
      {...props}
    />
  );
}

function TabsContent({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Content> & {
  return (
    <TabsPrimitive.Content
      data-slot="tabs-content"
      className={cn("flex-1 outline-none", className)}
      {...props}
    />
  );
}

export { Tabs, TabsList, TabsTrigger, TabsContent };
```

■ File: src/components/uitextarea.tsx

```
import * as React from "react";

import { cn } from "../utils";

function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        "resize-none border-input placeholder:text-muted-foreground
        focus-visible:border-ring focus-visible:ring-ring/50
        aria-invalid:ring-destructive/20 dark:aria-invalid:ring-destructive/40
        aria-invalid:border-destructive dark:bg-input/30 flex field-sizing-content min-h-16
        w-full rounded-md border bg-input-background px-3 py-2 text-base
        transition-[color,box-shadow] outline-none focus-visible:ring-[3px]
        disabled:cursor-not-allowed disabled:opacity-50 md:text-sm",
        className,
      )}
      {...props}
    />
  );
}

export { Textarea };
```

■ File: src\components\uitoggle-group.tsx

```
"use client";

import * as React from "react";
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group@1.1.2";
import { type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "../utils";
import { toggleVariants } from "../toggle";

const ToggleGroupContext = React.createContext<
  VariantProps<typeof toggleVariants>&
>({
  size: "default",
  variant: "default",
});

function ToggleGroup({
  className,
  variant,
  size,
  children,
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Root> &
  VariantProps<typeof toggleVariants> & ) {
  return (
    <ToggleGroupPrimitive.Root
      data-slot="toggle-group"
      data-variant={variant}
      data-size={size}
      className={cn(
        "group/toggle-group flex w-fit items-center rounded-md
        data-[variant=outline]:shadow-xs",
        className,
      )}
      {...props}
    >
      <ToggleGroupContext.Provider value={{ variant, size }}>
        {children}
      </ToggleGroupContext.Provider>
    </ToggleGroupPrimitive.Root>
  );
}

function ToggleGroupItem({
  className,
  children,
  variant,
  size,
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Item> &
  VariantProps<typeof toggleVariants> & ) {
  const context = React.useContext(ToggleGroupContext);

  return (
    <ToggleGroupPrimitive.Item
      data-slot="toggle-group-item"
      data-variant={context.variant || variant}
      data-size={context.size || size}
      className={cn(
        toggleVariants({
          variant: context.variant || variant,
          size: context.size || size,
        }),
        "min-w-0 flex-1 shrink-0 rounded-none shadow-none first:rounded-l-md
        last:rounded-r-md focus:z-10 focus-visible:z-10 data-[variant=outline]:border-l-0
        data-[variant=outline]:first:border-l",
        className,
      )}
      {...props}
    >
      {children}
    </ToggleGroupPrimitive.Item>
  );
}

export { ToggleGroup, ToggleGroupItem };
```

■ File: src/components/uitoggle.tsx

```
"use client";

import * as React from "react";
import * as TogglePrimitive from "@radix-ui/react-toggle@1.1.2";
import { cva, type VariantProps } from "class-variance-authority@0.7.1";

import { cn } from "../utils";

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm font-medium
  hover:bg-muted hover:text-muted-foreground disabled:pointer-events-none
  disabled:opacity-50 data-[state=on]:bg-accent data-[state=on]:text-accent-foreground
  [&_svg]:pointer-events-none [&_svg]:not([class*='size-']):size-4 [&_svg]:shrink-0
  focus-visible:border-ring focus-visible:ring-ring/50 focus-visible:ring-[3px]
  outline-none transition-[color,box-shadow] aria-invalid:ring-destructive/20
  dark:aria-invalid:ring-destructive/40 aria-invalid:border-destructive
  whitespace-nowrap",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent hover:bg-accent
          hover:text-accent-foreground",
      },
      size: {
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
);

function Toggle({
  className,
  variant,
  size,
  ...props
}: React.ComponentProps<typeof TogglePrimitive.Root> &
  VariantProps<typeof toggleVariants> & {
    return (
      <TogglePrimitive.Root
        data-slot="toggle"
        className={cn(toggleVariants({ variant, size, className })}
        {...props}
      />
    );
  }

export { Toggle, toggleVariants };
```

■ File: src\components\uil\tooltip.tsx

```
"use client";

import * as React from "react";
import * as TooltipPrimitive from "@radix-ui/react-tooltip@1.1.8";

import { cn } from "../utils";

function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
  return (
    <TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    />
  );
}

function Tooltip({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return (
    <TooltipProvider>
      <TooltipPrimitive.Root data-slot="tooltip" {...props} />
    </TooltipProvider>
  );
}

function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />;
}

function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          "bg-primary text-primary-foreground animate-in fade-in-0 zoom-in-95
          data-[state=closed]:animate-out data-[state=closed]:fade-out-0
          data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2
          data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
          data-[side=top]:slide-in-from-bottom-2 z-50 w-fit
          origin-(--radix-tooltip-content-transform-origin) rounded-md px-3 py-1.5 text-xs
          text-balance",
          className,
        )}
        {...props}
      >
        {children}
      </TooltipPrimitive.Content>
    </TooltipPrimitive.Portal>
  );
}

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```


■ File: src\components\ui\use-mobile.ts

```
import * as React from "react";

const MOBILE_BREAKPOINT = 768;

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(<
    undefined,
  >);

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    };
    mql.addEventListener("change", onChange);
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
    return () => mql.removeEventListener("change", onChange);
  }, []);

  return !isMobile;
}
```

■ File: src\components\ui\utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

■ File: src\config.ts

```
// Central configuration for API endpoints
export const API_BASE_URL = import.meta.env.VITE_API_URL ||
  `http://${window.location.hostname}:8080`;
```

■ File: srclindex.css

```
/*! tailwindcss v4.1.3 | MIT License | https://tailwindcss.com */
@layer properties {
  @supports (((-webkit-hyphens: none)) and (not (margin-trim: inline))) or ((-moz-orient:
    inline) and (not (color: rgb(from red r g b)))) {
    *, :before, :after, ::backdrop {
      --tw-translate-x: 0;
      --tw-translate-y: 0;
      --tw-translate-z: 0;
      --tw-space-y-reverse: 0;
      --tw-border-style: solid;
      --tw-gradient-position: initial;
      --tw-gradient-from: #0000;
      --tw-gradient-via: #0000;
      --tw-gradient-to: #0000;
      --tw-gradient-stops: initial;
      --tw-gradient-via-stops: initial;
      --tw-gradient-from-position: 0%;
      --tw-gradient-via-position: 50%;
      --tw-gradient-to-position: 100%;
      --tw-shadow: 0 0 #0000;
      --tw-shadow-color: initial;
      --tw-shadow-alpha: 100%;
      --tw-inset-shadow: 0 0 #0000;
      --tw-inset-shadow-color: initial;
      --tw-inset-shadow-alpha: 100%;
      --tw-ring-color: initial;
      --tw-ring-shadow: 0 0 #0000;
      --tw-inset-ring-color: initial;
      --tw-inset-ring-shadow: 0 0 #0000;
      --tw-ring-inset: initial;
      --tw-ring-offset-width: 0px;
      --tw-ring-offset-color: #fff;
      --tw-ring-offset-shadow: 0 0 #0000;
      --tw-blur: initial;
      --tw-brightness: initial;
      --tw-contrast: initial;
      --tw-grayscale: initial;
      --tw-hue-rotate: initial;
      --tw-invert: initial;
      --tw-opacity: initial;
      --tw-saturate: initial;
      --tw-sepia: initial;
      --tw-drop-shadow: initial;
      --tw-drop-shadow-color: initial;
      --tw-drop-shadow-alpha: 100%;
      --tw-drop-shadow-size: initial;
    }
  }
}

@layer theme {
  :root, :host {
    --font-sans: ui-sans-serif, system-ui, sans-serif;
    --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation
      Mono", "Courier New", monospace;
    --color-red-50: oklch(.971 .013 17.38);
    --color-red-100: oklch(.936 .032 17.717);
    --color-orange-100: oklch(.954 .038 75.164);
    --color-yellow-50: oklch(.987 .026 102.212);
    --color-yellow-100: oklch(.973 .071 103.193);
    --color-green-100: oklch(.962 .044 156.743);
    --color-green-600: oklch(.627 .194 149.214);
    --color-cyan-50: oklch(.984 .019 200.873);
    --color-cyan-100: oklch(.956 .045 203.388);
    --color-blue-50: oklch(.97 .014 254.604);
    --color-blue-100: oklch(.932 .032 255.585);
    --color-blue-200: oklch(.882 .059 254.128);
    --color-purple-100: oklch(.946 .033 307.174);
    --color-purple-600: oklch(.558 .288 302.321);
    --color-gray-50: oklch(.985 .002 247.839);
    --color-gray-100: oklch(.967 .003 264.542);
    --color-gray-200: oklch(.928 .006 264.531);
    --color-black: #000;
    --color-white: #fff;
    --spacing: .25rem;
    --container-md: 28rem;
    --container-3xl: 48rem;
    --container-4xl: 56rem;
    --text-sm: .875rem;
```

```

--text-sm--line-height: calc(1.25 / .875);
--radius-2xl: 1rem;
--default-transition-duration: .15s;
--default-transition-timing-function: cubic-bezier(.4, 0, .2, 1);
--default-font-family: var(--font-sans);
--default-font-feature-settings: var(--font-sans--font-feature-settings);
--default-font-variation-settings: var(--font-sans--font-variation-settings);
--default-mono-font-family: var(--font-mono);
--default-mono-font-feature-settings: var(--font-mono--font-feature-settings);
--default-mono-font-variation-settings: var(--font-mono--font-variation-settings);
--color-background: #f8fafc;
--color-primary: #3b82f6;
--color-secondary: #06b6d4;
--color-border: #e2e8f0;
--color-primary-dark: #2563eb;
--color-medical-blue: #e8f4f8;
--color-success: #10b981;
--color-warning: #f59e0b;
--color-danger: #ef4444;
--color-text-primary: #1e293b;
--color-text-secondary: #64748b;
}
}

@layer base {
*, :after, :before, ::backdrop {
  box-sizing: border-box;
  border: 0 solid;
  margin: 0;
  padding: 0;
}

::file-selector-button {
  box-sizing: border-box;
  border: 0 solid;
  margin: 0;
  padding: 0;
}

html, :host {
  -webkit-text-size-adjust: 100%;
  tab-size: 4;
  line-height: 1.5;
  font-family: var(--default-font-family, ui-sans-serif, system-ui, sans-serif, "Apple
    Color Emoji", "Segoe UI Emoji", "Segoe UI Symbol", "Noto Color Emoji");
  font-feature-settings: var(--default-font-feature-settings, normal);
  font-variation-settings: var(--default-font-variation-settings, normal);
  -webkit-tap-highlight-color: transparent;
}

body {
  line-height: inherit;
}

hr {
  height: 0;
  color: inherit;
  border-top-width: 1px;
}

abbr:where([title]) {
  -webkit-text-decoration: underline dotted;
  text-decoration: underline dotted;
}

h1, h2, h3, h4, h5, h6 {
  font-size: inherit;
  font-weight: inherit;
}

a {
  color: inherit;
  -webkit-text-decoration: inherit;
  text-decoration: inherit;
}

b, strong {
  font-weight: bolder;
}

code, kbd, samp, pre {

```

```

font-family: var(--default-mono-font-family, ui-monospace, SFMono-Regular, Menlo,
  Monaco, Consolas, "Liberation Mono", "Courier New", monospace);
font-feature-settings: var(--default-mono-font-feature-settings, normal);
font-variation-settings: var(--default-mono-font-variation-settings, normal);
font-size: 1em;
}

small {
  font-size: 80%;
}

sub, sup {
  vertical-align: baseline;
  font-size: 75%;
  line-height: 0;
  position: relative;
}

sub {
  bottom: -.25em;
}

sup {
  top: -.5em;
}

table {
  text-indent: 0;
  border-color: inherit;
  border-collapse: collapse;
}

:-moz-focusring {
  outline: auto;
}

progress {
  vertical-align: baseline;
}

summary {
  display: list-item;
}

ol, ul, menu {
  list-style: none;
}

img, svg, video, canvas, audio, iframe, embed, object {
  vertical-align: middle;
  display: block;
}

img, video {
  max-width: 100%;
  height: auto;
}

button, input, select, optgroup, textarea {
  font: inherit;
  font-feature-settings: inherit;
  font-variation-settings: inherit;
  letter-spacing: inherit;
  color: inherit;
  opacity: 1;
  background-color: #0000;
  border-radius: 0;
}

::file-selector-button {
  font: inherit;
  font-feature-settings: inherit;
  font-variation-settings: inherit;
  letter-spacing: inherit;
  color: inherit;
  opacity: 1;
  background-color: #0000;
  border-radius: 0;
}

:where(select:is([multiple], [size])) optgroup {
  font-weight: bolder;
}

```

```

}

:where(select:is([multiple], [size])) optgroup option {
  padding-inline-start: 20px;
}

::file-selector-button {
  margin-inline-end: 4px;
}

::placeholder {
  opacity: 1;
  color: currentColor;
}

@supports (color: color-mix(in lab, red, red)) {
  ::placeholder {
    color: color-mix(in oklab, currentColor 50%, transparent);
  }
}

textarea {
  resize: vertical;
}

::-webkit-search-decoration {
  -webkit-appearance: none;
}

::-webkit-date-and-time-value {
  min-height: 1lh;
  text-align: inherit;
}

::-webkit-datetime-edit {
  display: inline-flex;
}

::-webkit-datetime-edit-fields-wrapper {
  padding: 0;
}

::-webkit-datetime-edit {
  padding-block: 0;
}

::-webkit-datetime-edit-year-field {
  padding-block: 0;
}

::-webkit-datetime-edit-month-field {
  padding-block: 0;
}

::-webkit-datetime-edit-day-field {
  padding-block: 0;
}

::-webkit-datetime-edit-hour-field {
  padding-block: 0;
}

::-webkit-datetime-edit-minute-field {
  padding-block: 0;
}

::-webkit-datetime-edit-second-field {
  padding-block: 0;
}

::-webkit-datetime-edit-millisecond-field {
  padding-block: 0;
}

::-webkit-datetime-edit-meridiem-field {
  padding-block: 0;
}

:-moz-ui-invalid {
  box-shadow: none;
}

```

```

button, input:where([type="button"], [type="reset"], [type="submit"]) {
  appearance: button;
}

::file-selector-button {
  appearance: button;
}

::-webkit-inner-spin-button {
  height: auto;
}

::-webkit-outer-spin-button {
  height: auto;
}

[hidden]:where(:not([hidden="until-found"])) {
  display: none !important;
}

* {
  border-color: var(--color-border);
  outline-color: color-mix(in oklab, oklch(.708 0 0) 50%, transparent);
}

body {
  background-color: var(--color-background);
  color: oklch(.145 0 0);
}

@layer utilities {
  .absolute {
    position: absolute;
  }

  .fixed {
    position: fixed;
  }

  .relative {
    position: relative;
  }

  .sticky {
    position: sticky;
  }

  .inset-0 {
    inset: calc(var(--spacing) * 0);
  }

  .top-0 {
    top: calc(var(--spacing) * 0);
  }

  .top-1\2 {
    top: 50%;
  }

  .top-24 {
    top: calc(var(--spacing) * 24);
  }

  .bottom-0 {
    bottom: calc(var(--spacing) * 0);
  }

  .left-3 {
    left: calc(var(--spacing) * 3);
  }

  .z-50 {
    z-index: 50;
  }

  .col-span-2 {
    grid-column: span 2 / span 2;
  }

  .mx-auto {
    margin-inline: auto;
  }

```



```

}

.mt-0\5 {
  margin-top: calc(var(--spacing) * .5);
}

.mt-1 {
  margin-top: calc(var(--spacing) * 1);
}

.mt-2 {
  margin-top: calc(var(--spacing) * 2);
}

.mt-3 {
  margin-top: calc(var(--spacing) * 3);
}

.mt-4 {
  margin-top: calc(var(--spacing) * 4);
}

.mt-6 {
  margin-top: calc(var(--spacing) * 6);
}

.mb-1 {
  margin-bottom: calc(var(--spacing) * 1);
}

.mb-2 {
  margin-bottom: calc(var(--spacing) * 2);
}

.mb-3 {
  margin-bottom: calc(var(--spacing) * 3);
}

.mb-4 {
  margin-bottom: calc(var(--spacing) * 4);
}

.mb-6 {
  margin-bottom: calc(var(--spacing) * 6);
}

.mb-8 {
  margin-bottom: calc(var(--spacing) * 8);
}

.mb-16 {
  margin-bottom: calc(var(--spacing) * 16);
}

.ml-2 {
  margin-left: calc(var(--spacing) * 2);
}

.block {
  display: block;
}

.flex {
  display: flex;
}

.grid {
  display: grid;
}

.hidden {
  display: none;
}

.inline-flex {
  display: inline-flex;
}

.h-2 {
  height: calc(var(--spacing) * 2);
}

```

```

.h-3 {
  height: calc(var(--spacing) * 3);
}

.h-4 {
  height: calc(var(--spacing) * 4);
}

.h-5 {
  height: calc(var(--spacing) * 5);
}

.h-6 {
  height: calc(var(--spacing) * 6);
}

.h-8 {
  height: calc(var(--spacing) * 8);
}

.h-10 {
  height: calc(var(--spacing) * 10);
}

.h-12 {
  height: calc(var(--spacing) * 12);
}

.h-16 {
  height: calc(var(--spacing) * 16);
}

.h-20 {
  height: calc(var(--spacing) * 20);
}

.h-96 {
  height: calc(var(--spacing) * 96);
}

.max-h-96 {
  max-height: calc(var(--spacing) * 96);
}

.max-h-[90vh] {
  max-height: 90vh;
}

.min-h-screen {
  min-height: 100vh;
}

.w-2 {
  width: calc(var(--spacing) * 2);
}

.w-4 {
  width: calc(var(--spacing) * 4);
}

.w-5 {
  width: calc(var(--spacing) * 5);
}

.w-6 {
  width: calc(var(--spacing) * 6);
}

.w-8 {
  width: calc(var(--spacing) * 8);
}

.w-10 {
  width: calc(var(--spacing) * 10);
}

.w-12 {
  width: calc(var(--spacing) * 12);
}

.w-16 {
  width: calc(var(--spacing) * 16);
}

```

```

}

.w-20 {
  width: calc(var(--spacing) * 20);
}

.w-full {
  width: 100%;
}

.max-w-3xl {
  max-width: var(--container-3xl);
}

.max-w-4xl {
  max-width: var(--container-4xl);
}

.max-w-full {
  max-width: 100%;
}

.max-w-md {
  max-width: var(--container-md);
}

.flex-1 {
  flex: 1;
}

.flex-shrink-0 {
  flex-shrink: 0;
}

.-translate-y-1\2 {
  --tw-translate-y: calc(calc(1 / 2 * 100%) * -1);
  translate: var(--tw-translate-x) var(--tw-translate-y);
}

.cursor-pointer {
  cursor: pointer;
}

.resize-none {
  resize: none;
}

.list-inside {
  list-style-position: inside;
}

.list-disc {
  list-style-type: disc;
}

.appearance-none {
  appearance: none;
}

.grid-cols-1 {
  grid-template-columns: repeat(1, minmax(0, 1fr));
}

.grid-cols-2 {
  grid-template-columns: repeat(2, minmax(0, 1fr));
}

.grid-cols-5 {
  grid-template-columns: repeat(5, minmax(0, 1fr));
}

.grid-cols-7 {
  grid-template-columns: repeat(7, minmax(0, 1fr));
}

.flex-col {
  flex-direction: column;
}

.flex-wrap {
  flex-wrap: wrap;
}

```

```

.items-center {
  align-items: center;
}

.items-start {
  align-items: flex-start;
}

.justify-between {
  justify-content: space-between;
}

.justify-center {
  justify-content: center;
}

.justify-end {
  justify-content: flex-end;
}

.gap-2 {
  gap: calc(var(--spacing) * 2);
}

.gap-3 {
  gap: calc(var(--spacing) * 3);
}

.gap-4 {
  gap: calc(var(--spacing) * 4);
}

.gap-6 {
  gap: calc(var(--spacing) * 6);
}

.gap-8 {
  gap: calc(var(--spacing) * 8);
}

:where(.space-y-1 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 1) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 1) * calc(1 -
    var(--tw-space-y-reverse)));
}

:where(.space-y-2 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 2) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 2) * calc(1 -
    var(--tw-space-y-reverse)));
}

:where(.space-y-3 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 3) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 3) * calc(1 -
    var(--tw-space-y-reverse)));
}

:where(.space-y-4 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 4) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 4) * calc(1 -
    var(--tw-space-y-reverse)));
}

:where(.space-y-5 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 5) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 5) * calc(1 -
    var(--tw-space-y-reverse)));
}

:where(.space-y-6 &gt; :not(:last-child)) {
  --tw-space-y-reverse: 0;
  margin-block-start: calc(calc(var(--spacing) * 6) * var(--tw-space-y-reverse));
  margin-block-end: calc(calc(var(--spacing) * 6) * calc(1 -
    var(--tw-space-y-reverse)));
}

```

```

.self-start {
  align-self: flex-start;
}

.overflow-hidden {
  overflow: hidden;
}

.overflow-x-auto {
  overflow-x: auto;
}

.overflow-y-auto {
  overflow-y: auto;
}

.rounded {
  border-radius: .25rem;
}

.rounded-2xl {
  border-radius: var(--radius-2xl);
}

.rounded-full {
  border-radius: 3.40282e38px;
}

.rounded-lg {
  border-radius: .625rem;
}

.rounded-xl {
  border-radius: 1.025rem;
}

.border {
  border-style: var(--tw-border-style);
  border-width: 1px;
}

.border-2 {
  border-style: var(--tw-border-style);
  border-width: 2px;
}

.border-t {
  border-top-style: var(--tw-border-style);
  border-top-width: 1px;
}

.border-b {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 1px;
}

.border-b-2 {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 2px;
}

.border-blue-200 {
  border-color: var(--color-blue-200);
}

.border-border {
  border-color: var(--color-border);
}

.border-primary {
  border-color: var(--color-primary);
}

.border-warning\30 {
  border-color: #f59e0b4d;
}

@supports (color: color-mix(in lab, red, red)) {
  .border-warning\30 {
    border-color: color-mix(in oklab, var(--color-warning) 30%, transparent);
  }
}

```

```

.bg-background {
  background-color: var(--color-background);
}

.bg-black\50 {
  background-color: #00000080;
}

@supports (color: color-mix(in lab, red, red)) {
  .bg-black\50 {
    background-color: color-mix(in oklab, var(--color-black) 50%, transparent);
  }
}

.bg-blue-50 {
  background-color: var(--color-blue-50);
}

.bg-blue-100 {
  background-color: var(--color-blue-100);
}

.bg-cyan-100 {
  background-color: var(--color-cyan-100);
}

.bg-gray-50 {
  background-color: var(--color-gray-50);
}

.bg-gray-100 {
  background-color: var(--color-gray-100);
}

.bg-gray-200 {
  background-color: var(--color-gray-200);
}

.bg-green-100 {
  background-color: var(--color-green-100);
}

.bg-medical-blue {
  background-color: var(--color-medical-blue);
}

.bg-orange-100 {
  background-color: var(--color-orange-100);
}

.bg-primary {
  background-color: var(--color-primary);
}

.bg-purple-100 {
  background-color: var(--color-purple-100);
}

.bg-red-50 {
  background-color: var(--color-red-50);
}

.bg-red-100 {
  background-color: var(--color-red-100);
}

.bg-secondary {
  background-color: var(--color-secondary);
}

.bg-success {
  background-color: var(--color-success);
}

.bg-white {
  background-color: var(--color-white);
}

.bg-yellow-50 {
  background-color: var(--color-yellow-50);
}

```

```

.bg-yellow-100 {
  background-color: var(--color-yellow-100);
}

.bg-gradient-to-br {
  --tw-gradient-position: to bottom right in oklab;
  background-image: linear-gradient(var(--tw-gradient-stops));
}

.bg-gradient-to-r {
  --tw-gradient-position: to right in oklab;
  background-image: linear-gradient(var(--tw-gradient-stops));
}

.from-blue-50 {
  --tw-gradient-from: var(--color-blue-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
    var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
    var(--tw-gradient-to-position));
}

.via-white {
  --tw-gradient-via: var(--color-white);
  --tw-gradient-via-stops: var(--tw-gradient-position), var(--tw-gradient-from)
    var(--tw-gradient-from-position), var(--tw-gradient-via)
    var(--tw-gradient-via-position), var(--tw-gradient-to)
    var(--tw-gradient-to-position);
  --tw-gradient-stops: var(--tw-gradient-via-stops);
}

.to-blue-50 {
  --tw-gradient-to: var(--color-blue-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
    var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
    var(--tw-gradient-to-position));
}

.to-cyan-50 {
  --tw-gradient-to: var(--color-cyan-50);
  --tw-gradient-stops: var(--tw-gradient-via-stops, var(--tw-gradient-position),
    var(--tw-gradient-from) var(--tw-gradient-from-position), var(--tw-gradient-to)
    var(--tw-gradient-to-position));
}

.p-2 {
  padding: calc(var(--spacing) * 2);
}

.p-3 {
  padding: calc(var(--spacing) * 3);
}

.p-4 {
  padding: calc(var(--spacing) * 4);
}

.p-6 {
  padding: calc(var(--spacing) * 6);
}

.p-8 {
  padding: calc(var(--spacing) * 8);
}

.px-2 {
  padding-inline: calc(var(--spacing) * 2);
}

.px-3 {
  padding-inline: calc(var(--spacing) * 3);
}

.px-4 {
  padding-inline: calc(var(--spacing) * 4);
}

.px-6 {
  padding-inline: calc(var(--spacing) * 6);
}

.py-1 {
  padding-block: calc(var(--spacing) * 1);
}

```

```

}

.py-2 {
  padding-block: calc(var(--spacing) * 2);
}

.py-3 {
  padding-block: calc(var(--spacing) * 3);
}

.py-4 {
  padding-block: calc(var(--spacing) * 4);
}

.py-8 {
  padding-block: calc(var(--spacing) * 8);
}

.py-12 {
  padding-block: calc(var(--spacing) * 12);
}

.pt-4 {
  padding-top: calc(var(--spacing) * 4);
}

.pr-4 {
  padding-right: calc(var(--spacing) * 4);
}

.pb-4 {
  padding-bottom: calc(var(--spacing) * 4);
}

.pl-10 {
  padding-left: calc(var(--spacing) * 10);
}

.text-center {
  text-align: center;
}

.text-left {
  text-align: left;
}

.text-sm {
  font-size: var(--text-sm);
  line-height: var(--tw-leading, var(--text-sm--line-height));
}

.whitespace-nowrap {
  white-space: nowrap;
}

.text-danger {
  color: var(--color-danger);
}

.text-primary {
  color: var(--color-primary);
}

.text-purple-600 {
  color: var(--color-purple-600);
}

.text-secondary {
  color: var(--color-secondary);
}

.text-success {
  color: var(--color-success);
}

.text-text-primary {
  color: var(--color-text-primary);
}

.text-text-secondary {
  color: var(--color-text-secondary);
}

```



```

.text-warning {
  color: var(--color-warning);
}

.text-white {
  color: var(--color-white);
}

.accent-primary {
  accent-color: var(--color-primary);
}

.opacity-50 {
  opacity: .5;
}

.shadow-2xl {
  --tw-shadow: 0 25px 50px -12px var(--tw-shadow-color, #00000040);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-lg {
  --tw-shadow: 0 10px 15px -3px var(--tw-shadow-color, #0000001a), 0 4px 6px -4px
    var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-md {
  --tw-shadow: 0 4px 6px -1px var(--tw-shadow-color, #0000001a), 0 2px 4px -2px
    var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-sm {
  --tw-shadow: 0 1px 3px 0 var(--tw-shadow-color, #0000001a), 0 1px 2px -1px
    var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.shadow-xl {
  --tw-shadow: 0 20px 25px -5px var(--tw-shadow-color, #0000001a), 0 8px 10px -6px
    var(--tw-shadow-color, #0000001a);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.filter {
  filter: var(--tw-blur, ) var(--tw-brightness, ) var(--tw-contrast, )
    var(--tw-grayscale, ) var(--tw-hue-rotate, ) var(--tw-invert, ) var(--tw-saturate, )
    var(--tw-sepia, ) var(--tw-drop-shadow, );
}

.transition-all {
  transition-property: all;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

.transition-colors {
  transition-property: color, background-color, border-color, outline-color,
    text-decoration-color, fill, stroke, --tw-gradient-from, --tw-gradient-via,
    --tw-gradient-to;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

.transition-shadow {
  transition-property: box-shadow;
  transition-timing-function: var(--tw-ease, var(--default-transition-timing-function));
  transition-duration: var(--tw-duration, var(--default-transition-duration));
}

@media (hover: hover) {
  .group-hover\:text-primary:is(:where(.group):hover *) {
    color: var(--color-primary);
  }
}

```

```

.last\:border-b-0:last-child {
  border-bottom-style: var(--tw-border-style);
  border-bottom-width: 0;
}

@media (hover: hover) {
  .hover\:border-primary:hover {
    border-color: var(--color-primary);
  }
}

@media (hover: hover) {
  .hover\:bg-gray-50:hover {
    background-color: var(--color-gray-50);
  }
}

@media (hover: hover) {
  .hover\:bg-gray-100:hover {
    background-color: var(--color-gray-100);
  }
}

@media (hover: hover) {
  .hover\:bg-green-600:hover {
    background-color: var(--color-green-600);
  }
}

@media (hover: hover) {
  .hover\:bg-medical-blue:hover {
    background-color: var(--color-medical-blue);
  }
}

@media (hover: hover) {
  .hover\:bg-primary-dark:hover {
    background-color: var(--color-primary-dark);
  }
}

@media (hover: hover) {
  .hover\:bg-red-50:hover {
    background-color: var(--color-red-50);
  }
}

@media (hover: hover) {
  .hover\:text-primary:hover {
    color: var(--color-primary);
  }
}

@media (hover: hover) {
  .hover\:shadow-lg:hover {
    --tw-shadow: 0 10px 15px -3px var(--tw-shadow-color, #0000001a), 0 4px 6px -4px
    var(--tw-shadow-color, #0000001a);
    box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
    var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
  }
}

.focus\:border-transparent:focus {
  border-color: #0000;
}

.focus\:ring-2:focus {
  --tw-ring-shadow: var(--tw-ring-inset, ) 0 0 0 calc(2px +
  var(--tw-ring-offset-width)) var(--tw-ring-color, currentcolor);
  box-shadow: var(--tw-inset-shadow), var(--tw-inset-ring-shadow),
  var(--tw-ring-offset-shadow), var(--tw-ring-shadow), var(--tw-shadow);
}

.focus\:ring-primary:focus {
  --tw-ring-color: var(--color-primary);
}

.focus\:outline-none:focus {
  --tw-outline-style: none;
  outline-style: none;
}

```

```

@media (width >= 40rem) {
  .sm\:block {
    display: block;
  }
}

@media (width >= 48rem) {
  .md\:block {
    display: block;
  }
}

@media (width >= 48rem) {
  .md\:flex {
    display: flex;
  }
}

@media (width >= 48rem) {
  .md\:hidden {
    display: none;
  }
}

@media (width >= 48rem) {
  .md\:w-48 {
    width: calc(var(--spacing) * 48);
  }
}

@media (width >= 48rem) {
  .md\:grid-cols-2 {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

@media (width >= 48rem) {
  .md\:grid-cols-3 {
    grid-template-columns: repeat(3, minmax(0, 1fr));
  }
}

@media (width >= 48rem) {
  .md\:grid-cols-4 {
    grid-template-columns: repeat(4, minmax(0, 1fr));
  }
}

@media (width >= 48rem) {
  .md\:flex-row {
    flex-direction: row;
  }
}

@media (width >= 48rem) {
  .md\:items-center {
    align-items: center;
  }
}

@media (width >= 64rem) {
  .lg\:grid-cols-2 {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}

@media (width >= 64rem) {
  .lg\:grid-cols-4 {
    grid-template-columns: repeat(4, minmax(0, 1fr));
  }
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  background-color: var(--color-background);
  color: oklch(.145 0 0);
}

```

```

width: 100%;
min-height: 100vh;
overflow-x: hidden;
}

#root {
width: 100%;
min-height: 100vh;
}

h1 {
color: var(--color-text-primary);
font-size: 2rem;
font-weight: 700;
line-height: 1.2;
}

h2 {
color: var(--color-text-primary);
font-size: 1.5rem;
font-weight: 600;
line-height: 1.3;
}

h3 {
color: var(--color-text-primary);
font-size: 1.25rem;
font-weight: 600;
line-height: 1.4;
}

h4 {
color: var(--color-text-primary);
font-size: 1.125rem;
font-weight: 600;
line-height: 1.4;
}

p {
font-size: 1rem;
line-height: 1.6;
}

button {
font-size: 1rem;
font-weight: 500;
line-height: 1.5;
}

input, textarea, select {
font-size: 1rem;
line-height: 1.5;
}

label {
color: var(--color-text-primary);
font-size: .875rem;
font-weight: 500;
line-height: 1.5;
}

@property --tw-translate-x {
syntax: "";
inherits: false;
initial-value: 0;
}

@property --tw-translate-y {
syntax: "";
inherits: false;
initial-value: 0;
}

@property --tw-translate-z {
syntax: "";
inherits: false;
initial-value: 0;
}

@property --tw-space-y-reverse {
syntax: "";
inherits: false;
}

```

```

    initial-value: 0;
}

@property --tw-border-style {
    syntax: "*";
    inherits: false;
    initial-value: solid;
}

@property --tw-gradient-position {
    syntax: "*";
    inherits: false;
}

@property --tw-gradient-from {
    syntax: "<color>";
    inherits: false;
    initial-value: #0000;
}

@property --tw-gradient-via {
    syntax: "<color>";
    inherits: false;
    initial-value: #0000;
}

@property --tw-gradient-to {
    syntax: "<color>";
    inherits: false;
    initial-value: #0000;
}

@property --tw-gradient-stops {
    syntax: "*";
    inherits: false;
}

@property --tw-gradient-via-stops {
    syntax: "*";
    inherits: false;
}

@property --tw-gradient-from-position {
    syntax: "<length-percentage>";
    inherits: false;
    initial-value: 0%;
}

@property --tw-gradient-via-position {
    syntax: "<length-percentage>";
    inherits: false;
    initial-value: 50%;
}

@property --tw-gradient-to-position {
    syntax: "<length-percentage>";
    inherits: false;
    initial-value: 100%;
}

@property --tw-shadow {
    syntax: "*";
    inherits: false;
    initial-value: 0 0 #0000;
}

@property --tw-shadow-color {
    syntax: "*";
    inherits: false;
}

@property --tw-shadow-alpha {
    syntax: "<percentage>";
    inherits: false;
    initial-value: 100%;
}

@property --tw-inset-shadow {
    syntax: "*";
    inherits: false;
    initial-value: 0 0 #0000;
}

```

```

@property --tw-inset-shadow-color {
  syntax: "*";
  inherits: false
}

@property --tw-inset-shadow-alpha {
  syntax: "<percentage>";
  inherits: false;
  initial-value: 100%;
}

@property --tw-ring-color {
  syntax: "*";
  inherits: false
}

@property --tw-ring-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-inset-ring-color {
  syntax: "*";
  inherits: false
}

@property --tw-inset-ring-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-ring-inset {
  syntax: "*";
  inherits: false
}

@property --tw-ring-offset-width {
  syntax: "<length>";
  inherits: false;
  initial-value: 0;
}

@property --tw-ring-offset-color {
  syntax: "*";
  inherits: false;
  initial-value: #fff;
}

@property --tw-ring-offset-shadow {
  syntax: "*";
  inherits: false;
  initial-value: 0 0 #0000;
}

@property --tw-blur {
  syntax: "*";
  inherits: false
}

@property --tw-brightness {
  syntax: "*";
  inherits: false
}

@property --tw-contrast {
  syntax: "*";
  inherits: false
}

@property --tw-grayscale {
  syntax: "*";
  inherits: false
}

@property --tw-hue-rotate {
  syntax: "*";
  inherits: false
}

@property --tw-invert {

```

```

    syntax: "";
    inherits: false
}

@property --tw-opacity {
    syntax: "";
    inherits: false
}

@property --tw-saturate {
    syntax: "";
    inherits: false
}

@property --tw-sepia {
    syntax: "";
    inherits: false
}

@property --tw-drop-shadow {
    syntax: "";
    inherits: false
}

@property --tw-drop-shadow-color {
    syntax: "";
    inherits: false
}

@property --tw-drop-shadow-alpha {
    syntax: "<percentage>";
    inherits: false;
    initial-value: 100%;
}

@property --tw-drop-shadow-size {
    syntax: "";
    inherits: false
}

```

■ File: src/index.html

```
<!DOCTYPE html>
<html lang="zh-TW">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>■■■■■■■■■■</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/main.tsx"></script>
  </body>
</html>
```


■ File: src/main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css'; // ■ ■■■■■■■ CSS ■■

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

■ File: src\store\useAuthStore.ts

```
import { create } from 'zustand';

export type UserRole = 'patient' | 'doctor' | 'caseManager' | null;

export interface User {
  id: string;
  name: string;
  role: UserRole;
  age?: number;
  gender?: string;
  phone?: string;
  email?: string;
}

interface AuthState {
  currentUser: User | null;
  login: (user: User) => void;
  logout: () => void;
  updateUserFields: (fields: Partial<User>) => void;
}

export const useAuthStore = create<AuthState>((set) => ({
  currentUser: null,
  login: (user) => set({ currentUser: user }),
  logout: () => set({ currentUser: null }),
  updateUserFields: (fields) => set((state) => ({
    currentUser: state.currentUser ? { ...state.currentUser, ...fields } : null
  })),
}));
```

■ File: src\styles\globals.css

```
@import "tailwindcss";

@theme {
  /* Color Palette - Medical Theme */
  --color-primary: #3b82f6;
  --color-primary-light: #60a5fa;
  --color-primary-dark: #2563eb;
  --color-secondary: #06b6d4;
  --color-background: #f8fafc;
  --color-surface: #ffffff;
  --color-medical-blue: #e8f4f8;
  --color-success: #10b981;
  --color-warning: #f59e0b;
  --color-danger: #ef4444;
  --color-text-primary: #1e293b;
  --color-text-secondary: #64748b;
  --color-border: #e2e8f0;

  /* Typography */
  --font-sans: ui-sans-serif, system-ui, sans-serif;
}

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  background-color: var(--color-background);
  color: oklch(.145 0 0);

  /* ■■■■■■■■■■ */
  min-height: 100vh;
  width: 100%;
  overflow-x: hidden;
}

/* ■■■■■■ # ■■■■■ React ■■■■■■ */
#root {
  width: 100%;
  min-height: 100vh;
}

h1 {
  font-size: 2rem;
  font-weight: 700;
  line-height: 1.2;
  color: var(--color-text-primary);
}

h2 {
  font-size: 1.5rem;
  font-weight: 600;
  line-height: 1.3;
  color: var(--color-text-primary);
}

h3 {
  font-size: 1.25rem;
  font-weight: 600;
  line-height: 1.4;
  color: var(--color-text-primary);
}

h4 {
  font-size: 1.125rem;
  font-weight: 600;
  line-height: 1.4;
  color: var(--color-text-primary);
}

p {
  font-size: 1rem;
  line-height: 1.6;
}

button {
  font-size: 1rem;
```

```
    font-weight: 500;
    line-height: 1.5;
}

input, textarea, select {
    font-size: 1rem;
    line-height: 1.5;
}

label {
    font-size: 0.875rem;
    font-weight: 500;
    line-height: 1.5;
    color: var(--color-text-primary);
}
```

■ File: src\supabase\functions\serverindex.tsx

```
import { Hono } from "npm:hono";
import { cors } from "npm:hono/cors";
import { logger } from "npm:hono/logger";
import * as kv from "./kv_store.tsx";
const app = new Hono();

// Enable logger
app.use('*', logger(console.log));

// Enable CORS for all routes and methods
app.use(
  "/*",
  cors({
    origin: "*",
    allowHeaders: ["Content-Type", "Authorization"],
    allowMethods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    exposeHeaders: ["Content-Length"],
    maxAge: 600,
  })
);

// Health check endpoint
app.get("/make-server-826d6f62/health", (c) => {
  return c.json({ status: "ok" });
});

Deno.serve(app.fetch);
```

■ File: src\supabase\functions\server\kv_store.tsx

```
/* AUTOGENERATED FILE - DO NOT EDIT CONTENTS */

/* Table schema:
CREATE TABLE kv_store_826d6f62 (
  key TEXT NOT NULL PRIMARY KEY,
  value JSONB NOT NULL
);
*/

// View at https://supabase.com/dashboard/project/hkwvplgzliyvvrpoqpksh/database/tables

// This file provides a simple key-value interface for storing Figma Make data. It should
// be adequate for most small-scale use cases.
import { createClient } from "jsr:@supabase/supabase-js@2.49.8";

const client = () => createClient(
  Deno.env.get("SUPABASE_URL"),
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY"),
);

// Set stores a key-value pair in the database.
export const set = async (key: string, value: any): Promise<void> => {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").upsert({
    key,
    value
  });
  if (error) {
    throw new Error(error.message);
  }
};

// Get retrieves a key-value pair from the database.
export const get = async (key: string): Promise<any> => {
  const supabase = client()
  const { data, error } = await supabase.from("kv_store_826d6f62").select("value").eq("key", key).maybeSingle();
  if (error) {
    throw new Error(error.message);
  }
  return data?.value;
};

// Delete deletes a key-value pair from the database.
export const del = async (key: string): Promise<void> => {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").delete().eq("key", key);
  if (error) {
    throw new Error(error.message);
  }
};

// Sets multiple key-value pairs in the database.
export const mset = async (keys: string[], values: any[]): Promise<void> => {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").upsert(keys.map((k, i) => ({
    key: k, value: values[i]
  })));
  if (error) {
    throw new Error(error.message);
  }
};

// Gets multiple key-value pairs from the database.
export const mget = async (keys: string[]): Promise<any[]> => {
  const supabase = client()
  const { data, error } = await supabase.from("kv_store_826d6f62").select("value").in("key", keys);
  if (error) {
    throw new Error(error.message);
  }
  return data?.map((d) => d.value) ?? [];
};

// Deletes multiple key-value pairs from the database.
export const mdel = async (keys: string[]): Promise<void> => {
  const supabase = client()
  const { error } = await supabase.from("kv_store_826d6f62").delete().in("key", keys);
  if (error) {
    throw new Error(error.message);
  }
};
```

```

    }
  };

  // Search for key-value pairs by prefix.
  export const getByPrefix = async (prefix: string): Promise<any[]> => {
    const supabase = client()
    const { data, error } = await supabase.from("kv_store_826d6f62").select("key,
      value").like("key", prefix + "%");
    if (error) {
      throw new Error(error.message);
    }
    return data?.map((d) => d.value) ?? [];
  };

```

■ File: src\utils\supabase\info.tsx

```
/* AUTOGENERATED FILE - DO NOT EDIT CONTENTS */
```

```
export const projectId = "hkwpplgzliyvrrpqpqksh"
export const publicAnonKey = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSI1
```


■ File: srclvite.config.ts

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import path from 'path';
import tailwindcss from '@tailwindcss/vite'; // ■■■■ Tailwind ■■

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(), // ■■ Tailwind ■■
  ],
  resolve: {
    extensions: ['.js', '.jsx', '.ts', '.tsx', '.json'],
    alias: {
      'vaul@1.1.2': 'vaul',
      'sonner@2.0.3': 'sonner',
      'recharts@2.15.2': 'recharts',
      'react-resizable-panels@2.1.7': 'react-resizable-panels',
      'react-hook-form@7.55.0': 'react-hook-form',
      'react-day-picker@8.10.1': 'react-day-picker',
      'next-themes@0.4.6': 'next-themes',
      'lucide-react@0.487.0': 'lucide-react',
      'input-otp@1.4.2': 'input-otp',
      'embla-carousel-react@8.6.0': 'embla-carousel-react',
      'cmdk@1.1.1': 'cmdk',
      'class-variance-authority@0.7.1': 'class-variance-authority',
      '@radix-ui/react-tooltip@1.1.8': '@radix-ui/react-tooltip',
      '@radix-ui/react-toggle@1.1.2': '@radix-ui/react-toggle',
      '@radix-ui/react-toggle-group@1.1.2': '@radix-ui/react-toggle-group',
      '@radix-ui/react-tabs@1.1.3': '@radix-ui/react-tabs',
      '@radix-ui/react-switch@1.1.3': '@radix-ui/react-switch',
      '@radix-ui/react-slot@1.1.2': '@radix-ui/react-slot',
      '@radix-ui/react-slider@1.2.3': '@radix-ui/react-slider',
      '@radix-ui/react-separator@1.1.2': '@radix-ui/react-separator',
      '@radix-ui/react-select@2.1.6': '@radix-ui/react-select',
      '@radix-ui/react-scroll-area@1.2.3': '@radix-ui/react-scroll-area',
      '@radix-ui/react-radio-group@1.2.3': '@radix-ui/react-radio-group',
      '@radix-ui/react-progress@1.1.2': '@radix-ui/react-progress',
      '@radix-ui/react-popover@1.1.6': '@radix-ui/react-popover',
      '@radix-ui/react-navigation-menu@1.2.5': '@radix-ui/react-navigation-menu',
      '@radix-ui/react-menubar@1.1.6': '@radix-ui/react-menubar',
      '@radix-ui/react-label@2.1.2': '@radix-ui/react-label',
      '@radix-ui/react-hover-card@1.1.6': '@radix-ui/react-hover-card',
      '@radix-ui/react-dropdown-menu@2.1.6': '@radix-ui/react-dropdown-menu',
      '@radix-ui/react-dialog@1.1.6': '@radix-ui/react-dialog',
      '@radix-ui/react-context-menu@2.2.6': '@radix-ui/react-context-menu',
      '@radix-ui/react-collapsible@1.1.3': '@radix-ui/react-collapsible',
      '@radix-ui/react-checkbox@1.1.4': '@radix-ui/react-checkbox',
      '@radix-ui/react-avatar@1.1.3': '@radix-ui/react-avatar',
      '@radix-ui/react-aspect-ratio@1.1.2': '@radix-ui/react-aspect-ratio',
      '@radix-ui/react-alert-dialog@1.1.6': '@radix-ui/react-alert-dialog',
      '@radix-ui/react-accordion@1.2.3': '@radix-ui/react-accordion',
      '@': path.resolve(__dirname, './src'),
    },
  },
  build: {
    target: 'esnext',
    outDir: 'dist', // ■■■■■■■■■■ 'dist'
  },
  server: {
    port: 3000,
    open: true,
  },
});
```

■ File: thomas-backend/build.gradle

```
plugins {  
    ■ id 'java'  
    ■ id 'org.springframework.boot' version '4.0.5'  
    ■ id 'io.spring.dependency-management' version '1.1.7'  
}  
  
group = 'com.thomas'  
version = '0.0.1-SNAPSHOT'  
  
java {  
    ■ toolchain {  
        ■ languageVersion = JavaLanguageVersion.of(21)  
    }  
}  
  
repositories {  
    ■ mavenCentral()  
}  
  
dependencies {  
    ■ implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
    ■ implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
    ■ compileOnly 'org.projectlombok:lombok'  
    ■ runtimeOnly 'org.postgresql:postgresql'  
    ■ annotationProcessor 'org.projectlombok:lombok'  
    ■ testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'  
    ■ testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    ■ testCompileOnly 'org.projectlombok:lombok'  
    ■ testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
    ■ testAnnotationProcessor 'org.projectlombok:lombok'  
}  
  
tasks.named('test') {  
    ■ useJUnitPlatform()  
}
```

■ File: thomas-backend\gradle\wrapper\gradle-wrapper.properties

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-9.4.1-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

■ File: thomas-backend\settings.gradle

```
rootProject.name = 'backend'
```

■ File: thomas-backend\src\main\java\com\thomas\backend\BackendApplication.java

```
package com.thomas.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BackendApplication {

    ■public static void main(String[] args) {
    ■■SpringApplication.run(BackendApplication.class, args);
    ■}

}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\AuthController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.repository.UserRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;
import java.util.Optional;

@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "**")
public class AuthController {

    private final UserRepository userRepository;

    public AuthController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // A helper route to init some default users if needed for testing
    @GetMapping("/init")
    public ResponseEntity<?> initUsers() {
        if (userRepository.count() == 0) {
            User patient = new User();
            patient.setName("patient1");
            patient.setPassword("1234");
            patient.setRole("patient");
            userRepository.save(patient);

            User doctor = new User();
            doctor.setName("doctor1");
            doctor.setPassword("1234");
            doctor.setRole("doctor");
            userRepository.save(doctor);

            return ResponseEntity.ok("Dummy initialized");
        }
        return ResponseEntity.ok("Already initialized");
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody User user) {
        if (user.getEmail() != null && !user.getEmail().trim().isEmpty()) {
            if (userRepository.findByEmail(user.getEmail().toLowerCase().trim()).isPresent()) {
                return ResponseEntity.status(400).body(Map.of("message", "■ Email ■■■■"));
            }
        } else {
            if (userRepository.findByName(user.getName()).isPresent()) {
                return ResponseEntity.status(400).body(Map.of("message", "■■■■■■■■"));
            }
        }
        User saved = userRepository.save(user);
        return ResponseEntity.ok(saved);
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> creds) {
        String nameOrEmail = creds.get("name");
        String password = creds.get("password");

        Optional<User> userOpt = Optional.empty();
        if (nameOrEmail != null && nameOrEmail.contains("@")) {
            userOpt = userRepository.findByEmail(nameOrEmail.toLowerCase().trim());
        }
        if (userOpt.isEmpty() && nameOrEmail != null) {
            userOpt = userRepository.findByName(nameOrEmail.trim());
        }

        if (userOpt.isPresent() && userOpt.get().getPassword().equals(password)) {
            return ResponseEntity.ok(userOpt.get());
        }
        return ResponseEntity.status(401).body(Map.of("message", "■■■■■■■■"));
    }
}
```

■ File:

thomas-backend\src\main\java\com\thomas\backend\controller\CaseManagerController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.model.PatientTracking;
import com.thomas.backend.repository.UserRepository;
import com.thomas.backend.repository.DiaryEntryRepository;
import com.thomas.backend.repository.PatientTrackingRepository;
import com.thomas.backend.repository.QuestionnaireResponseRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/casemanager")
@CrossOrigin(origins = "**")
public class CaseManagerController {

    private final UserRepository userRepository;
    private final DiaryEntryRepository diaryRepository;
    private final PatientTrackingRepository trackingRepository;
    private final QuestionnaireResponseRepository questionnaireResponseRepository;

    public CaseManagerController(UserRepository userRepository, DiaryEntryRepository
diaryRepository, PatientTrackingRepository trackingRepository,
QuestionnaireResponseRepository questionnaireResponseRepository) {
        this.userRepository = userRepository;
        this.diaryRepository = diaryRepository;
        this.trackingRepository = trackingRepository;
        this.questionnaireResponseRepository = questionnaireResponseRepository;
    }

    @GetMapping("/patients")
    public ResponseEntity<List<Map<String, Object>>> getHighRiskPatients() {
        List<User> patients = userRepository.findAll().stream()
            .filter(u -> "patient".equals(u.getRole()))
            .collect(Collectors.toList());

        List<Map<String, Object>> response = patients.stream().map(p -> {
            long headacheDays = diaryRepository.findById(p.getId()).size();

            PatientTracking tracking = trackingRepository.findById(p.getId()).orElseGet(()
-> {
                PatientTracking t = new PatientTracking();
                t.setPatientId(p.getId());
                t.setStatus("pending");
                t.setNotes("");
                return trackingRepository.save(t);
            });

            // Get actual latest MIDAS score
            Optional<com.thomas.backend.model.QuestionnaireResponse> latestMidas =
questionnaireResponseRepository
                .findTopByPatientIdAndQuestionnaireIdOrderByCompletedAtDesc(p.getId(),
"midas");
            int midasScore = latestMidas.map(com.thomas.backend.model.QuestionnaireRespons
e::getScore).orElse(0);

            // Dynamic risk factors based on actual data
            List<String> riskFactors = new ArrayList<>();
            if (headacheDays >= 10) {
                riskFactors.add("■■■■■■■");
            }
            if (midasScore > 20) {
                riskFactors.add("MIDAS Grade IV");
            } else if (midasScore > 10) {
                riskFactors.add("MIDAS Grade III");
            }

            // Check BDI (beck depression)
            Optional<com.thomas.backend.model.QuestionnaireResponse> bdi =
questionnaireResponseRepository
                .findTopByPatientIdAndQuestionnaireIdOrderByCompletedAtDesc(p.getId(),
"bdi");
            if (bdi.isPresent() && bdi.get().getScore() > 19) {
                riskFactors.add("■■■■■■■");
            }
        });

        return ResponseEntity.ok(response);
    }
}
```

```

    }

    // Check PSQI (sleep quality)
    Optional<com.thomas.backend.model.QuestionnaireResponse> psqi =
questionnaireResponseRepository
        .findTopByPatientIdAndQuestionnaireIdOrderByCompletedAtDesc(p.getId(),
"psqi");
    if (psqi.isPresent() && psqi.get().getScore() > 5) {
        riskFactors.add("■■■■■■");
    }

    // Check PSS (stress scale)
    Optional<com.thomas.backend.model.QuestionnaireResponse> pss =
questionnaireResponseRepository
        .findTopByPatientIdAndQuestionnaireIdOrderByCompletedAtDesc(p.getId(),
"pss");
    if (pss.isPresent() && pss.get().getScore() > 13) {
        riskFactors.add("■■■■■■");
    }

    Map<String, Object> map = new HashMap<>();
    map.put("id", p.getId());
    map.put("name", p.getName());
    map.put("age", p.getAge() != null ? p.getAge() : 42);
    map.put("phone", p.getPhone() != null ? p.getPhone() : "■■■■");
    map.put("email", p.getEmail() != null ? p.getEmail() : (p.getName() +
"@test.com"));
    map.put("midasScore", midasScore);
    map.put("headacheDays", headacheDays);
    map.put("lastVisit", "■■");
    map.put("nextAppointment", tracking.getNextAppointment());
    map.put("riskFactors", riskFactors);
    map.put("status", tracking.getStatus() != null ? tracking.getStatus() :
"pending");
    map.put("notes", tracking.getNotes() == null ? "" : tracking.getNotes());

    return map;
}).collect(Collectors.toList());

return ResponseEntity.ok(response);
}

@PutMapping("/tracking/{patientId}")
public ResponseEntity<?> updateTracking(@PathVariable String patientId, @RequestBody
Map<String, String> payload) {
    PatientTracking t = trackingRepository.findById(patientId).orElseGet(PatientTracki
ng::new);
    t.setPatientId(patientId);

    if (payload.containsKey("status")) {
        t.setStatus(payload.get("status"));
    }
    if (payload.containsKey("nextAppointment")) {
        t.setNextAppointment(payload.get("nextAppointment"));
    }
    if (payload.containsKey("notes")) {
        t.setNotes(payload.get("notes"));
    }

    trackingRepository.save(t);
    return ResponseEntity.ok(Map.of("message", "Updated successfully"));
}
}

```


■ File: thomas-backend\src\main\java\com\thomas\backend\controller\DiaryController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.DiaryEntry;
import com.thomas.backend.repository.DiaryEntryRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/diaries")
@CrossOrigin(origins = "")
public class DiaryController {

    private final DiaryEntryRepository repository;

    public DiaryController(DiaryEntryRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/user/{userId}")
    public ResponseEntity<List<DiaryEntry>> getByUserId(@PathVariable String userId) {
        return ResponseEntity.ok(repository.findByIdOrderByDateDesc(userId));
    }

    @PostMapping
    public ResponseEntity<DiaryEntry> save(@RequestBody DiaryEntry entry) {
        return ResponseEntity.ok(repository.save(entry));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> delete(@PathVariable String id) {
        repository.deleteById(id);
        return ResponseEntity.ok().build();
    }
}
```

■ File:

thomas-backend\src\main\java\com\thomas\backend\controller\PrescriptionController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.Prescription;
import com.thomas.backend.repository.PrescriptionRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/prescriptions")
@CrossOrigin(origins = "**")
public class PrescriptionController {

    private final PrescriptionRepository repository;

    public PrescriptionController(PrescriptionRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/patient/{patientId}")
    public ResponseEntity<List<Prescription>> getByPatientId(@PathVariable String
        patientId) {
        return ResponseEntity.ok(repository.findByPatientIdOrderByCreatedAtDesc(patientId)
    );
    }

    @PostMapping
    public ResponseEntity<Prescription> save(@RequestBody Prescription prescription) {
        return ResponseEntity.ok(repository.save(prescription));
    }
}
```

■ File:

thomas-backend\src\main\java\com\thomas\backend\controller\QuestionnaireController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.QuestionnaireResponse;
import com.thomas.backend.repository.QuestionnaireResponseRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/questionnaires")
@CrossOrigin(origins = "")
public class QuestionnaireController {

    private final QuestionnaireResponseRepository repository;

    public QuestionnaireController(QuestionnaireResponseRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/patient/{patientId}")
    public ResponseEntity<List<QuestionnaireResponse>> getByPatientId(@PathVariable
        String patientId) {
        return ResponseEntity.ok(repository.findByPatientId(patientId));
    }

    @PostMapping
    public ResponseEntity<QuestionnaireResponse> save(@RequestBody QuestionnaireResponse
        response) {
        return ResponseEntity.ok(repository.save(response));
    }
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\controller\UserController.java

```
package com.thomas.backend.controller;

import com.thomas.backend.model.User;
import com.thomas.backend.repository.UserRepository;
import com.thomas.backend.repository.DiaryEntryRepository;
import com.thomas.backend.repository.QuestionnaireResponseRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "**")
public class UserController {

    private final UserRepository userRepository;
    private final DiaryEntryRepository diaryRepository;
    private final QuestionnaireResponseRepository questionnaireResponseRepository;

    public UserController(UserRepository userRepository, DiaryEntryRepository
        diaryRepository, QuestionnaireResponseRepository questionnaireResponseRepository) {
        this.userRepository = userRepository;
        this.diaryRepository = diaryRepository;
        this.questionnaireResponseRepository = questionnaireResponseRepository;
    }

    @GetMapping("/patients")
    public ResponseEntity<List<Map<String, Object>>> getPatients() {
        List<User> patients = userRepository.findAll().stream()
            .filter(u -> "patient".equals(u.getRole()))
            .collect(Collectors.toList());

        List<Map<String, Object>> response = patients.stream().map(p -> {
            Map<String, Object> map = new HashMap<>();
            map.put("id", p.getId());
            map.put("name", p.getName());
            map.put("age", p.getAge() != null ? p.getAge() : 40);
            map.put("gender", p.getGender() != null ? p.getGender() : "■■■■");
            map.put("lastVisit", "■■■");

            // Get actual latest MIDAS score
            Optional<com.thomas.backend.model.QuestionnaireResponse> latestMidas =
                questionnaireResponseRepository
                    .findTopByPatientIdAndQuestionnaireIdOrderByCompletedAtDesc(p.getId(),
                        "midas");
            int midasScore = latestMidas.map(com.thomas.backend.model.QuestionnaireResponse
                ::getScore).orElse(0);
            map.put("midasScore", midasScore);

            long headacheDays = diaryRepository.findByUserIdOrderByDateDesc(p.getId()).size();
            map.put("headacheDays", headacheDays);

            map.put("riskLevel", headacheDays >= 10 ? "high" : headacheDays >= 5 ?
                "medium" : "low");

            // Questionnaire completion count this month
            List<com.thomas.backend.model.QuestionnaireResponse> patientResponses =
                questionnaireResponseRepository.findByPatientId(p.getId());
            java.time.LocalDateTime thirtyDaysAgo =
                java.time.LocalDateTime.now().minusDays(30);
            long completedCount = patientResponses.stream()
                .filter(r -> r.getCompletedAt() != null &&
                    r.getCompletedAt().isAfter(thirtyDaysAgo))
                .map(com.thomas.backend.model.QuestionnaireResponse::getQuestionnaireId)
                .distinct()
                .count();
            map.put("completedCount", completedCount);

            return map;
        }).collect(Collectors.toList());

        return ResponseEntity.ok(response);
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> updateUser(@PathVariable String id, @RequestBody Map<String,
        Object> updates) {
    }
```

```

Optional<User> userOpt = userRepository.findById(id);
if (userOpt.isPresent()) {
    User user = userOpt.get();
    if (updates.containsKey("age")) {
        Object val = updates.get("age");
        if (val instanceof Integer) {
            user.setAge((Integer) val);
        } else if (val instanceof String) {
            try {
                user.setAge(Integer.parseInt((String) val));
            } catch (Exception e) {}
        }
    }
    if (updates.containsKey("gender")) {
        user.setGender((String) updates.get("gender"));
    }
    if (updates.containsKey("phone")) {
        user.setPhone((String) updates.get("phone"));
    }
    User saved = userRepository.save(user);
    return ResponseEntity.ok(saved);
}
return ResponseEntity.notFound().build();
}
}

```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\DiaryEntry.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.util.List;

@Entity
@Data
@Table(name = "diary_entries")
public class DiaryEntry {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String userId; // foreign key mapping to the User

    private String date;
    private String time;
    private Integer intensity;

    @ElementCollection
    private List<String> symptoms;

    private String medication;
    private String notes;
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\PatientTracking.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "patient_tracking")
public class PatientTracking {
    @Id
    private String patientId;

    private String status = "pending";
    private String nextAppointment;

    @Column(columnDefinition = "TEXT")
    private String notes;

    private LocalDateTime updatedAt = LocalDateTime.now();
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\Prescription.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "prescriptions")
public class Prescription {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String doctorId;
    private String patientId;

    @Column(columnDefinition = "TEXT")
    private String medicationDetails;

    @Column(columnDefinition = "TEXT")
    private String instructions;

    private LocalDateTime createdAt = LocalDateTime.now();
}
```


■ File:

thomas-backend\src\main\java\com\thomas\backend\model\QuestionnaireResponse.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Entity
@Data
@Table(name = "questionnaire_responses")
public class QuestionnaireResponse {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String patientId;
    private String questionnaireId; // e.g. "midas"
    private Integer score;

    @Column(columnDefinition = "TEXT")
    private String details;

    private LocalDateTime completedAt = LocalDateTime.now();
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\model\User.java

```
package com.thomas.backend.model;

import jakarta.persistence.*;
import lombok.Data;

@Entity
@Data
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    private String name;

    private String email;

    private String password;

    private String role; // 'patient', 'doctor', 'caseManager'

    private Integer age;
    private String gender;
    private String phone;
}
```

■ File:

thomas-backend\src\main\java\com\thomas\backend\repository\DiaryEntryRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.DiaryEntry;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DiaryEntryRepository extends JpaRepository<DiaryEntry, String> {
    List<DiaryEntry> findByUserIdOrderByDateDesc(String userId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\PatientTrackingRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.PatientTracking;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PatientTrackingRepository extends JpaRepository<PatientTracking, String>
{
}
```

■ File:

thomas-backend\src\main\java\com\thomas\backend\repository\PrescriptionRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.Prescription;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface PrescriptionRepository extends JpaRepository<Prescription, String> {
    List<Prescription> findByPatientIdOrderByCreatedAtDesc(String patientId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\QuestionnaireResponseRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.QuestionnaireResponse;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;
import java.util.Optional;

public interface QuestionnaireResponseRepository extends
    JpaRepository<QuestionnaireResponse, String> {
    List<QuestionnaireResponse> findByPatientId(String patientId);
    Optional<QuestionnaireResponse> findTopByPatientIdAndQuestionnaireIdOrderByCompletedAt
        Desc(String patientId, String questionnaireId);
}
```

■ File: thomas-backend\src\main\java\com\thomas\backend\repository\UserRepository.java

```
package com.thomas.backend.repository;

import com.thomas.backend.model.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, String> {
    Optional<User> findByName(String name);
    Optional<User> findByEmail(String email);
}
```

■ File: thomas-backend\src\main\resources\application.properties

```
spring.application.name=thomas-backend

# Database Configuration
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:postgresql://localhost:5432/thomas_db}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:postgres}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:postgres}
spring.datasource.driver-class-name=org.postgresql.Driver

# Hibernate ORM Settings
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Server Configuration
server.port=8080
```


■ File: thomas-backend\src\test\java\com\thomas\backend\BackendApplicationTests.java

```
package com.thomas.backend;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class BackendApplicationTests {

    ■@Test
    ■void contextLoads() {
    ■}

}
```

■ File: vite.config.ts

```
// Thomas-main/vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import path from 'path';
import tailwindcss from '@tailwindcss/vite'; // ■■■■ Tailwind ■■

export default defineConfig({
  plugins: [
    react(),
    tailwindcss(), // ■■ Tailwind ■■
  ],
  resolve: {
    extensions: ['.js', '.jsx', '.ts', '.tsx', '.json'],
    alias: {
      'vaul@1.1.2': 'vaul',
      'sonner@2.0.3': 'sonner',
      'recharts@2.15.2': 'recharts',
      'react-resizable-panels@2.1.7': 'react-resizable-panels',
      'react-hook-form@7.55.0': 'react-hook-form',
      'react-day-picker@8.10.1': 'react-day-picker',
      'next-themes@0.4.6': 'next-themes',
      'lucide-react@0.487.0': 'lucide-react',
      'input-otp@1.4.2': 'input-otp',
      'embla-carousel-react@8.6.0': 'embla-carousel-react',
      'cmdk@1.1.1': 'cmdk',
      'class-variance-authority@0.7.1': 'class-variance-authority',
      '@radix-ui/react-tooltip@1.1.8': '@radix-ui/react-tooltip',
      '@radix-ui/react-toggle@1.1.2': '@radix-ui/react-toggle',
      '@radix-ui/react-toggle-group@1.1.2': '@radix-ui/react-toggle-group',
      '@radix-ui/react-tabs@1.1.3': '@radix-ui/react-tabs',
      '@radix-ui/react-switch@1.1.3': '@radix-ui/react-switch',
      '@radix-ui/react-slot@1.1.2': '@radix-ui/react-slot',
      '@radix-ui/react-slider@1.2.3': '@radix-ui/react-slider',
      '@radix-ui/react-separator@1.1.2': '@radix-ui/react-separator',
      '@radix-ui/react-select@2.1.6': '@radix-ui/react-select',
      '@radix-ui/react-scroll-area@1.2.3': '@radix-ui/react-scroll-area',
      '@radix-ui/react-radio-group@1.2.3': '@radix-ui/react-radio-group',
      '@radix-ui/react-progress@1.1.2': '@radix-ui/react-progress',
      '@radix-ui/react-popover@1.1.6': '@radix-ui/react-popover',
      '@radix-ui/react-navigation-menu@1.2.5': '@radix-ui/react-navigation-menu',
      '@radix-ui/react-menubar@1.1.6': '@radix-ui/react-menubar',
      '@radix-ui/react-label@2.1.2': '@radix-ui/react-label',
      '@radix-ui/react-hover-card@1.1.6': '@radix-ui/react-hover-card',
      '@radix-ui/react-dropdown-menu@2.1.6': '@radix-ui/react-dropdown-menu',
      '@radix-ui/react-dialog@1.1.6': '@radix-ui/react-dialog',
      '@radix-ui/react-context-menu@2.2.6': '@radix-ui/react-context-menu',
      '@radix-ui/react-collapsible@1.1.3': '@radix-ui/react-collapsible',
      '@radix-ui/react-checkbox@1.1.4': '@radix-ui/react-checkbox',
      '@radix-ui/react-avatar@1.1.3': '@radix-ui/react-avatar',
      '@radix-ui/react-aspect-ratio@1.1.2': '@radix-ui/react-aspect-ratio',
      '@radix-ui/react-alert-dialog@1.1.6': '@radix-ui/react-alert-dialog',
      '@radix-ui/react-accordion@1.2.3': '@radix-ui/react-accordion',
      '@jsr/supabase__supabase-js@2.49.8': '@jsr/supabase__supabase-js',
      '@': path.resolve(__dirname, './src'),
    },
  },
  build: {
    target: 'esnext',
    outDir: 'dist', // ■■: ■■■■■■■■ 'dist'
  },
  server: {
    port: 3000,
    open: true,
  },
});
```